

Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with LaTeX using the `book` class.
Figures produced with TikZ; code listings with `listings`.

Contact: `jgarcia@strathmore.edu` Version 0.1 — October 19, 2026

Contents

Preface **xv**

I Foundations and Classical Machine Learning **1**

1 Introduction to Machine Learning **3**
1.1 Why This Chapter Comes First 4
1.2 What Is Machine Learning? 5
1.2.1 An operational definition 5
1.2.2 ML inverts the direction of programming 6
1.2.3 When ML earns its keep — and when it does not . . . 7
1.2.4 The formal core: empirical risk minimization 7
1.2.5 The Bayes-optimal reference 10
1.2.6 Notation for the rest of the book 11
1.3 The Machine-Learning Landscape 11
1.3.1 The four learning paradigms 11
1.3.2 Two orthogonal operational axes 13
1.3.3 No free lunch, and what it licenses 15
1.4 Why ML Systems Fail in Production 16
1.4.1 The small green box 16
1.4.2 The notebook-to-production gap, localized 17
1.5 The End-to-End ML Lifecycle 18
1.5.1 From CRISP-DM to a monitoring-driven loop 18
1.5.2 It is a loop, not a waterfall 19
1.6 Framing the Problem and Choosing the Metric 20
1.6.1 Step 1 is where projects are won 20
1.6.2 Regression metrics: how the business pays for errors 21
1.6.3 Classification under imbalance: why accuracy lies . . 21
1.7 A First Look at the Data 24
1.7.1 The raw material 24
1.7.2 A feature dictionary, declared up front 24
1.7.3 What to interrogate, and why 25
1.8 Data, Splits, and Leakage 27

1.8.1 Splits: the discipline that protects your estimate of $R(f)$ 27
1.8.2 Data leakage: the silent killer of fintech models 28
1.9 Baselines, Evaluation, and Generalization 30
1.9.1 Why generalization is hard: the bias–variance decomposition 30
1.9.2 Always ship a baseline first 31
1.9.3 Cross-validation: cheap insurance on the estimate . . 31
1.10 Presenting and Deciding 33
1.10.1 The decision memo 33
1.10.2 Champion, challenger, and shadow mode 33
1.11 Fairness, Explainability, and Regulation 34
1.11.1 Making "fair" precise enough to test 34
1.11.2 Explainability as a deployment requirement 35
1.12 From Model to System: MLOps and the Agentic Outlook . . 35
1.12.1 The MLOps spine of this course 36
1.12.2 Where LLMs and agents fit — and where they do not 37
1.13 Chapter Summary 38
Exercises 41
Key Terms 42
Further Reading 42

2 Data Preparation and Feature Engineering 45
2.1 Introduction: From Raw Data to Model-Ready Data 46
2.1.1 The Six Dimensions of Data Quality 47
2.1.2 The Unit of Prediction 48
2.2 Missing Data 48
2.2.1 Rubin's Taxonomy of Missingness 49
2.2.2 Diagnosis Before Treatment 50
2.2.3 Imputation I: Simple Statistics with Indicators 50
2.2.4 Imputation II: Model-Based Methods 51
2.2.5 A Decision Framework 52
2.3 Outliers and Duplicates 53
2.3.1 Detecting Outliers 53
2.3.2 Treating Outliers, and When Not To 54
2.3.3 Duplicates and Entity Resolution 54
2.4 Scaling and Transformations 55
2.4.1 Why Scale, and With What 55
2.4.2 Taming Skew: Log and Power Transforms 56
2.5 Encoding Categorical Variables 57
2.5.1 Low Cardinality: One-Hot and Ordinal 57
2.5.2 High Cardinality I: Frequency Encoding and the Hashing Trick 58
2.5.3 High Cardinality II: Target Encoding 58
2.6 Feature Engineering 59

2.6.1 Engineered versus Learned Features 59
2.6.2 Domain Features for Mobile-Money Credit Scoring . 60
2.6.3 Datetime Features and Cyclical Encoding 61
2.6.4 Interactions, Binning, and Crosses 61
2.6.5 What Makes a Feature Good? 62
2.7 Data Leakage 63
2.7.1 Definition and Why It Is Dangerous 63
2.7.2 Six Common Causes 64
2.7.3 Temporal and Group-Aware Splits 64
2.7.4 A Detection Protocol 65
2.7.5 The Structural Antidote: Pipelines 66
2.8 From Pipelines to Feature Platforms 67
Assignment 2 (graded, 20 marks) 68
Exercises 69
Key Terms 70
Further Reading 70

3 Classical Machine Learning and Rigorous Evaluation 71
3.1 Introduction 72
3.2 The Classical Toolbox 73
3.2.1 Why classical models still run production tabular ML 73
3.2.2 Logistic regression 73
3.2.3 Softmax regression for multiclass problems 75
3.2.4 Maximum-margin classification: support vector machines 75
3.2.5 Neighbours and Bayes: two more baselines 76
3.2.6 Choosing among the classical learners 77
3.3 Beyond Accuracy: Metrics that Match the Problem 78
3.3.1 The accuracy trap 78
3.3.2 The confusion matrix 78
3.3.3 Precision, recall, and the F_β family 79
3.3.4 From metrics to money: cost-sensitive evaluation . . 79
3.4 Ranking Quality: ROC and Precision–Recall Analysis 80
3.4.1 Scores, not labels: the threshold as a free parameter . 80
3.4.2 The ROC curve and AUC 81
3.4.3 Precision–recall curves: the imbalance-honest view . 82
3.5 Calibration: Do the Probabilities Mean Anything? 82
3.6 Cross-Validation and Honest Uncertainty 84
3.6.1 Why one split is not enough 84
3.6.2 Cross-validating the pipeline 85
3.6.3 Nested cross-validation for model selection 86
3.6.4 Comparing models honestly 86
3.7 Learning from Imbalanced Data 87
3.7.1 Three families of remedies 87

3.7.2 SMOTE: synthesising minority examples 88
3.7.3 The cardinal sin: resampling before splitting 89
3.8 Case Study: Evaluation Design for the Thin-File Credit Scorer 90
3.9 Chapter Summary 91

4 Decision Trees, Ensembles and Hyperparameter Tuning 95
4.1 Introduction 96
4.2 Decision Trees: The Building Block 98
4.2.1 A tree is a sequence of questions 98
4.2.2 The same object, drawn as a partition 98
4.2.3 How a tree is grown 99
4.2.4 Controlling complexity: stopping and pruning 100
4.2.5 The flaw that motivates the rest of the chapter 101
4.3 Why Ensembles Work 102
4.3.1 The wisdom of independent crowds 102
4.3.2 The variance of an average 103
4.4 Voting Classifiers 104
4.5 Bagging and Random Forests 106
4.5.1 Bootstrap aggregating 106
4.5.2 Out-of-bag evaluation 106
4.5.3 Random Forests: decorrelating the trees 107
4.6 Boosting 109
4.6.1 The idea, before the algebra 109
4.6.2 A worked trace 110
4.6.3 From residuals to gradients 111
4.6.4 AdaBoost, the historical special case 112
4.6.5 Shrinkage and early stopping 113
4.7 Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost 113
4.7.1 Regularisation inside the objective 114
4.7.2 The library landscape 115
4.7.3 Monotonic constraints and regulated deployment . . 115
4.8 Stacked Generalisation 117
4.9 Hyperparameter Optimisation 118
4.9.1 What is a hyperparameter, and why is it different? . . 118
4.9.2 A ladder of strategies 119
4.9.3 Grid search versus random search 119
4.9.4 Learning from previous trials: Bayesian search and TPE 121
4.9.5 Spending less per trial: successive halving and Hyperband 122
4.9.6 Designing the search space 123
4.9.7 Honest evaluation of a tuned model 123
4.10 Case Study: Round Three of the Credit Scorer 125
Exercises 127
Key Terms 128

Further Reading . . . 129

5 Unsupervised Learning and Dimensionality Reduction 131
5.1 Introduction: Learning Without Labels . . . 132
5.2 The Unsupervised Landscape . . . 134
5.3 Clustering with K-Means . . . 136
5.3.1 The objective . . . 136
5.3.2 Choosing k . . . 137
5.3.3 Implementation . . . 139
5.4 Density-Based Clustering: DBSCAN . . . 140
5.5 Gaussian Mixtures and EM . . . 142
5.5.1 The model . . . 142
5.5.2 Fitting by Expectation–Maximisation . . . 143
5.5.3 Selecting k by information criteria . . . 145
5.5.4 Mixtures as anomaly detectors . . . 146
5.6 The Curse of Dimensionality . . . 147
5.7 Principal Component Analysis . . . 148
5.7.1 Variance maximisation . . . 148
5.7.2 Reconstruction-error minimisation . . . 149
5.7.3 Computation and the choice of d . . . 149
5.7.4 Variants you will actually need . . . 151
5.8 Case Study: The Segmentation Pipeline in Production . . . 152
5.8.1 Profiling: turning clusters into personas . . . 152
5.8.2 Stability: a segment that moves is not a segment . . . 153
Exercises . . . 156
Key Terms . . . 157
Further Reading . . . 158

II Deep Learning and Generative AI 159

6 Deep Learning Fundamentals 161
6.1 Introduction: from engineered features to learned representations . . . 162
6.1.1 When a network is the right instrument . . . 164
6.2 The artificial neuron and the multi-layer perceptron . . . 165
6.3 Activation functions . . . 168
6.4 The training objective . . . 170
6.4.1 Binary cross-entropy from the Bernoulli likelihood . . . 171
6.4.2 Choosing the output layer and loss together . . . 172
6.5 Gradient descent . . . 173
6.5.1 Why the learning rate dominates everything else . . . 174
6.6 Backpropagation . . . 174
6.7 Optimisers beyond plain gradient descent . . . 177

6.7.1 Momentum . . . 177
6.7.2 Adam . . . 178
6.7.3 Learning-rate schedules . . . 179
6.8 Pathologies of deep training . . . 179
6.9 Initialisation . . . 180
6.10 Normalisation layers . . . 181
6.11 Regularisation . . . 182
6.11.1 Dropout . . . 182
6.11.2 Early stopping and weight decay . . . 184
6.12 Input scaling and conditioning . . . 184
6.13 Entity embeddings . . . 185
6.14 Implementation in TensorFlow and Keras . . . 187
6.14.1 The training loop, written out . . . 187
6.14.2 The baseline network . . . 188
6.14.3 The embedding challenger . . . 189
6.14.4 Hyperparameter search and debugging . . . 190
6.15 Evaluation, calibration and governance . . . 190
6.15.1 Ranking versus calibration . . . 191
6.15.2 Explanation . . . 191
6.15.3 Fairness and segment-level performance . . . 192
Exercises . . . 194
Key Terms . . . 197
Further Reading . . . 198

7 Computer Vision with CNNs 199
7.1 Introduction . . . 200
7.1.1 Four tasks, one backbone . . . 202
7.2 Why Convolution? . . . 203
7.2.1 Why a dense network fails on images . . . 203
7.2.2 The convolution operator . . . 205
7.2.3 Pooling and global average pooling . . . 209
7.2.4 Receptive fields and the case for depth . . . 210
7.2.5 The data-efficiency argument . . . 212
7.3 Modern Architectures: ResNet and EfficientNet . . . 212
7.3.1 Residual networks . . . 213
7.3.2 EfficientNet and compound scaling . . . 216
7.3.3 Choosing a backbone . . . 218
7.4 Transfer Learning . . . 218
7.4.1 Two regimes . . . 219
7.4.2 The two-phase recipe . . . 221
7.4.3 Implementation in TensorFlow and Keras . . . 222
7.4.4 The batch-normalisation trap . . . 224
7.4.5 Data augmentation as a statement about your domain 225
7.4.6 Reading the training curves . . . 227

7.5 Evaluation, Explainability and Deployment 227
7.5.1 Evaluation under imbalance and asymmetric cost . . 227
7.5.2 Explaining a convolutional prediction 229
7.5.3 Fairness, governance and deployment 230
Exercises 234
Key Terms 236
Further Reading 236

8 Sequence Models and Forecasting **239**
8.1 Introduction: from images to sequences 240
8.2 The anatomy of a time series 242
8.2.1 The forecasting task, formally 243
8.2.2 Stationarity and autocorrelation 244
8.3 Evaluating a forecast without fooling yourself 245
8.3.1 Baselines that keep everyone honest 245
8.3.2 Backtesting, not random cross-validation 246
8.3.3 Metrics 247
8.3.4 The pinball loss, and why the business lives in the tail 247
8.4 Forecasting as supervised learning 248
8.4.1 Three ways to cover the horizon 249
8.5 Recurrent neural networks 251
8.5.1 Backpropagation through time, and why it fails . . . 251
8.6 Gated cells: LSTM and GRU 253
8.6.1 The LSTM cell 253
8.6.2 The GRU cell 255
8.6.3 A forecaster we can actually train 256
8.6.4 From a point to a fan 256
8.7 Attention 258
8.7.1 The bottleneck 259
8.7.2 Attention as soft dictionary lookup 259
8.7.3 Self-attention, and what queries, keys and values are 260
8.7.4 Causal masking 261
8.7.5 Multiple heads 261
8.7.6 Reading an attention map 262
8.8 The Temporal Fusion Transformer 263
8.8.1 Gated residual networks 264
8.8.2 Variable selection 264
8.8.3 Interpretable attention, and the rest of the model . . . 265
8.8.4 Choosing a model 267
8.9 The case study end to end 268
Exercises 270
Key Terms 273
Further Reading 274

9 Transformers, LLMs and RAG **275**
9.1 Introduction . . . 276
9.2 What a language model actually computes . . . 278
9.2.1 One job . . . 278
9.2.2 Tokens, not words . . . 279
9.2.3 Why one narrow objective produces broad capability 280
9.2.4 The same objective, many applications . . . 281
9.2.5 The problem this leaves us with . . . 281
9.3 Embeddings: words become vectors . . . 281
9.3.1 A lookup table of parameters . . . 282
9.3.2 Meaning lives in directions . . . 282
9.3.3 How the table gets trained . . . 283
9.3.4 The limitation that motivates attention . . . 284
9.4 Attention: adding context . . . 284
9.4.1 From a hard lookup to a soft one . . . 285
9.4.2 Queries, keys and values . . . 286
9.4.3 Scaled dot-product attention . . . 287
9.4.4 A worked example . . . 287
9.4.5 Why the scores are divided by the square root of d_k . 288
9.4.6 Causal masking . . . 289
9.4.7 Multi-head attention . . . 289
9.4.8 The price: cost grows with the square of the context . 290
9.5 The Transformer: the full machine . . . 291
9.5.1 Position, and the information attention discards . . . 291
9.5.2 One block, two sublayers . . . 292
9.5.3 What the MLP layers are for . . . 293
9.5.4 Residuals and LayerNorm . . . 293
9.5.5 The last layer: back to a probability over the vocabulary 294
9.5.6 Where the parameters actually are . . . 294
9.5.7 Three architectural families . . . 295
9.5.8 From base model to assistant . . . 296
9.6 Adapting a pretrained model . . . 296
9.6.1 The HuggingFace ecosystem . . . 296
9.6.2 Three ways to make a frozen model serve you . . . 298
9.6.3 Fine-tuning cheaply: LoRA and QLoRA . . . 299
9.6.4 Retrieval-augmented generation . . . 302
9.6.5 Evaluating a retrieval system . . . 304
9.7 Governance, risk and the Kenyan context . . . 305
9.8 Summary . . . 306
Exercises . . . 307
Key terms . . . 309
Further reading . . . 310

III MLOps and Production Systems **311**

10 MLOps: packaging, serving and CI/CD **313**
10.1 Introduction: from notebook to production system 314
10.1.1 The MLOps lifecycle and maturity levels 316
10.2 Packaging the model artefact 317
10.2.1 What exactly is "the model"? 318
10.2.2 Training–serving skew 318
10.2.3 Serialisation formats 320
10.2.4 The threshold belongs in the package 321
10.2.5 Metadata and environment 322
10.3 Serving the model: design and API 323
10.3.1 Four ways to serve predictions 323
10.3.2 Latency, capacity and model placement 324
10.3.3 The FastAPI service 327
10.3.4 Operating the service: health, errors and tests 331
10.3.5 Serving language-model and agent workloads 333
10.4 Tracking and the registry 334
10.4.1 Experiment tracking with MLflow 335
10.4.2 Tracing language-model agents 339
10.4.3 The model registry 341
10.5 Testing machine learning systems 343
10.5.1 Why ML testing is different 343
10.5.2 Unit tests and data contracts 344
10.5.3 Behavioural tests 344
10.5.4 Statistically honest gates 346
10.5.5 Gating language-model agents 348
10.6 Shipping, delivery and governance 348
10.6.1 Containerising with Docker 348
10.6.2 Continuous integration, delivery and training 352
10.6.3 The three workflows 353
10.6.4 Deployment strategies and the whole pipeline 355
10.6.5 Governance and auditability 356
10.7 Conclusions 357
Exercises 358
Key terms 360
Further reading 360

11 Monitoring, drift and system design **363**
11.1 Introduction: the model that quietly stopped working 364
11.1.1 Not every degradation is drift 367
11.2 A taxonomy of drift 367
11.2.1 Which part of the joint distribution moved? 367
11.2.2 Covariate shift 368

11.2.3 Label shift . . . 368
11.2.4 Concept drift . . . 370
11.2.5 How drift unfolds in time . . . 371
11.2.6 What can be seen without labels . . . 372
11.3 Detecting drift statistically . . . 373
11.3.1 Reference and current windows . . . 373
11.3.2 Drift detection as a two-sample test . . . 374
11.3.3 The Kolmogorov–Smirnov statistic . . . 374
11.3.4 Why p-values make bad alerts . . . 375
11.3.5 The Wasserstein distance . . . 375
11.3.6 The population stability index . . . 376
11.3.7 Categorical features and the out-of-vocabulary signal 378
11.3.8 Multivariate drift: the domain classifier . . . 378
11.3.9 Prediction drift . . . 380
11.3.10 Many features, many false alarms . . . 380
11.4 Drift detection with Evidently AI . . . 381
11.4.1 What the library does . . . 381
11.4.2 What the preset decides for you . . . 382
11.4.3 From a report to a gate . . . 383
11.5 Monitoring in production . . . 384
11.5.1 Four layers . . . 384
11.5.2 The delayed-label problem . . . 385
11.5.3 Estimating performance before the labels arrive . . . 386
11.5.4 The decision log . . . 387
11.5.5 Operational metrics and alerting . . . 388
11.5.6 Triage: from alert to action . . . 389
11.5.7 When to retrain . . . 390
11.5.8 Monitoring language-model agents . . . 391
11.6 ML system design principles . . . 391
11.6.1 Four properties of a good ML system . . . 391
11.6.2 Scaling out . . . 392
11.6.3 Graceful degradation . . . 393
11.6.4 Safe rollout under delayed labels . . . 394
11.6.5 Feedback loops and selective labels . . . 395
11.6.6 Designing for Kenyan infrastructure . . . 395
11.7 Responsible ML and governance . . . 396
11.7.1 Fairness can drift . . . 396
11.7.2 Governance as a by-product . . . 398
11.7.3 Model cards . . . 398
11.8 Summary . . . 399
Exercises . . . 400
Key terms . . . 403
Further reading . . . 403

Appendices **407**

A Capstone Project **407**

B Solutions to Exercises **409**
Chapter 1 409
Chapter 2 416
Chapter 3 421
Chapter 7 — Computer Vision with CNNs 446
Chapter 8 solutions 453
B.1 Chapter 9: Transformers, LLMs and retrieval 459
B.2 Chapter 10: MLOps, packaging, serving and CI/CD 464
B.3 Chapter 11: monitoring, drift and system design 468

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.

2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.

3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix A.

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	8
9	Transformers, LLMs & RAG	9
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is Géron (2022); Bishop and Bishop (2024) and Huyen (2022) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Part II

Deep Learning and Generative AI

Part III

MLOps and Production Systems

CHAPTER 11

Model Monitoring, Drift and ML System Design

Course week 12

Learning Objectives

After working through this chapter you will be able to:

- Explain why a deployed model degrades silently, and why a passing health check says nothing about whether decisions are still good.
- Classify an observed change as a data bug, covariate shift, label shift or concept drift, and choose the remedy that each one calls for.
- Detect distribution change with the Kolmogorov–Smirnov statistic, the Wasserstein distance, the population stability index, the Jensen–Shannon distance and a domain classifier, and say when each is appropriate.
- Explain why drift alerts must be based on effect sizes rather than on p-values, and why monitoring many features at once invites false alarms.
- Automate drift detection with Evidently AI, producing a report for people and an exit code for a pipeline.
- Design a production monitoring stack in four layers, and estimate model performance while the ground truth is still months away.
- Apply the design principles that make an ML system reliable, scalable, maintainable and adaptable, including graceful degradation and safe rollout under delayed labels.
- Monitor fairness across slices over time, and produce the governance artefacts that Kenyan regulation expects as a by-product of the pipeline.

11.1 Introduction: the model that quietly stopped working

Chapter 10 ended with a scoring service in production: packaged, containerised, tracked, gated and deployed by a pipeline that a risk officer approves. Every test passed on the day it shipped. This chapter is about the months that follow, and it begins with an uncomfortable observation. Of all the ways a machine learning system can fail, the most common one produces no error, no alert and no complaint from the engineering side. The service returns HTTP 200, the JSON is well formed, the latency is excellent, and the decisions are progressively worse.

Software engineering has trained us to expect failure to be loud (Fig-

ure 11.1). A crashed process restarts, a bad deployment turns dashboards red, a timeout pages the on-call engineer. Machine learning failure is quiet, because the model always produces a number. Nothing in the serving stack can tell that the number has stopped being right. The liveness and readiness probes of Chapter 10 prove that the service is *alive*; they cannot prove that it is *correct*.

Software failure: loud

HTTP 500, timeouts, crash loops
dashboards turn red at once
the on-call engineer is paged
diagnosed and fixed in hours

ML failure: silent

HTTP 200, valid JSON, fast
every health check passes
decisions are quietly worse
discovered when defaults arrive

Figure 11.1: The asymmetry that motivates this chapter. Only one of these two failures announces itself.

The underlying reason is an assumption that every supervised learning method in this book has quietly relied on. Training and validation data are drawn from one joint distribution, $P_{t_0}(X, Y)$, and the guarantees of empirical risk minimisation are guarantees about *that* distribution. Production data is drawn from $P_t(X, Y)$, the distribution of the world at the time the model is used, and the world does not hold still (Figure 11.2). Borrowers change, products change, competitors change, the weather and the tax code change. When P_t moves away from P_{t_0}, the model does not become invalid in any detectable computational sense; it becomes invalid in the only sense that matters, which is that its decisions cost more money and do more harm than they used to.

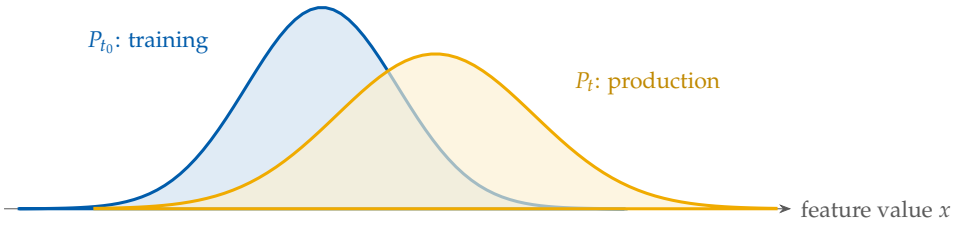


Figure 11.2: The assumption that breaks. Empirical risk minimisation guarantees nothing once P_t moves away from P_{t_0}.

Two consequences follow, and they organise the chapter. The first is that model quality is a function of time rather than a constant. Figure 11.3 shows the pattern that backtests of credit scorecards reproduce almost universally: a frozen model loses discrimination steadily, while a model retrained on a regular cadence recovers most of the loss each time. The second is that someone has to notice. Monitoring is the discipline of noticing early, and

system design is the discipline of building so that noticing is possible and acting on it is cheap.

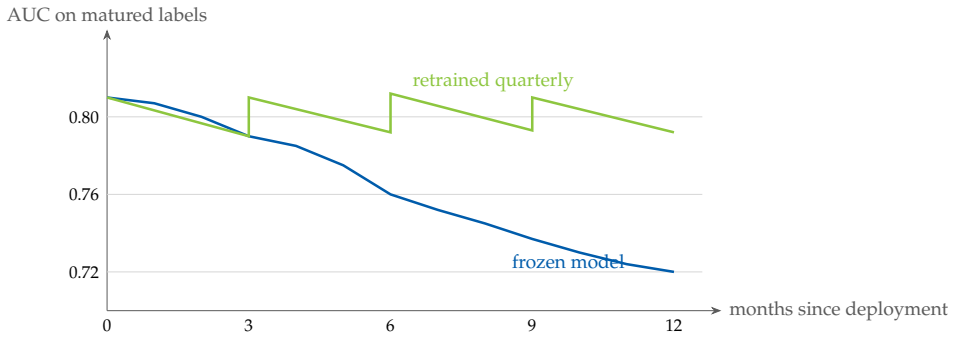


Figure 11.3: Model quality decays after deployment (illustrative). Retraining recovers it, but only if you know when to retrain.

Case Study

PesaCred, round eleven: the model is live PesaCred's thin-file scorer now serves USSD loan applications in under two seconds. It was packaged as a decision function, promoted as `@champion` in the MLflow registry, and deployed by the pipeline of Chapter 10. Its threshold, $t^\star \approx 0.074$, comes from the cost matrix of KES 10,000 per missed default against KES 800 per wrongly declined borrower, and its default rate at training time was about $\pi = 4\%$.

The operations team watches latency and error rates on a Grafana dashboard. Nobody watches the decisions. Three facts make that gap dangerous.

1. Whether a loan defaults is known only 30 to 90 days after the money is disbursed, so the model's true quality this week cannot be measured this week.
2. The service decides roughly 1,800 applications a day, so a problem that takes three months to surface will have shaped about 162,000 decisions by the time anyone can prove it exists.
3. The Central Bank's Digital Credit Providers Regulations and the Data Protection Act both hold PesaCred accountable for those decisions, not for the state of its dashboards.

This chapter builds the monitoring layer that closes the gap, and then revisits the architecture around it so that the answer to a drift alert is a routine procedure rather than an emergency.

11.1.1 Not every degradation is drift

Before reaching for statistics, it is worth being disciplined about diagnosis, because the word "drift" is used to explain far more failures than it causes. When predictions get worse, four families of cause are possible (Figure 11.4), and the cheapest explanations should be eliminated first (Huyen, 2022).

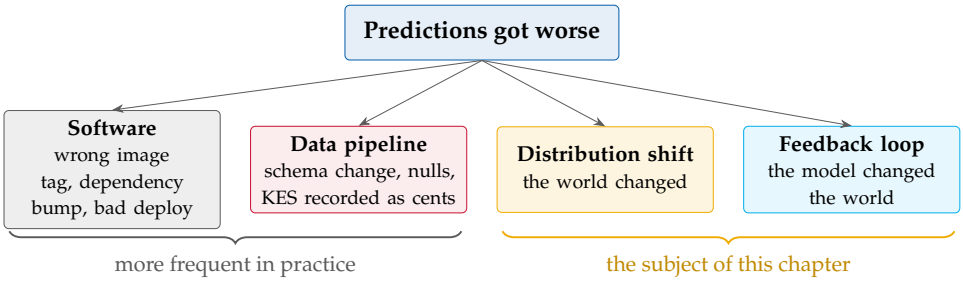


Figure 11.4: Four families of cause. Drift is the interesting one, not the common one.

Pitfall

Blaming drift for a bug An upstream team changes `avg_monthly_inflow_kes` from shillings to cents. Every statistical detector in this chapter will fire, spectacularly and correctly: the distribution has moved by two orders of magnitude. Retraining on the new data would "fix" the alert and permanently enshrine a unit error in the model. The first question after any drift alert is not "how do we retrain?" but "did the data pipeline change?". Section 11.5.6 makes that question the first node of a triage procedure.

11.2 A taxonomy of drift

11.2.1 Which part of the joint distribution moved?

Genuine distribution shift comes in three kinds, and telling them apart matters because each has a different detector, a different remedy and a different urgency. The distinction follows from writing the joint distribution in its two factorisations,

$$P(X, Y) = \underbrace{P(Y \mid X)}_{\text{the concept}} \underbrace{P(X)}_{\text{the inputs}} = \underbrace{P(X \mid Y)}_{\text{the class profiles}} \underbrace{P(Y)}_{\text{the prior}}, \tag{11.1}$$

and asking which factor has changed (Figure 11.5).

Definition 11.1 (Covariate shift, label shift, concept drift)**.** Let P_{t_0} be the training distribution and P_t the production distribution.

- *Covariate shift* (also called data drift): $P_t(X) \neq P_{t_0}(X)$ while $P_t(Y \mid X) = P_{t_0}(Y \mid X)$.
- *Label shift* (also called prior shift): $P_t(Y) \neq P_{t_0}(Y)$ while $P_t(X \mid Y) = P_{t_0}(X \mid Y)$.
- *Concept drift*: $P_t(Y \mid X) \neq P_{t_0}(Y \mid X)$, that is, the relationship the model learned has itself changed.

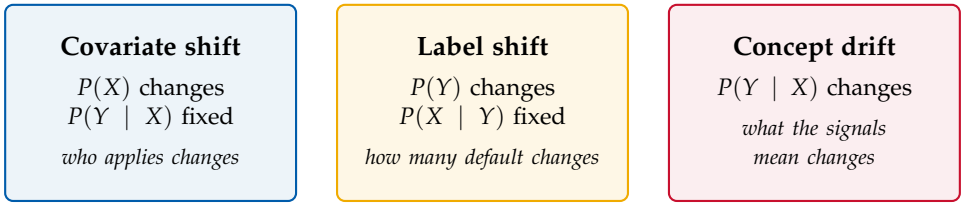


Figure 11.5: Three kinds of shift, read off the two factorisations of Equation (11.1).

11.2.2 Covariate shift

Under covariate shift the rule the model learned is still correct; it is simply being applied to a population it saw rarely during training (Figure 11.6). PesaCred signs a distribution partnership with a sacco in western Kenya, and the applicant mix moves towards younger borrowers with shorter mobile-money tenure. Nothing about the relationship between tenure and default has changed. What has changed is that the model is now extrapolating in a region where its training data was thin, and its errors there were never measured.

Covariate shift is the friendliest of the three, for two reasons. It is fully visible without labels, since it lives entirely in $P(X)$, which the service observes on every request. And it often requires no action at all: a model that was accurate over the whole feature space remains accurate on a new mix of that space. The action it does require is vigilance about *coverage*. If the new region is one where training data was sparse, error there is unmeasured rather than known to be small, and the sensible response is to collect labelled outcomes from that region and retrain when they mature.

11.2.3 Label shift

Under label shift the proportion of defaulters changes while borrowers of each kind look the same as before. A drought in the maize-growing counties, a national election, a new levy on mobile-money transfers: all can raise

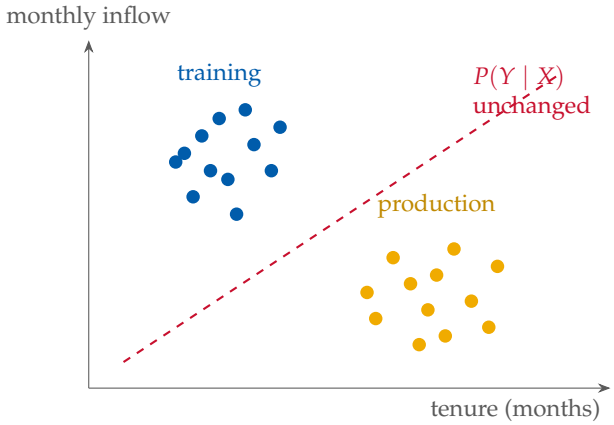


Figure 11.6: Covariate shift: the population moves, the relationship does not.

the default rate from four per cent to seven without changing what a defaulting borrower's transaction history looks like.

This is the one kind of shift with a clean mathematical remedy, because the assumption $P(X \mid Y)$ constant is exactly what is needed to transport the old model's probabilities to the new world (Saerens et al., 2002; Lipton et al., 2018).

Proposition 11.1 (Prior correction under label shift)**.** *Suppose $P_t(X \mid Y) = P_{t_0}(X \mid Y)$ and let $\pi = P_{t_0}(Y = 1)$ and $\pi' = P_t(Y = 1)$. If $p(x) = P_{t_0}(Y = 1 \mid x)$ is the calibrated output of the deployed model, then the correct probability under the new prior is*

$$\tilde{p}(x) \;=\; \frac{w\,p(x)}{w\,p(x) + \big(1 - p(x)\big)}, \qquad w \;=\; \frac{\pi'}{\pi} \cdot \frac{1-\pi}{1-\pi'}. \tag{11.2}$$

Proof. By Bayes' theorem, the posterior odds equal the likelihood ratio times the prior odds: $\frac{P(Y=1\mid x)}{P(Y=0\mid x)} = \frac{P(x\mid Y=1)}{P(x\mid Y=0)} \cdot \frac{P(Y=1)}{P(Y=0)}$. The likelihood ratio is unchanged by assumption, so the new posterior odds equal the old posterior odds multiplied by the ratio of prior odds, which is w. Writing $\tilde{p}/(1-\tilde{p}) = w\,p/(1-p)$ and solving for $\tilde{p}$ gives Equation (11.2). □

The practical value of the result is that it tells you what *not* to do. A rising default rate is not, by itself, a reason to retrain: the model's ranking of borrowers is still correct, and only its scale is wrong. Re-estimating π' and recalibrating is faster, cheaper and less risky than a retraining cycle. Equivalently, one may leave the scores alone and move the threshold instead. Setting $\tilde{p}(x) = t^\star$ and solving for the old score gives the equivalent threshold on the uncorrected scale,

$$t' \;=\; \frac{t^\star}{w\,(1 - t^\star) + t^\star}, \tag{11.3}$$

which for $\pi : 4\% \to 7\%$ and $t^\star = 0.074$ gives $w = 1.806$ and $t' = 0.042$: a stricter cut on the old scores, exactly compensating for the fact that the old model now understates risk (Figure 11.7).

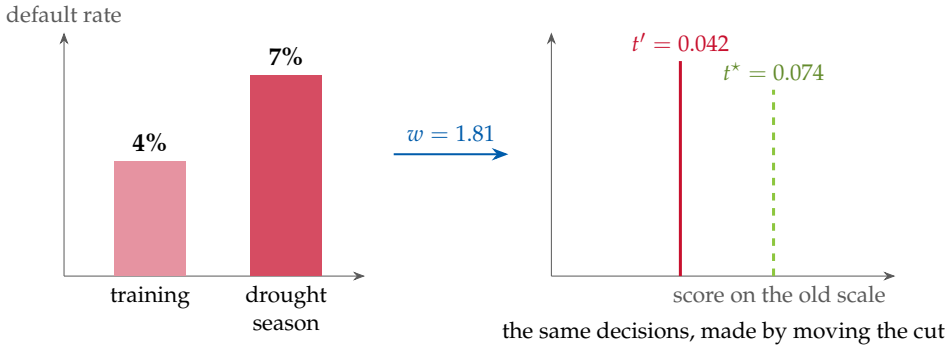


Figure 11.7: Label shift and its remedy. The ranking is still right; only the scale is wrong.

11.2.4 Concept drift

Concept drift is the hardest of the three, because the model's learned relationship has itself become wrong. A high count of mobile-money transactions used to indicate a healthy small business. After a competitor launches an instant overdraft product, the same high count can indicate a borrower juggling several short-term loans. The feature has not moved; its meaning has (Figure 11.8).

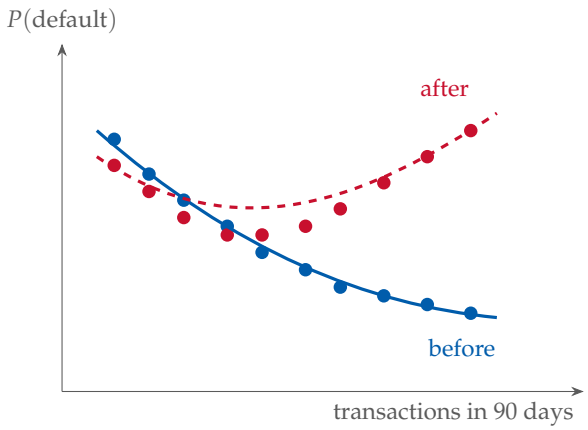


Figure 11.8: Concept drift: the same feature values now carry a different meaning.

Two properties make concept drift dangerous. It is invisible in $P(X)$: the input distribution can be perfectly stable while $P(Y \mid X)$ rotates underneath it, so every input-drift detector in Section 11.3 stays silent. And

its only reliable evidence is labelled outcomes, which for a lender arrive months late. The remedy, when it is finally confirmed, is a genuine retraining on recent labelled data, with the caveat that "recent" and "enough" pull in opposite directions.

11.2.5 How drift unfolds in time

Drift also has a shape in time, and the shape decides what detection window will work (Figure 11.9). *Sudden* drift is a step: a change in excise duty on mobile-money transfers takes effect on a date. *Gradual* drift is a mixture whose weights change, as users migrate to a competitor's app. *Incremental* drift is a slow ramp, as inflation erodes the real value of a nominal inflow threshold. *Recurring* drift is a cycle: month-end salary spikes, school-fee months, planting and harvest seasons.

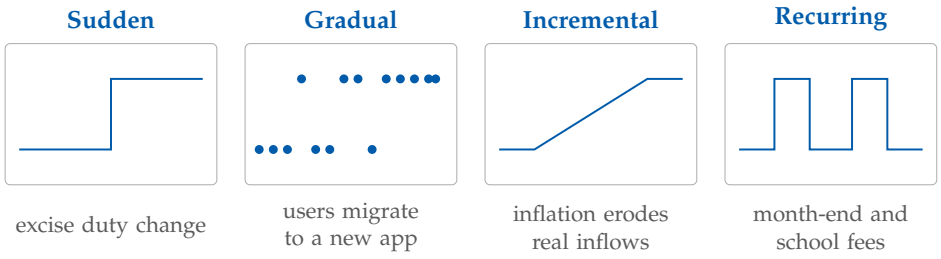


Figure 11.9: Four temporal patterns. A window short enough to catch the first will alarm constantly on the fourth.

The distinction between recurring drift and the rest is the single most practical idea in this section. Recurring variation is not drift at all: it is a feature of the domain that the model should know about. In a Kenyan lending book, several cycles are predictable in advance (Figure 11.10). School fees fall due in January, May and September. The long rains run from March to May and the short rains from October to December, moving agricultural incomes with them. The Finance Act takes effect on 1 July. Salaries land at month end. A monitoring system that alarms every September has not detected anything; it has rediscovered the school calendar.

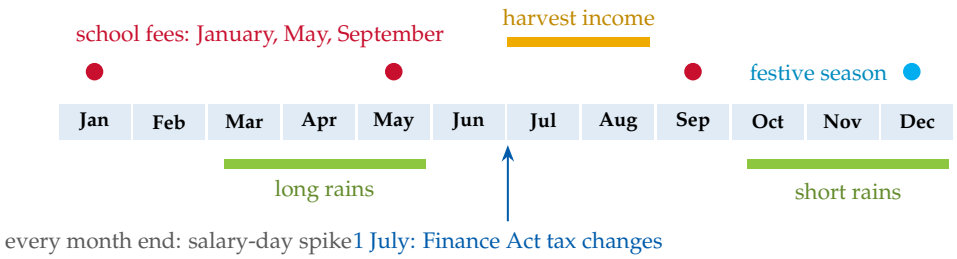


Figure 11.10: A drift calendar for a Kenyan loan book. Know the rhythms before you alarm on them.

Kenya Context

Seasonal effects that recur belong *inside* the model, as features: a school-fee-month indicator, a rainfall-season indicator, a month-end flag. Once the model knows about them, the monitoring system can compare a September window with last September rather than with August, and the alerts that remain are the ones worth investigating: a general election, a new mobile-money levy, a competitor's product launch, a regulatory cap on interest rates. The practical rule is that predictable variation is a modelling problem and unpredictable variation is a monitoring problem.

11.2.6 What can be seen without labels

The chapter's structure now follows from a single table (Table 11.1). Covariate shift is fully visible in the inputs and in the scores, and needs no labels. Label shift is partly visible, since a changing prior moves the score distribution, but confirming it needs outcomes. Concept drift is invisible in both until the labels arrive.

Drift type	Visible in the inputs	Visible in the scores	Needs labels?
Covariate	yes	yes	no
Label	partly	yes	to confirm
Concept	no	rarely	yes

Table 11.1: What each kind of drift reveals before the ground truth arrives.

Key Idea

Early warning and verdict Label-free monitoring of inputs and scores is an *early warning*: cheap, fast, and available on every request, but capable of being both wrong and silent. Performance measured on matured labels is the *verdict*: correct, but months late. A production system needs both, and needs to know which one it is looking at.

11.3 Detecting drift statistically

11.3.1 Reference and current windows

Every drift check compares two samples (Figure 11.11). The *reference* window is a period in which the model was known to be performing well, normally the validation set frozen when the model was promoted; storing it, or its summary statistics, alongside the artefact is part of the packaging discipline of Chapter 10. The *current* window is a recent slice of production traffic, drawn from the decision log of Section 11.5.4.

Definition 11.2 (Reference and current windows)**.** A *reference window* is a fixed sample $\{\boldsymbol{x}_i^{\text{ref}}\}_{i=1}^{n}$ from the period in which the deployed model was validated. A *current window* is a sample $\{\boldsymbol{x}_j^{\text{cur}}\}_{j=1}^{m}$ covering the most recent production period of length w. Drift detection is the comparison of the distributions underlying the two.

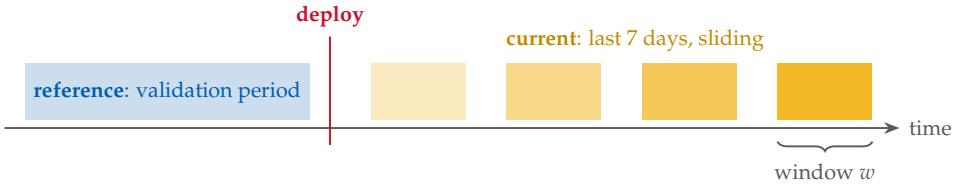


Figure 11.11: Drift detection compares a fixed reference window with a sliding current window.

The window length w is the first real design decision, and it is a bias–variance trade-off in disguise. A short window reacts quickly to sudden drift and is noisy; a long window is stable and slow. Two rules keep the choice sane. Use a window that spans a whole weekly cycle, so that Sunday traffic is not mistaken for drift; and choose w so that even the smallest monitored slice has enough observations to estimate a distribution, which for PesaCred's smaller regions means a week rather than a day.

11.3.2 Drift detection as a two-sample test

Formally, drift detection asks whether two samples come from the same distribution: $H_0 : P_{\text{ref}} = P_{\text{cur}}$ against $H_1 : P_{\text{ref}} \neq P_{\text{cur}}$. The tools divide into two families whose difference matters more in practice than any choice within them. *Tests*, such as Kolmogorov–Smirnov and χ^2, return a p-value: the probability of seeing a discrepancy this large if the distributions were identical. *Distances*, such as Wasserstein, the population stability index and Jensen–Shannon, return an effect size: how far apart the two distributions are, in units that do not depend on how much data you have.

11.3.3 The Kolmogorov–Smirnov statistic

For a continuous feature, the most familiar test compares the two empirical cumulative distribution functions and takes their largest vertical gap (Figure 11.12):

$$D = \sup_x \left| \hat{F}_{\text{ref}}(x) - \hat{F}_{\text{cur}}(x) \right|. \tag{11.4}$$

The statistic is distribution-free, which is its great virtue: the null distribution of D does not depend on the shape of the underlying distribution, so no assumption of normality is needed. At the 5% level the null is rejected when D exceeds approximately $1.36\sqrt{(n+m)/(nm)}$, a threshold that shrinks as the windows grow. The statistic is most sensitive near the centre of the distribution and least sensitive in the tails, which is a real limitation for credit data, where the interesting movement is often among the highest-risk applicants.

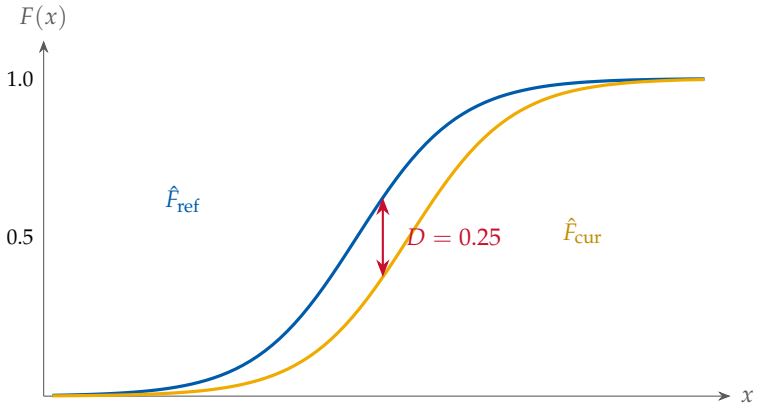


Figure 11.12: The Kolmogorov–Smirnov statistic is the largest vertical gap between two empirical CDFs.

11.3.4 Why p-values make bad alerts

There is a deeper problem with tests, and it is the reason production monitoring systems are built on distances instead. A p-value answers "could this difference be an accident of sampling?". As the sample grows, the answer becomes "no" for differences of any size, including differences far too small to matter. PesaCred scores about 1,800 applications a day; a monthly window holds over fifty thousand. At that size the KS test rejects the null for a shift of two hundredths of a standard deviation (Figure 11.13), a change no risk officer could act on.

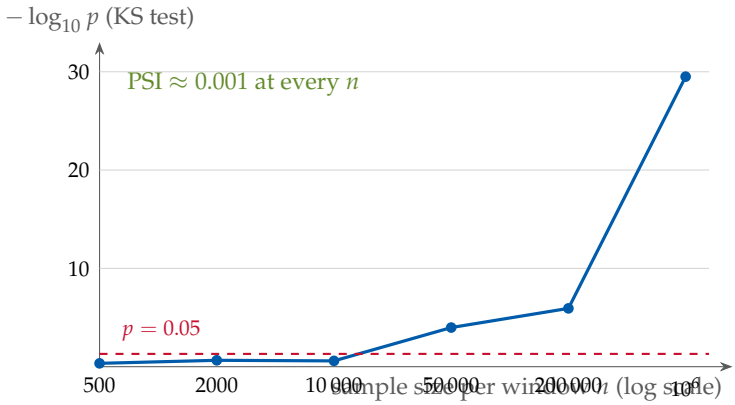


Figure 11.13: The same negligible shift, tested at six sample sizes (simulated). The p-value collapses; the effect size does not move.

Key Idea

Alert on effect size, not on significance A drift alert should fire when the distribution has moved *enough to matter*, not when a test has enough data to notice that it moved at all. Use tests for small windows, where noise is the dominant concern, and distances for large ones, where it is not. Whichever is used, tune the threshold on your own history: replay the last year of data through the detector and see how often it would have fired, and on what.

11.3.5 The Wasserstein distance

The first-order Wasserstein distance measures how much probability mass must move, and how far, to turn one distribution into the other. For one-dimensional data it has a pleasant closed form as the area between the two CDFs (Figure 11.14),

$$W_1 = \int_{-\infty}^{\infty} \left| F_{\text{ref}}(x) - F_{\text{cur}}(x) \right| dx. \tag{11.5}$$

Where KS takes the largest gap, W_1 takes the whole area, so it is sensitive to movement anywhere, including in the tails. It is expressed in the units of the feature, which makes it easy to explain to a risk committee: "average monthly inflow has moved by KES 3,400" is a sentence anyone can act on. Monitoring tools normally report it divided by the reference standard deviation, so that a single threshold, commonly 0.1, can apply to every feature. Unlike a p-value, it converges to the true distance as the windows grow rather than to a decision.

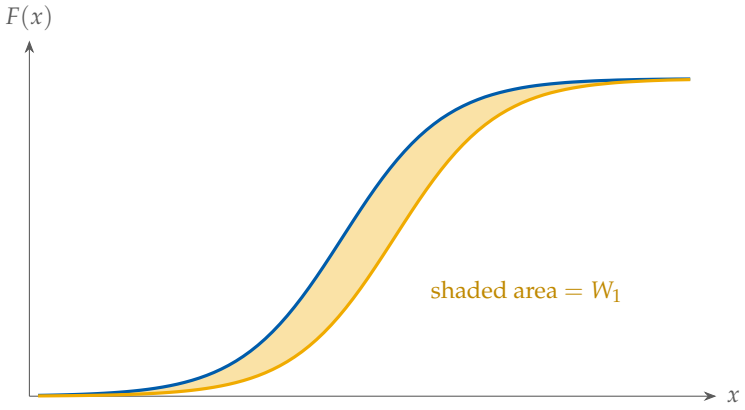


Figure 11.14: The Wasserstein distance is the area between the CDFs, in the units of the feature.

11.3.6 The population stability index

Credit risk has its own standard measure, older than the machine learning literature on drift and still the one a Kenyan risk committee will ask for (Siddiqi, 2006). The population stability index bins the reference distribution into quantiles and compares the share of observations falling in each bin.

Definition 11.3 (Population stability index)**.** Partition the reference distribution into B bins, normally its deciles, and let p_b and q_b be the proportions of the reference and current windows in bin b. The *population stability index* is

$$\mathrm{PSI} \;=\; \sum_{b=1}^{B} (q_b - p_b)\,\ln\frac{q_b}{p_b}. \tag{11.6}$$

The formula is a symmetrised Kullback–Leibler divergence: adding $\mathrm{KL}(P \,\|\, Q)$ to $\mathrm{KL}(Q \,\|\, P)$ gives exactly Equation (11.6). Its practical appeal is that each term is attributable: the sum can be broken down by bin, so a PSI alert comes with a statement of *where* the population moved (Figure 11.15). In the illustrated case, applicants have shifted towards the lower deciles of

monthly inflow, giving PSI $= 0.15$, and the two extreme deciles account for about two thirds of that total.

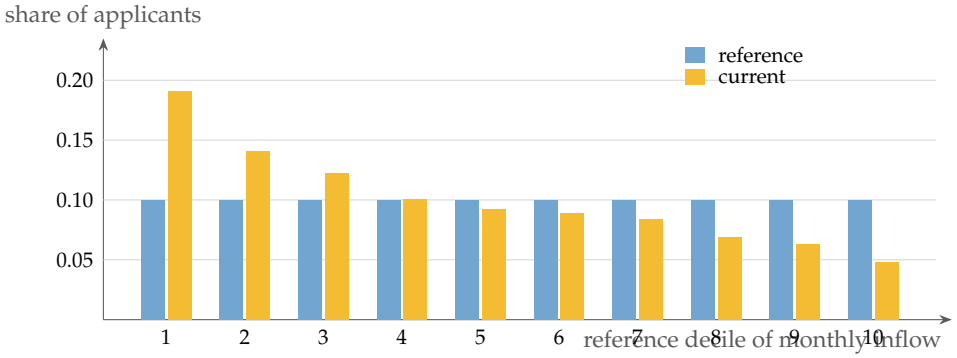


Figure 11.15: Population stability index by reference decile (simulated). Here PSI $= 0.15$.

Interpretation follows a convention that scorecard practice has used for decades: below 0.10 the population is stable, between 0.10 and 0.25 it warrants investigation, and above 0.25 it has shifted enough to act on (Figure 11.16). These cut-offs are conventions, not theorems, and they were calibrated on scorecards with a handful of binned variables; treat them as a starting point and re-tune them against your own replayed history.

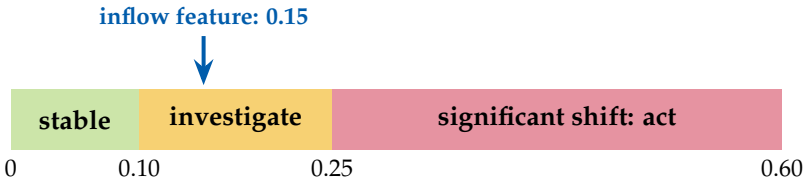


Figure 11.16: Conventional PSI traffic lights from credit-risk practice.

Two implementation details decide whether a PSI value means anything at all. The bin edges must come from the *reference* window and be stored with the model, because recomputing them on the current window compares each window with itself and guarantees a PSI near zero. And empty bins make $\ln(q_b/p_b)$ diverge, so shares are floored at a small ε. Listing 11.1 implements both, alongside the KS and normalised Wasserstein statistics for comparison.

```
import numpy as np
from scipy import stats

def psi(ref, cur, bins=10, eps=1e-6):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))    # reference deciles
    edges[0], edges[-1] = -np.inf, np.inf                    # catch out-of-range
     values
```

```
    p = np.histogram(ref, edges)[0] / len(ref)
    q = np.histogram(cur, edges)[0] / len(cur)
    p, q = np.clip(p, eps, None), np.clip(q, eps, None)      # no log(0)
    return float(np.sum((q - p) * np.log(q / p)))

col = "avg_monthly_inflow_kes"
print("PSI:", psi(ref_df[col], cur_df[col]))                          # 0.15  [
    SIMULATED]
print("KS :", stats.ks_2samp(ref_df[col], cur_df[col]).statistic)     # 0.16
print("W1 :", stats.wasserstein_distance(ref_df[col], cur_df[col]) / ref_df[col].std
    ())
```

Listing 11.1: PSI, KS and normalised Wasserstein in a few lines. Bin edges come from the reference only.

Pitfall

Recomputing the bin edges If the quantile edges are recomputed from the current window, each window is compared with its own quantiles and every bin holds exactly a tenth of the data, so the PSI is zero no matter how far the distribution has moved. Persist the edges next to `scorer.keras`: they are part of the monitoring contract, exactly as the threshold is part of the decision function.

11.3.7 Categorical features and the out-of-vocabulary signal

For a categorical feature such as region, the comparison is between two sets of category shares. The χ^2 test is the classical choice for small windows, and the Jensen–Shannon distance the usual effect size for large ones. With $M = \frac{1}{2}(P + Q)$,

$$\mathrm{JS}(P \,\|\, Q) \;=\; \tfrac{1}{2}\,\mathrm{KL}(P \,\|\, M) + \tfrac{1}{2}\,\mathrm{KL}(Q \,\|\, M), \tag{11.7}$$

which is symmetric, bounded, and defined even when a category appears in one window and not the other, where the raw KL divergence is infinite.

Categorical monitoring also offers a free and unusually informative signal. The preprocessing graph of Chapter 10 routes unseen category codes to a dedicated out-of-vocabulary bucket, so the service can count how often that bucket is used without any extra instrumentation (Figure 11.17). A sudden rise in the out-of-vocabulary share almost never means that a new population has appeared; it usually means that an upstream system changed its codes, which is a data-pipeline problem and not drift at all.

11.3.8 Multivariate drift: the domain classifier

Feature-by-feature tests miss a whole class of change: the marginal distribution of each feature can be unchanged while their *joint* distribution moves. If the proportion of long-tenure borrowers and the proportion of

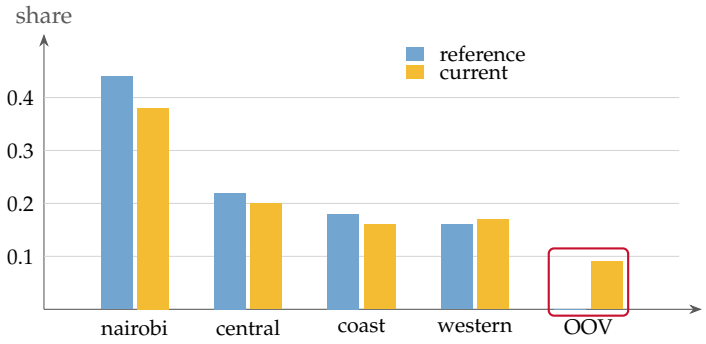


Figure 11.17: Category shares, with the out-of-vocabulary bucket circled. Nine per cent of requests now carry an unrecognised region code.

high-inflow borrowers both stay constant but the two stop occurring together, no univariate detector will notice.

The standard trick is to turn the question into a supervised problem (Rabanser et al., 2019). Label reference rows $s = 0$ and current rows $s = 1$, pool them, and train a classifier to tell the two windows apart (Figure 11.18). If the windows come from the same distribution, no classifier can do better than chance and the cross-validated AUC is about 0.5. An AUC appreciably above 0.5 proves that something distinguishes them, and the classifier's feature importances say what.

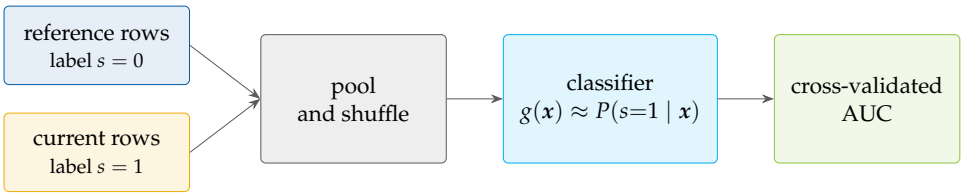


Figure 11.18: The domain classifier turns drift detection into a supervised problem.

```
import numpy as np, pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import roc_auc_score

X = pd.concat([ref_df[FEATURES], cur_df[FEATURES]], ignore_index=True)
s = np.r_[np.zeros(len(ref_df)), np.ones(len(cur_df))]        # which window?

clf = HistGradientBoostingClassifier(max_iter=200, categorical_features="from_dtype"
    )
p = cross_val_predict(clf, X, s, cv=5, method="predict_proba")[:, 1]   # out-of-fold
print(f"domain AUC = {roc_auc_score(s, p):.3f}")              # 0.50 none; 0.62
    investigate
```

Listing 11.2: A domain classifier, reusing the gradient-boosting tools of Chapter 4.

Two details protect the result. The AUC must be computed from out-of-fold predictions, since a flexible model can separate any two samples in-sample. And identifiers, timestamps and row counters must be dropped before training, or the classifier will learn the calendar and report perfect separation with no information content.

11.3.9 Prediction drift

The cheapest and often the most informative monitor watches the model's own output. The predicted probability is a one-dimensional summary of everything the model saw, it is available on every request without labels, and it moves when inputs move in ways that the model considers relevant, which is exactly the subset of input drift worth alarming on.

Two statistics suffice. The PSI of the score distribution against its reference detects a change in shape; the approval rate, the proportion of applicants below $t^\star$, translates that change into the language the business already speaks (Figure 11.19). An approval rate that has fallen from 81% to 66% will be noticed by the sales team within a week. It should be noticed by the monitoring system within a day.

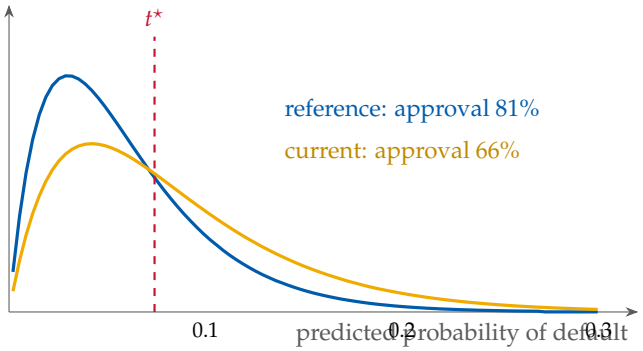


Figure 11.19: Prediction drift. The score distribution has moved right, and fewer applicants now fall below the threshold.

11.3.10 Many features, many false alarms

A model with forty features invites forty drift tests, and running forty tests at the 5% level on stable data produces an alert on most days. If the tests were independent, the probability of at least one false alarm is

$$\Pr(\text{at least one alarm}) \;=\; 1 - (1 - \alpha)^{m}, \tag{11.8}$$

which for $m = 40$ and $\alpha = 0.05$ is 0.87 (Figure 11.20). A system that cries wolf on seven days out of eight will be ignored within a month, and the one true alarm will be ignored with it.

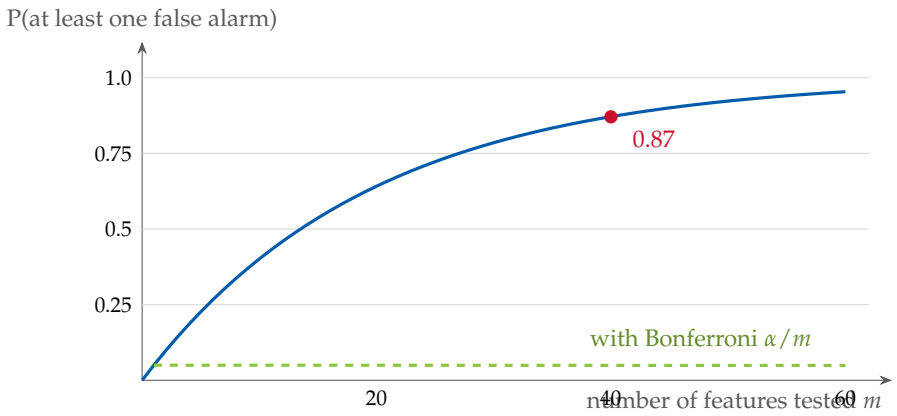


Figure 11.20: False alarms accumulate with the number of monitored columns.

Three remedies are used, and they combine. A **Bonferroni correction** tests each feature at α / m, restoring the overall rate to about α at the cost of sensitivity. A **drift share** rule ignores individual features and alarms only when a fraction of columns, commonly 30%, drift together, which is both robust to isolated noise and well matched to the way real shifts announce themselves. And **weighting by importance** concentrates attention where it matters: drift in the model's top three features is worth an alert, drift in a feature with negligible attribution is worth a line in a weekly report.

11.4 Drift detection with Evidently AI

11.4.1 What the library does

Evidently is an open-source Python library that turns two data frames into drift evidence (Evidently AI, 2025). It chooses a sensible statistical method for each column, runs it, and renders the result in two forms at once: an HTML report for a human, and a dictionary for a program (Figure 11.21). That duality is the reason to use a library rather than the thirty lines of Listing 11.1. The numbers are easy; the plots that let a risk officer see *how* a distribution moved, and the machine-readable summary that lets a pipeline act on it, are the work.

Listing 11.3 runs a full drift report on PesaCred's last seven days. The `DataDefinition` declares which columns are numerical and which categorical, which matters because it decides the method; note that the model's own output, `pd`, is included as a numerical column, so prediction drift is computed alongside input drift. The `drift_share` argument sets the rule of Section 11.3.10: the dataset counts as drifted when at least 30% of columns do.

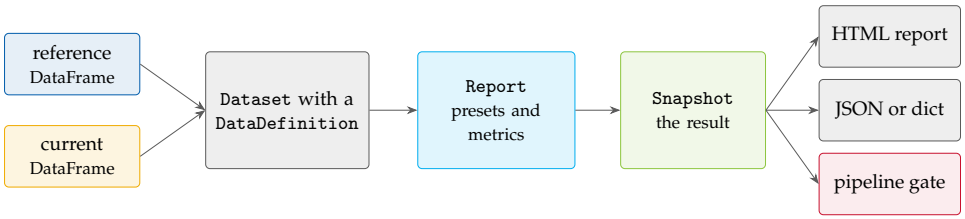


Figure 11.21: The Evidently workflow. One run produces evidence for people and a decision for machines.

```
from evidently import Report, Dataset, DataDefinition
from evidently.presets import DataDriftPreset

schema = DataDefinition(
    numerical_columns=["tenure_months", "avg_monthly_inflow_kes",
                       "txn_count_90d", "ontime_repayments_12m", "pd"],
    categorical_columns=["region"])

ref = Dataset.from_pandas(ref_df, data_definition=schema)   # frozen validation
    period
cur = Dataset.from_pandas(cur_df, data_definition=schema)   # last 7 days of
    decisions

report = Report([DataDriftPreset(drift_share=0.3)])
snapshot = report.run(current_data=cur, reference_data=ref)
snapshot.save_html("reports/drift_2026-10-19.html")         # for people
```

Listing 11.3: A drift report with the modern Evidently API (version 0.7 and later).

Pitfall

Two APIs in circulation Evidently was substantially redesigned at version 0.7. Tutorials that begin `from evidently.report import Report` target the older interface, which now lives under `evidently.legacy`; mixing the two produces confusing import errors. Pin the version in `requirements.lock`, as Chapter 10 pins everything else, and check which API a tutorial is using before copying from it.

11.4.2 What the preset decides for you

The preset applies the rule of Section 11.3.4 automatically, choosing tests for small windows and distances for large ones (Figure 11.22). For a numerical column it uses the KS test when the window has at most 1,000 rows and the normalised Wasserstein distance above that; for a categorical column, the χ^2 test below the same cut-off and the Jensen–Shannon distance above it. Knowing the defaults matters when reading a report: a column flagged as drifted at 0.12 has been judged by a distance against a threshold of 0.1, not by a p-value.

Defaults are a starting point, not a policy. PesaCred's risk team already

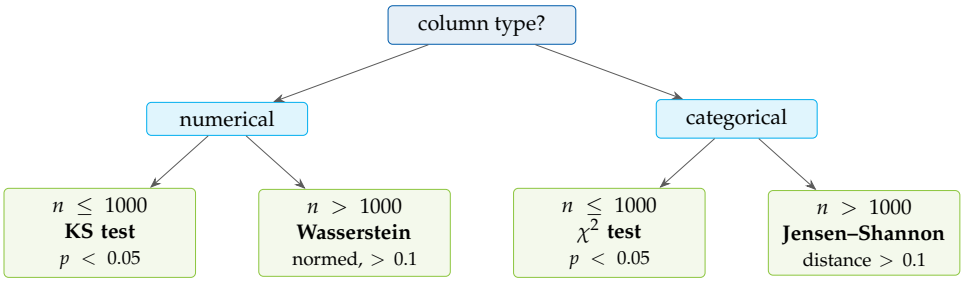


Figure 11.22: Default drift method by column type and window size. Tests for small windows, distances for large ones.

tracks the inflow feature and the score with PSI, and reports PSI values to its board, so the report should speak the same language for those two columns (Listing 11.4). The threshold on the score is set tighter than on any input, because the score aggregates every input and is the quantity closest to the decision. Every override is a policy choice and belongs in the model card of Section 11.7.3.

```
preset = DataDriftPreset(
    drift_share=0.3,
    per_column_method={"avg_monthly_inflow_kes": "psi", "pd": "psi"},
    per_column_threshold={"avg_monthly_inflow_kes": 0.25,   # scorecard convention
                          "pd": 0.10})                      # stricter on the score
snapshot = Report([preset]).run(current_data=cur, reference_data=ref)
```

Listing 11.4: Overriding the defaults where the business already has a convention.

11.4.3 From a report to a gate

A report that a human must open is a report that will eventually go unread. The same snapshot exposes its results as a dictionary, so a short script can turn them into an exit code (Listing 11.5). Non-zero means drift, which is a language every scheduler, pipeline and shell understands.

```
import json, sys

metrics = snapshot.dict()["metrics"]
summary = next(m for m in metrics if m["metric_name"].startswith("
    DriftedColumnsCount"))
share = summary["value"]["share"]                       # fraction of drifted
    columns
drifted = [m["config"]["column"] for m in metrics
           if m["metric_name"].startswith("ValueDrift")
           and m["value"] > m["config"]["threshold"]]    # distance-based methods

json.dump({"share": share, "drifted": drifted}, open("reports/drift.json", "w"))
print(f"drift share = {share:.2f}; columns: {drifted}")
sys.exit(1 if share >= 0.3 else 0)                      # non-zero exit = drift
    detected
```

Listing 11.5: `check_drift.py`: the same run, read by a machine.

Wiring that script into the pipeline of Chapter 10 completes the loop (Listing 11.6). A scheduled workflow exports the last window from the decision log, runs the gate, and on failure triggers the continuous-training workflow and notifies the model's owner. One sentence of policy governs the whole arrangement: *drift triggers a candidate, never a deployment*. The retrained model still enters as `@challenger` and still passes the cost, fairness, behaviour and latency gates of Chapter 10 and the risk officer's approval.

```yaml
name: drift-check
on:
  schedule:
    - cron: "0 2 * * *"              # 02:00 UTC = 05:00 EAT, off-peak
jobs:
  drift:
    runs-on: [self-hosted, nairobi-dc]
    steps:
      - uses: actions/checkout@v4
      - run: python monitoring/export_window.py --days 7
      - id: gate
        run: python monitoring/check_drift.py
      - if: failure()
        run: gh workflow run train.yml            # continuous training, Chapter 10
      - if: failure()
        run: python monitoring/notify.py --owner model-owner --report reports/drift.
     json
```

Listing 11.6: A nightly drift check that can trigger retraining.

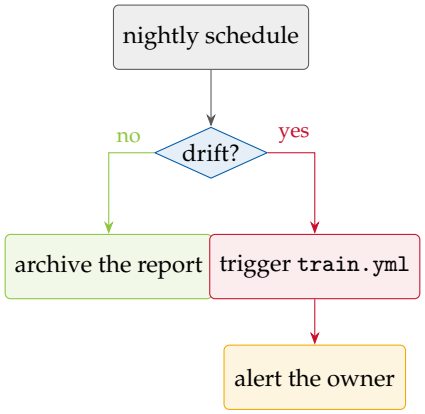


Figure 11.23: The drift gate and its two exits. The failing branch produces a candidate, not a deployment.

11.5 Monitoring in production

11.5.1 Four layers

A production monitoring stack has four layers, ordered from the cheapest and fastest signal to the slowest and most decisive (Figure 11.24). Op-

erational metrics, such as latency, error rate and saturation, arrive within seconds and catch the loud failures. Data-quality checks, such as schema conformity, null rates and out-of-vocabulary shares, arrive within minutes of each batch and catch the pipeline bugs that masquerade as drift. Model-output monitoring, the score distribution and the approval rate, arrives daily and catches shift that the model considers relevant. Performance on matured labels arrives monthly and is the only layer that settles the question.

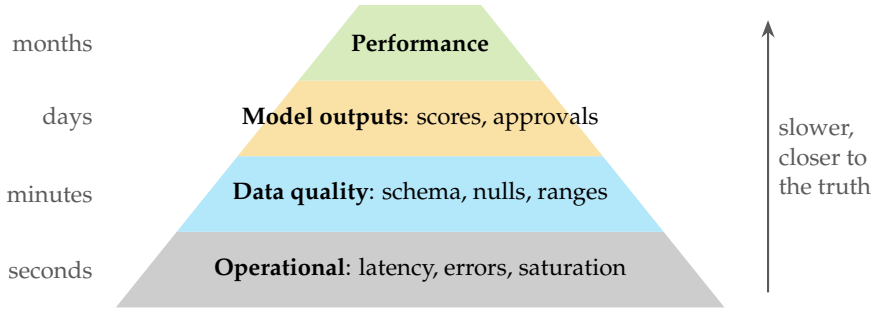


Figure 11.24: Four layers of monitoring, with the latency of each signal.

Each layer catches failures the others miss, which is why all four are needed. A container serving a stale model has perfect operational metrics and perfect data quality. A currency-unit bug has perfect operational metrics and catastrophic data quality. Concept drift passes the first three layers cleanly and shows up only in the fourth.

11.5.2 The delayed-label problem

Lending is the canonical example of a domain in which the ground truth arrives long after the decision (Figure 11.25). A thirty-day loan disbursed today has a due date in a month, and PesaCred's definition of default requires sixty days past due, so the label matures about ninety days after the decision.

Definition 11.4 (Feedback delay)**.** The *feedback delay* Δ of a decision system is the time between making a decision and observing the outcome that defines its correctness. A monitoring system can measure true performance only on decisions older than Δ.

The delay has a direct operational cost. Concept drift that begins today will not appear in an AUC computed on matured labels for three months, during which PesaCred will have decided roughly 162,000 applications with a model that is quietly wrong. Three partial remedies exist, and a serious system uses all of them. *Proxies* shorten the delay: first-payment behaviour at day seven is noisy but arrives twelve weeks earlier, and its

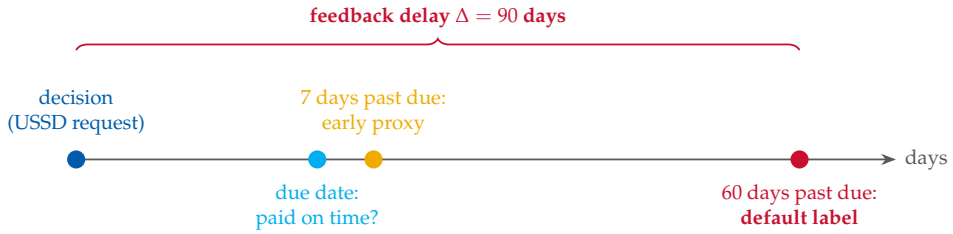


Figure 11.25: Label maturity for a thirty-day loan. Everything to the left of the last milestone must be monitored without labels.

relationship with eventual default can be estimated from history. *Label-free monitoring*, the subject of Sections 11.3 and 11.4, watches the inputs and the scores in the meantime. And *performance estimation*, next, uses the model's own calibrated probabilities to predict what the eventual cost will be.

11.5.3 Estimating performance before the labels arrive

If the model's probabilities are calibrated and the shift is purely covariate, the model can be used to estimate its own expected cost. For applicant i with calibrated probability p_i and approval decision $a_i = \mathbb{1}[p_i < t^\star]$, the expected cost of this week's decisions is

$$\widehat{\text{Cost}} = \frac{1}{n}\sum_{i=1}^{n}\Big[\, a_i\, p_i\, C_{FN} \;+\; (1-a_i)(1-p_i)\, C_{FP}\Big], \tag{11.9}$$

where the first term prices the approvals that will default and the second the declines that would have repaid. Nothing in Equation (11.9) requires an outcome, so it can be computed the moment the decisions are made (Figure 11.26). This is the idea behind confidence-based performance estimation, and it extends in the obvious way to AUC and to any metric expressible as an expectation over predicted probabilities.

Pitfall

The estimate is blind to concept drift Equation (11.9) trusts p_i. Under covariate shift that trust is justified, since $P(Y \mid X)$ is unchanged and the model's probabilities remain calibrated on the new population. Under concept drift it is not: the probabilities themselves are now wrong, so the estimate is wrong in exactly the situation where you most need it to be right, and it will usually be optimistic. Treat the estimate as a monitor of covariate shift's consequences, never as a substitute for the verdict on matured labels.

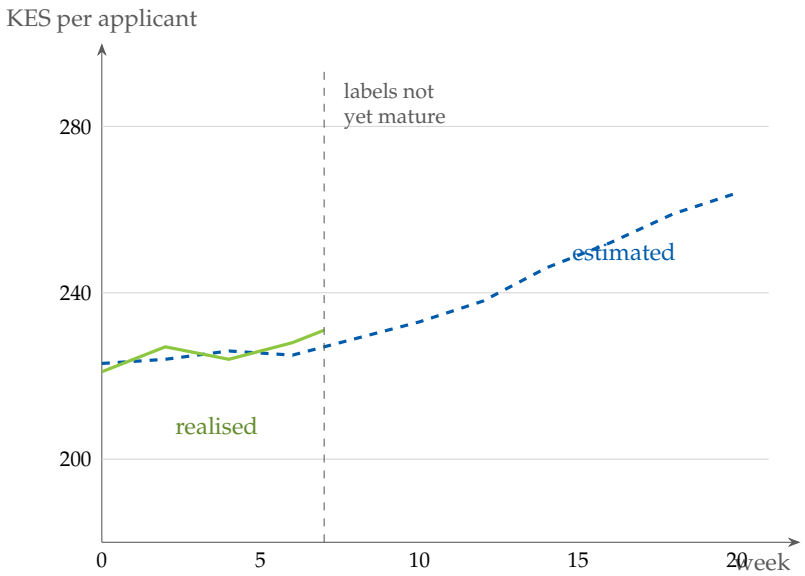


Figure 11.26: Estimated and realised cost per applicant (illustrative). The estimate is available where the realised curve cannot yet be drawn.

11.5.4 The decision log

Every monitor in this chapter reads from one place. The decision log is an append-only record, written by the scoring service on every request, that holds everything needed to reconstruct and later evaluate a decision (Listing 11.7). It is the raw material of drift detection, of the performance estimate, of the fairness slices of Section 11.7.1 and of the audit chain of Chapter 10.

```
record = {
    "request_id":    rid,                 # join key for labels and complaints
    "ts":            now_utc_iso(),
    "model_version": meta["version"],     # which model decided
    "threshold":     meta["threshold"],   # which rule it applied
    "features":      feats,               # raw inputs, no MSISDN
    "pd":            pd_,
    "decision":      decision,
    "latency_ms":    round(1e3 * elapsed, 1),
}
decision_log.write(json.dumps(record))    # append-only
```

Listing 11.7: The per-request decision log.

Three design choices in that record repay attention. The features are stored as the service received them, so drift is computed on exactly the values the model saw rather than on a reconstruction from the warehouse. The model version is stored on every row, which is what makes it possible to compare two versions during a canary and to list the decisions made

by a version that was later rolled back. And the join key is a request identifier rather than a customer identifier, so that outcomes can be attached ninety days later without the log itself becoming a customer database (Figure 11.27).

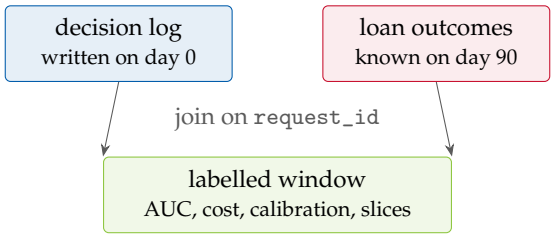


Figure 11.27: The verdict is produced by joining old decisions with newly matured outcomes.

Kenya Context

The decision log is a personal-data store and must be designed as one. Purpose limitation under the Data Protection Act 2019 means logging what monitoring and complaint handling need and no more: the salted phone-number hash rather than the number, the reason codes rather than free text about the customer. Set an explicit retention period, agreed with the data protection officer and long enough to cover the CBK complaint window and a supervisory review, and delete on schedule. Keep the hashing salt in a separate system from the logs, so that a compromise of the log store does not re-identify borrowers.

11.5.5 Operational metrics and alerting

The bottom layer of the pyramid is conventional software monitoring, and the conventional tools apply. The service exposes counters and histograms on a metrics endpoint; Prometheus scrapes them on an interval; Grafana draws them; Alertmanager pages a human when a rule fires (Listing 11.8). Labelling the decision counter by model version is a small detail with a large payoff, because it makes the approval rate of each version visible separately during a canary.

```
from prometheus_client import Counter, Histogram, make_asgi_app

LATENCY = Histogram("score_latency_seconds", "Scoring latency",
                    buckets=(.01, .05, .1, .15, .3, 1, 2))
DECISIONS = Counter("decisions_total", "Decisions", ["decision", "model_version"])
app.mount("/metrics", make_asgi_app())          # Prometheus scrape endpoint

@app.post("/v1/score", response_model=ScoreResponse)
def score(a: Applicant) -> ScoreResponse:
    with LATENCY.time():
```

```
    r = _score(a)                          # the Chapter 10 endpoint logic
DECISIONS.labels(r.decision, r.model_version).inc()
return r
```

Listing 11.8: Instrumenting the scoring service.

What turns metrics into a monitoring system is the alert policy, and the hardest part of an alert policy is restraint. Every alert should name a *threshold*, an *owner* and a *runbook*, and should distinguish what must wake someone tonight from what belongs in a ticket (Table 11.2). Latency and error alerts page the on-call engineer, because a broken service harms borrowers now. Drift alerts open a ticket for the model's owner, because the right response is an investigation, not a heroic 3 a.m. retraining. A shift in the approval rate goes to the risk committee, because it is a business event as much as a technical one.

Signal	Rule	Severity	Owner
Latency	$p_{99} > 150$ ms for 10 min	page	on-call engineer
Errors	5xx rate $> 1\%$ for 5 min	page	on-call engineer
Data quality	nulls or OOV $> 5\%$ in a batch	ticket	data engineering
Drift	score PSI > 0.1, or drift share ≥ 0.3	ticket	model owner
Business	approval rate moves > 5 points	review	risk committee

Table 11.2: An alert policy. Thresholds are illustrative and should be tuned on replayed history.

Pitfall

Alert fatigue An alert that nobody acts on teaches the team to ignore alerts, and the lesson generalises to the alert that mattered. The symptoms are familiar: a channel full of notifications, a rule that "always fires on Mondays", a threshold nobody remembers choosing. The cures are unglamorous. Replay history before setting a threshold, page only for what needs action tonight, require an owner and a runbook for every rule, and delete any rule that has fired ten times without once producing an action.

11.5.6 Triage: from alert to action

An alert is a question, not an instruction. The triage procedure of Figure 11.28 asks the four questions of Section 11.1.1 in increasing order of cost, and each answer selects a different remedy: fix the pipeline, recalibrate, retrain, or record and watch.

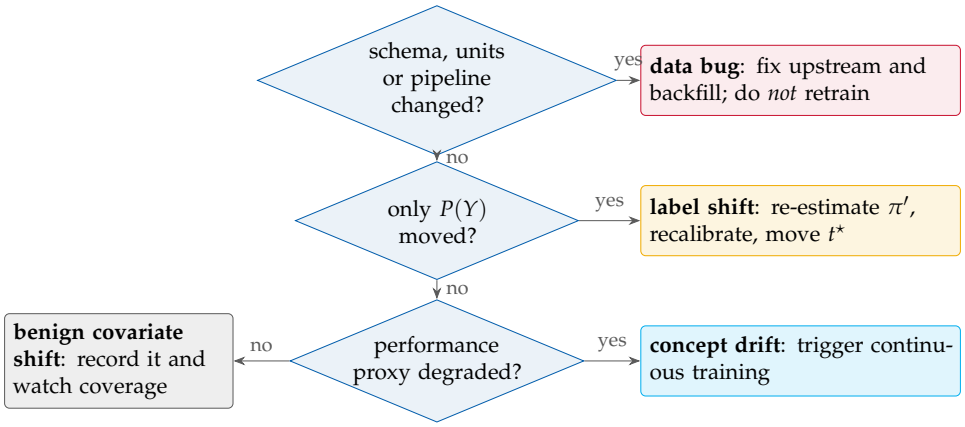


Figure 11.28: Triage, in order of increasing cost. Retraining is the last resort, not the first.

11.5.7 When to retrain

Three retraining strategies are in common use, and they trade staleness against cost and risk (Figure 11.29). *Scheduled* retraining runs on a fixed cadence, quarterly for most credit books, which is simple to operate and can lag a sudden change by weeks. *Triggered* retraining runs when monitoring fires, which is responsive but makes the release calendar unpredictable. *Continual* learning updates the model constantly from a stream of fresh labels, which requires labels that arrive quickly and therefore rarely suits lending.

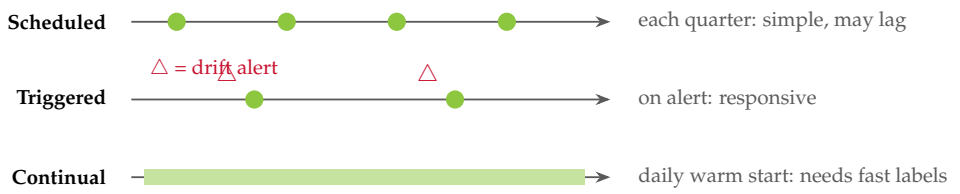


Figure 11.29: Three retraining strategies. Most lenders should start with the first two together.

The sensible default for PesaCred is scheduled plus triggered: a quarterly cadence that keeps the pipeline exercised, and a drift trigger for the quarters in which the world does not wait. Whichever strategy is chosen, one rule is invariant: every retrained model is a new version that must pass every gate of Chapter 10. A pipeline that retrains automatically and deploys automatically has replaced a slow human error with a fast machine one.

11.5.8 Monitoring language-model agents

The underwriting agent of Chapter 10 needs the same discipline with a different instrument. A single agent decision is not one prediction but a tree of calls, so the unit of monitoring is a *trace* rather than a row (Figure 11.30). Four families of signal cover most needs. Per run: latency, token counts, cost in shillings, and the number of steps taken, since an agent that starts looping is both expensive and broken. Per tool call: success rate, schema-validation failures and retries, which catch integration breakage early. For quality: groundedness and policy checks scored by evaluators on a sample of traces, since there is no label. And for drift: the distribution of prompt topics and languages, which in Kenya means English, Kiswahili and Sheng in proportions that move with the customer base.

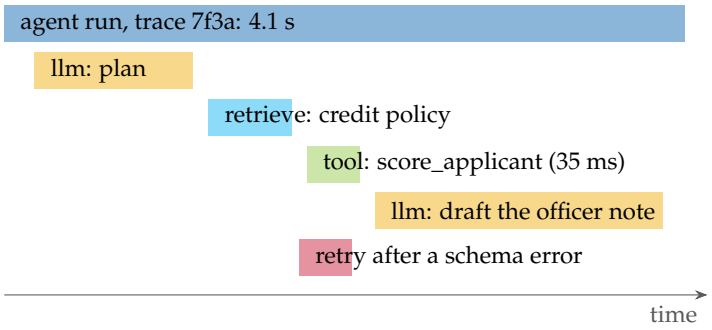


Figure 11.30: One agent decision as a trace. The audited scoring call is a thin sliver of the total.

Key Idea

The same four questions Whatever is being monitored, a tabular scorer, a retrieval system or an agent, the design is the same: choose a reference, choose a statistic, choose a threshold, and name an owner. Most monitoring failures are failures of one of those four choices rather than of the mathematics.

11.6 ML system design principles

11.6.1 Four properties of a good ML system

Monitoring tells you that something has changed. Whether that discovery is a routine Tuesday or a crisis depends on how the system around the model was designed. Huyen (2022) names four properties that a production machine learning system should have, and the three chapters of Part III have been building them (Figure 11.31).

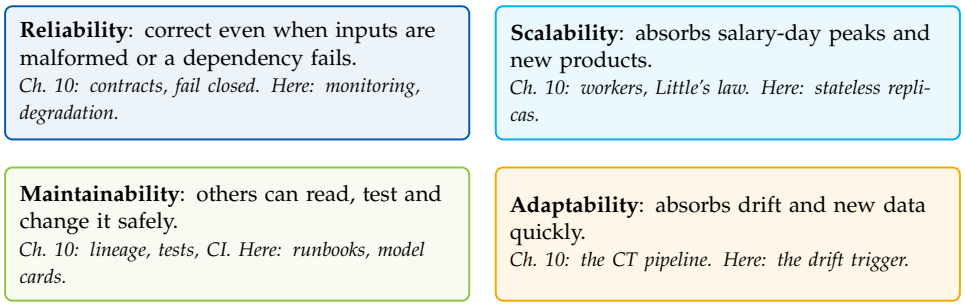


Figure 11.31: Four properties of a production ML system, and where this book builds each.

Design for these properties begins before any model exists, with requirements (Figure 11.32). The business goal, cutting losses on thin-file lending, becomes an ML objective, a calibrated probability with a cost-optimal threshold; the objective meets constraints, the two-second USSD budget, the Data Protection Act, the CBK complaint rules and a budget; and the constraints determine the metrics that will be monitored. Every metric in this chapter traces back through that funnel. The converse is the useful test: a metric with no requirement behind it is a metric nobody will act on, and a monitoring dashboard full of such metrics is worse than an empty one, because it is expensive and reassuring at the same time.

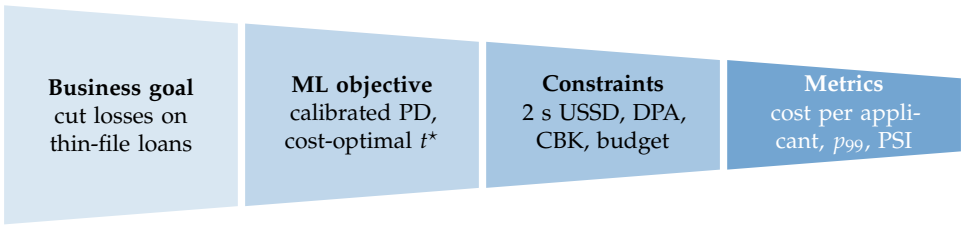


Figure 11.32: Requirements determine metrics. Monitor what a requirement made important.

11.6.2 Scaling out

Scalability in a scoring service is mostly a matter of keeping the service *stateless*, so that adding capacity means adding identical replicas behind a load balancer (Figure 11.33). State lives in the feature store and the decision log, never in a replica's memory, so any replica can serve any request and a replica can be destroyed without consequence. Autoscaling then reacts to the month-end peak automatically, using the sizing rule of Chapter 10: $c \geq \lceil \lambda \bar{S} / \rho_{\max} \rceil$, which takes PesaCred from five replicas at 120 requests per second to seventeen at 400.

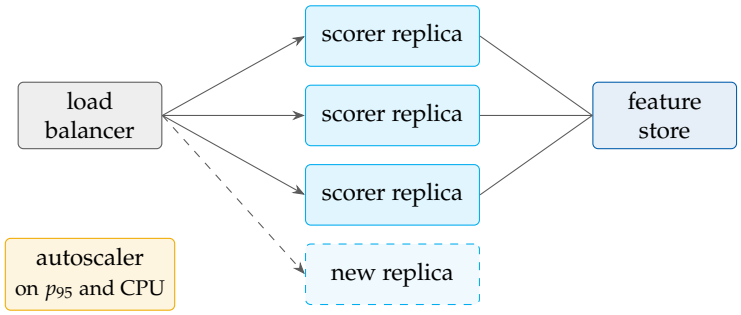


Figure 11.33: Stateless replicas behind a load balancer. State lives in the store, not in the service.

The feature store deserves a second mention here, because it solves a monitoring problem as well as a serving one (Figure 11.34). One definition of each feature is materialised into two stores: an offline store holding history, used for training and for building reference windows, and an online store holding the latest value, used for serving. Because both come from the same definition, drift is computed on exactly the values the model received, and the point-in-time joins that the offline store supports prevent the label leakage of Chapter 2 when training sets are rebuilt.

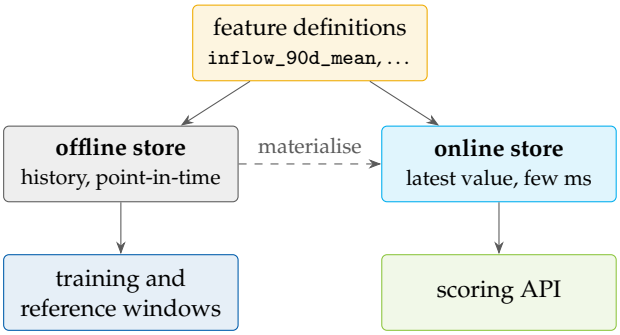


Figure 11.34: One definition, two stores. Training, serving and monitoring see the same numbers.

11.6.3 Graceful degradation

Reliability in a system with this many parts is not the absence of failure but the presence of a plan. A *degradation ladder* states in advance what the service does when each component becomes unavailable or untrustworthy (Figure 11.35). If the champion model fails its startup validation, or its drift gate has been red for a week, step down to the previous registered version. If the model cannot be loaded at all, fall back to a simple rule-based scorecard maintained for exactly this purpose. If the feature store times out,

refer the application to a human officer with an SMS promise of a decision. The last rung always *fails closed*: it never approves automatically.

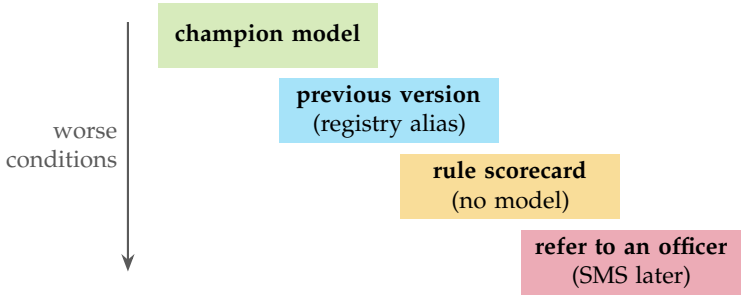


Figure 11.35: A degradation ladder. Every rung is implemented and tested in CI, like any other code path.

11.6.4 Safe rollout under delayed labels

Chapter 10 introduced shadow, canary and blue–green deployment. Monitoring is what makes them work, and the feedback delay is what decides their order (Figure 11.36). A credit model's canary cannot be judged on defaults, because none have matured; it is judged on the proxies that this chapter has been building: the score distribution, the approval rate, latency, error rates and the fairness slices of Section 11.7.1, all compared against the champion on the same traffic. Shadow deployment comes first precisely because it produces those comparisons on *every* request while exposing no borrower to the new model's decisions.

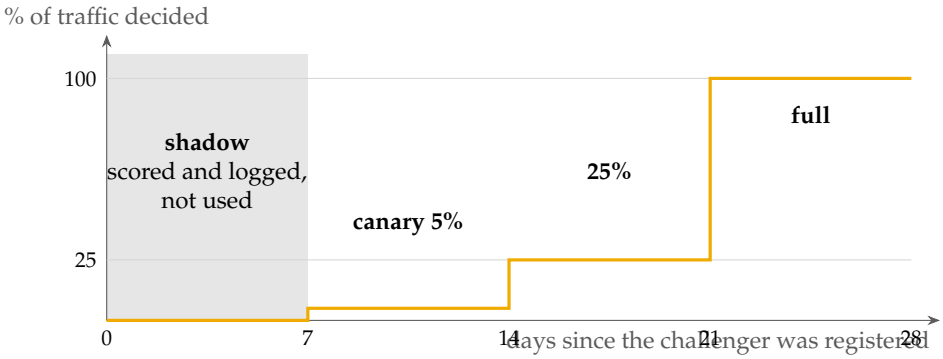


Figure 11.36: A staged rollout judged on proxies, because defaults will not mature for months.

Rollback deserves to be rehearsed rather than improvised. It is the alias move of Chapter 10: repoint `@champion` at the previous version, let the delivery pipeline redeploy, and confirm on the dashboard that the approval

rate has returned to its previous level. Because the decision log stores a model version on every row, the set of borrowers decided by the withdrawn version is a single query, and their cases can be reviewed if the eventual outcomes justify it.

11.6.5 Feedback loops and selective labels

One source of drift deserves separate treatment because the system causes it. A lender observes repayment only for the applicants it approved. The declined ones disappear without an outcome, so next year's training data describes $P(Y \mid X, \text{approved})$ rather than $P(Y \mid X)$ (Figure 11.37). This is the *selective labels* problem (Lakkaraju et al., 2017), and its effects compound: a model that is unnecessarily harsh on a segment stops receiving data about that segment, which makes it harder to discover the mistake, which keeps the segment excluded.

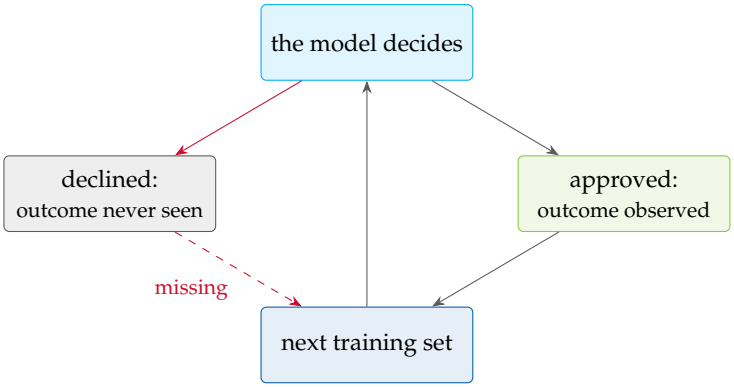


Figure 11.37: Selective labels. The model decides which outcomes it will ever be allowed to learn from.

Three partial remedies are standard. A small randomised *exploration* budget approves a fraction of marginal declines, buying unbiased outcomes at a known, budgeted cost. *Reject inference*, long used in credit scoring, models the missing outcomes explicitly, with assumptions that must be stated and revisited. And human review of a sample of declines produces expert judgements where outcomes are unavailable. Whatever the mix, the important point is that the choice is a deliberate one: a system with no exploration at all has decided, implicitly, to stop learning about the population it rejects.

11.6.6 Designing for Kenyan infrastructure

Constraints are design inputs. Connectivity is intermittent and USSD sessions time out, which means retries, which means that every write in the

request path must be idempotent: retrying a scoring request must not create a second loan. Data residency obligations under the Data Protection Act make it simpler to keep the decision log, the monitoring database and the artefact store in-country. Cost pressure favours CPU models and self-hosted infrastructure, and a monitoring stack that is too expensive to run will be switched off, which is the worst of both worlds. Power and link interruptions favour jobs that are resumable, images that are small, and a local registry mirror.

Kenya Context

The monitoring stack in this chapter runs comfortably on one modest self-hosted server in Nairobi: Evidently as a scheduled job, Prometheus and Grafana for operational metrics, PostgreSQL for the decision log, and the MLflow server of Chapter 10 for lineage. Keeping it in-country keeps cross-border transfer obligations simple, keeps latency low, and keeps the monthly cost within what a small lender can defend to its board. Schedule the heavy jobs off-peak, which is why the nightly drift check of Listing 11.6 runs at five in the morning, Nairobi time.

11.7 Responsible ML and governance

11.7.1 Fairness can drift

A model that satisfied a fairness criterion at launch need not satisfy it a year later, and the aggregate metrics of Section 11.5 will not reveal the change. Figure 11.38 shows the pattern: a drought depresses mobile-money inflows in one region first, the model reads lower inflows as higher risk, and the approval rate for good borrowers in that region falls by nineteen points while the national average barely moves.

Deciding what counts as unfair requires choosing a criterion, and the two in common use disagree by construction (Figure 11.39). *Demographic parity* requires equal approval rates across groups, $P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b)$. *Equal opportunity* requires equal approval rates among those who would in fact repay, $P(\hat{Y} = 1 \mid Y = 0, A = a) = P(\hat{Y} = 1 \mid Y = 0, A = b)$ (Hardt et al., 2016). When the base rates genuinely differ between groups, a calibrated model cannot satisfy both, and cannot satisfy either together with equal error rates (Chouldechova, 2017). There is no way to avoid the choice; there is only the choice between making it explicitly, with the risk committee's signature, and making it by accident.

Whichever criterion is chosen, monitoring it means computing it per slice, over time, with an honest treatment of small samples (Listing 11.9). A region with thirty applicants in a week will swing ten points by chance alone, so small slices are reported with their sample size and their interval,

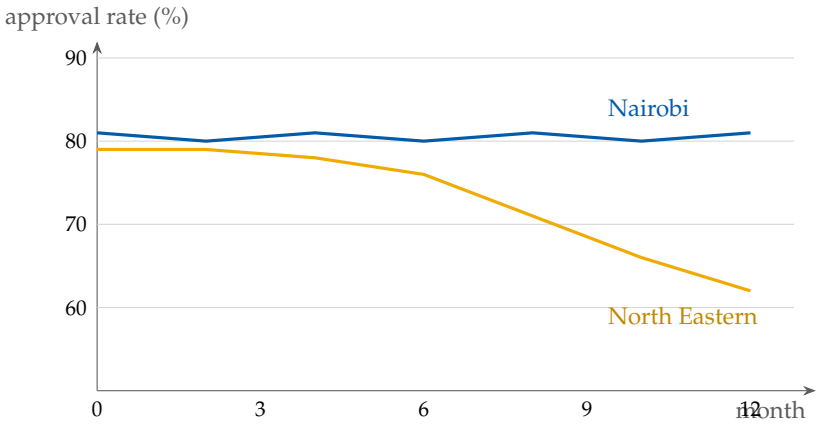


Figure 11.38: Fairness drift. The national approval rate is flat; one region's has collapsed.

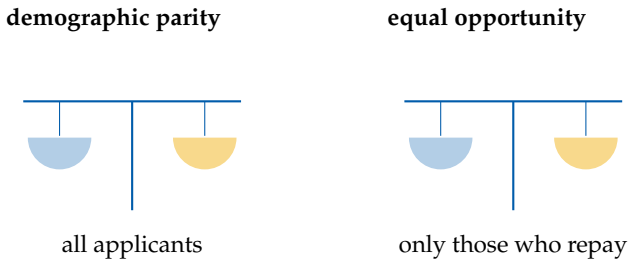


Figure 11.39: Two fairness criteria, comparing two different populations.

and alarmed on only when the interval supports it.

```
import pandas as pd

def slice_report(log: pd.DataFrame, by: str, min_n: int = 200) -> pd.DataFrame:
    g = log.groupby(by)
    out = pd.DataFrame({"n":        g.size(),
                        "approval": g["decision"].apply(lambda d: (d == "approve").
     mean()),
                        "mean_pd":  g["pd"].mean()})
    out["gap_pp"] = 100 * (out["approval"] - out["approval"].max())   # vs best-
     served group
    return out[out["n"] >= min_n]           # smaller slices: report, do not alarm

weekly = slice_report(decision_log_df, by="region")
alerts = weekly[weekly["gap_pp"] < -10]     # [PLACEHOLDER] policy threshold
```

Listing 11.9: Slice monitoring with a minimum-sample guard.

Pitfall

Alarming on tiny slices The narrower the slice, the wider its confidence interval, so a rule that alarms on point estimates will fire mostly on the smallest groups, which are often exactly the under-served ones. Report n next to every rate, use a confidence bound rather than a point estimate for the alarm, and remember who bears the cost of each error: a false alarm delays a release, while a missed one leaves borrowers in an under-banked region wrongly declined for months.

11.7.2 Governance as a by-product

The chapter's machinery was built for engineering reasons, and it produces most of what a Kenyan regulator asks for. Table 11.3 maps each obligation to the artefact that the pipeline already emits. The mapping is worth writing down explicitly, because it converts compliance from a periodic project into a property of the system, and because it reveals gaps: an obligation with no artefact beside it is a question that will have to be answered by hand, under time pressure, by someone who was not there.

11.7.3 Model cards

Documentation that is written once goes stale within a quarter. A *model card* avoids that fate by being generated, not written: the promotion pipeline of Chapter 10 renders it from the MLflow run's tags and evaluation outputs every time a version is registered (Mitchell et al., 2019). Its sections are familiar by now (Figure 11.40): the intended use and, as importantly, the out-of-scope uses; the data snapshot and its hash; headline metrics including calibration; performance on the slices that the fairness policy names; and the monitoring plan, meaning which quantities are watched, against

Obligation	Artefact the pipeline already emits
DPA 2019 s.25: data minimisation	hashed MSISDN; no raw identifiers in the decision log
DPA 2019 s.35: automated decisions	the `refer` path, the human review queue, reason codes
CBK DCP 2022: complaint handling	`request_id` audit chain, model version on every decision
Model risk management	MLflow lineage, promotion gates, archived drift reports
Fairness commitments	the weekly slice report, signed off by the risk owner

Table 11.3: Obligations and the artefacts that satisfy them. Compliance as a by-product of a well-built pipeline.

which reference, at which thresholds, by which named owner. That last section is what makes the card a living document rather than a historical one, and it is the natural place to record the policy overrides of Listing 11.4.

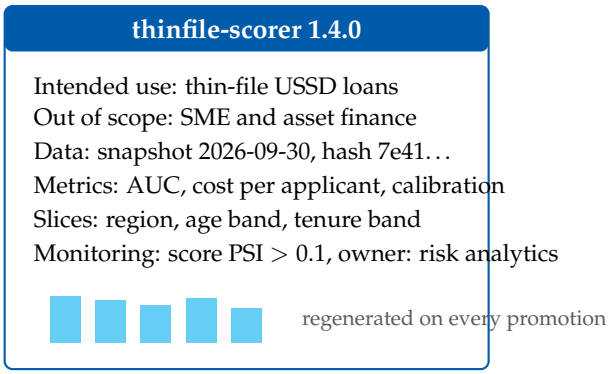


Figure 11.40: A generated model card. Its monitoring section is the contract this chapter implements.

11.8 Summary

Deployed models fail silently, because a wrong model and a right one are indistinguishable from the outside: both return well-formed answers quickly. Monitoring exists to make the difference visible, and it must watch the data and the predictions, not only the server.

Distribution shift comes in three kinds, and the distinction is practical rather than pedantic. Covariate shift moves $P(X)$ and is visible without labels, and often needs no action beyond watching coverage. Label shift moves $P(Y)$ and has a clean remedy, prior correction, which is cheaper and safer than retraining. Concept drift moves $P(Y \mid X)$, is invisible in the

inputs, and is the only one that truly requires new labelled data. Before invoking any of them, eliminate the data-pipeline bug, which is more common than all three.

Detection compares a reference window with a current one. Tests such as Kolmogorov–Smirnov are appropriate for small windows; on production-sized windows their p-values collapse for differences too small to matter, so alerts are built on effect sizes: the Wasserstein distance, the population stability index with its attributable per-bin contributions, the Jensen–Shannon distance for categories, and a domain classifier for joint shifts that per-feature tests miss. Monitoring many features multiplies false alarms, so corrections, drift-share rules and importance-weighting are part of the design rather than refinements to it. Evidently turns all of this into one run that produces an HTML report for a person and an exit code for a pipeline, and the failing branch of that gate produces a retraining *candidate*, never a deployment.

Production monitoring has four layers, from operational metrics in seconds to performance on matured labels in months, and in lending the last of those arrives about ninety days after the decision. Proxies, label-free drift detection and calibrated performance estimation fill the gap, with the caveat that the estimate is blind to exactly the drift that most needs catching. Every layer reads from the decision log, which is therefore designed with monitoring, joining and data protection in mind from the first line. Alerts name a threshold, an owner and a runbook, and triage asks the cheap questions first.

Around the model, design decides whether a drift alert is routine. Stateless replicas and a feature store give scalability and consistency between training, serving and monitoring; a degradation ladder that fails closed gives reliability; shadow and canary deployment judged on proxies gives safe rollout under delayed labels; and a deliberate exploration policy keeps the selective-labels loop from narrowing the model's view of the world. Fairness is monitored per slice over time, with a criterion chosen explicitly and small samples treated honestly, and the artefacts that all of this produces are, almost exactly, what the Data Protection Act and the CBK's digital credit rules require.

That closes the loop that Chapter 10 opened, and with it the arc of the book: from a model that fits, through a model that serves, to a system that knows when it has stopped being right.

Exercises

1. **Prior correction after a drought.** PesaCred's default rate rises from $\pi = 4\%$ to $\pi' = 7\%$, and the scorer's cost-optimal threshold is $t^\star = 0.074$. (a) Compute the likelihood-ratio weight w of Proposition 11.1.

(b) An applicant scored $p = 0.05$ by the deployed model: what is the corrected probability $\tilde{p}$, and does the decision change? (c) Compute the equivalent threshold t' on the uncorrected scores using Equation (11.3), and explain why moving the threshold is preferable to retraining in this situation.

2. **Reading a PSI.** The reference deciles of monthly inflow each hold 10% of applicants. The current window holds, from the lowest decile to the highest: 0.191, 0.141, 0.122, 0.101, 0.092, 0.089, 0.084, 0.069, 0.063, 0.048. (a) Compute the PSI and classify it on the scale of Figure 11.16. (b) Compute the contribution of the first and last deciles, and state what fraction of the total they account for. (c) Describe in one sentence what has happened to the applicant population, and say which of the three drift types of Definition 11.1 this evidence alone can and cannot establish.

3. **The large-sample trap.** A feature has shifted by 0.02 standard deviations, giving an empirical KS statistic of about $D = 0.02$. The 5% critical value for two windows of n observations each is approximately $1.36\sqrt{2/n}$. (a) Compute the critical value for $n = 2{,}000$ and for $n = 50{,}000$, and say in each case whether the test rejects. (b) Find the smallest n at which this shift becomes "significant". (c) PesaCred scores about 1,800 applications a day: how long a window does that correspond to, and what does the answer imply for a monitoring rule based on KS p-values?

4. **Alert budgets.** A monitoring job tests 40 features every night at $\alpha = 0.05$. (a) Compute the probability of at least one false alarm on a stable day, and the expected number of false alarms per night and per 30-day month. (b) Repeat with a Bonferroni correction. (c) Under a drift-share rule that alarms only when at least 30% of columns drift, how many columns must drift, and what is the probability of that happening by chance on a stable day? Use the binomial distribution with $m = 40$ and $p = 0.05$. (d) Which of the three designs would you deploy, and why?

5. **Interpreting a domain classifier.** A domain classifier separating last week's applicants from the reference window obtains a cross-validated AUC of 0.62, while every per-feature PSI is below 0.05. (a) What does the discrepancy suggest? (b) For two Gaussian populations differing by δ standard deviations in a single feature, the classifier's AUC is $\Phi(\delta/\sqrt{2})$; what δ would produce an AUC of 0.62 on its own? (c) List two diagnostics you would run next, and one mistake in the classifier's construction that could produce a spuriously high AUC.

6. **The cost of the feedback delay.** PesaCred decides 1,800 applications a day. Concept drift begins on day 0. Score PSI crosses its threshold on day 7; the AUC on matured labels cannot show anything until day 90. (a) How many decisions are made in each interval? (b) Suppose the drift raises the expected cost per applicant from KES 223 to KES 268. What is the total extra cost if the team acts on the day-7 signal and retrains within a week, and what if it waits for matured labels? (c) Name two signals that would have arrived before day 7, and state what each would and would not have told you.

7. **Estimating cost without labels.** Ten applicants receive calibrated probabilities 0.005, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.09, 0.15, 0.40, with $t^\star = 0.074$, $C_{FN} = 10{,}000$ and $C_{FP} = 800$. (a) State the decisions and compute the estimated cost per applicant from Equation (11.9). (b) A shift multiplies every applicant's true odds of default by 1.5 while the model's scores stay unchanged. Recompute the true probabilities, the decisions the unchanged model makes, and the realised expected cost. (c) Explain why the estimate of part (a) would not have revealed the shift of part (b), and which monitor would.

8. **Fairness gates on small slices.** In one week, 1,820 of 2,000 good borrowers in Nairobi were approved, and 95 of 120 in North Eastern. (a) Compute both approval rates and the gap. (b) Compute the 95% Wilson interval for each rate; the bounds are $\left(\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}\right) / \left(1 + \frac{z^2}{n}\right)$. (c) Give the range of gaps consistent with those intervals, and argue for a monitoring rule, bearing in mind who bears the cost of each kind of error.

9. **Triage.** For each of the following, state the most likely cause, the diagnostic you would run next, and the remedy. (a) Every numerical feature's PSI exceeds 1.0 overnight, and the out-of-vocabulary share for region jumps to 9%. (b) The score PSI is 0.02 and all input PSIs are below 0.03, but the AUC on labels that matured this month has fallen from 0.81 to 0.74. (c) Input PSIs rise steadily over six weeks, the approval rate falls by 4 points, and the estimated cost of Equation (11.9) is flat. (d) The approval rate falls by 6 points on the first Monday of September and recovers by mid-October.

10. **Designing the monitoring plan.** You are asked to write the monitoring section of the model card for a new version of the scorer. (a) List the quantities you would monitor at each of the four layers of Figure 11.24, with a reference, a statistic, a threshold and an owner for each. (b) State which two of them would trigger retraining automatically and which would only open a ticket, and justify the split. (c) Describe the degradation ladder you would implement, and explain

what must be true of the bottom rung. (d) Name one quantity you deliberately chose *not* to monitor, and say why.

Key terms

Silent failure; distribution shift; covariate shift; label shift; prior correction; concept drift; data-pipeline bug; feedback loop; sudden, gradual, incremental and recurring drift; drift calendar; reference window; current window; two-sample test; Kolmogorov–Smirnov statistic; effect size; Wasserstein distance; population stability index; bin edges; Jensen–Shannon distance; out-of-vocabulary share; domain classifier; prediction drift; approval rate; family-wise error rate; Bonferroni correction; drift share; Evidently; data definition; snapshot; drift gate; monitoring layers; operational metrics; data quality; feedback delay; label maturity; proxy label; confidence-based performance estimation; decision log; Prometheus; alert fatigue; runbook; triage; scheduled, triggered and continual retraining; agent trace; reliability; scalability; maintainability; adaptability; stateless replica; feature store; point-in-time join; graceful degradation; fail closed; shadow and canary deployment; rollback; selective labels; exploration budget; reject inference; demographic parity; equal opportunity; slice monitoring; Wilson interval; model card; audit chain.

Further reading

The natural companion to this chapter is Huyen (2022), whose chapters on data distribution shifts and monitoring, on continual learning and testing in production, and on responsible AI cover the same ground with a wider range of industrial examples; read them alongside Sections 11.2 and 11.5. Géron (2019) treats the operational side briefly but usefully in its section on launching, monitoring and maintaining a system, at the end of the end-to-end project chapter that opened this book.

The research literature on drift is large and older than the current interest in it. Gama et al. (2014) is the standard survey of concept drift and its adaptation methods, and its taxonomy of temporal patterns is the source of Figure 11.9. Rabanser et al. (2019) evaluates dataset-shift detectors empirically and makes the case for the domain classifier used in Section 11.3.8. Lipton et al. (2018) and Saerens et al. (2002) develop the label-shift correction of Proposition 11.1, the former with an estimator for the new prior that does not require labels. For the credit-industry view of population stability, Siddiqi (2006) remains the practical reference.

On the responsible and governance side, Hardt et al. (2016) introduces equal opportunity, Chouldechova (2017) proves the impossibility result that forces an explicit choice of criterion, Lakkaraju et al. (2017) formalises

the selective-labels problem, and Mitchell et al. (2019) proposes the model card. Finally, the tooling documentation is worth reading directly: Evidently (Evidently AI, 2025) for the API used in Section 11.4, and the Prometheus and Grafana documentation for the operational layer. The regulatory framing rests on primary sources, the Data Protection Act 2019 (Republic of Kenya, 2019) and the CBK Digital Credit Providers Regulations 2022 (Central Bank of Kenya, 2022).

Appendices

APPENDIX A

Capstone Project

APPENDIX B

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument, framing before formalism, business units before metrics, honesty caveats stated explicitly, that the assignments and the capstone are graded against.

Chapter 1: Introduction to Machine Learning

Solution to Exercise 1 (Framing). **(a) Streaming fraud detection with a 300-alerts/day operations team.**

Mitchell specification.

- T: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- E: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled "legitimate").
- P: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@k with $k = 300$/day, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction scoring latency

in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of Section 1.4.

(b) Thirty-day readmission from clinical pathway data.
Mitchell specification.

- T: at discharge, estimate the probability that the patient is readmitted within 30 days.
- E: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- P: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR–AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as "no readmission".

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model's own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of Section 1.4.

Solution to Exercise 2 (ML or not).

(a) Blocking transactions above a regulatory limit: rule. The requirement *is* a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of Table 1.1 on every criterion.

(b) SIM-swap account takeover: ML, wrapped in rules. The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion.

The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the "rules first, ML second" pattern of Section 1.2.3.

(c) Routing support tickets by language: ML, but off the shelf. Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: "use ML" does not imply "train your own" — the total cost-of-ownership argument of Table 1.1 favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c, the empirical risk is

$$J(c) \;=\; \frac{1}{n}\sum_{i=1}^{n}(y_i - c)^2, \qquad J'(c) \;=\; -\frac{2}{n}\sum_{i=1}^{n}(y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n}\sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n}\sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i,

$$G'(c) \;=\; \frac{1}{n}\Big(\#\{i : y_i < c\} \;-\; \#\{i : y_i > c\}\Big),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even n, anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to Section 1.6.2. Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between Eq. (1.4) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p}\, c_{\text{FN}}, \qquad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p})\, c_{\text{FP}}.$$

Decline exactly when $\hat{p}\, c_{\text{FN}} > (1 - \hat{p})\, c_{\text{FP}}$; rearranging,

$$\hat{p} \;>\; \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} \;=\; t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p}\, c_{\text{FN}}$ (approve), $(1-\hat{p})\, c_{\text{FP}}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p}\, c_{\text{FN}} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{\text{FN}}$;
- decline beats refer iff $(1-\hat{p})\, c_{\text{FP}} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{\text{FP}}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \le \hat{p} \le t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R \;<\; \frac{c_{\text{FN}}\, c_{\text{FP}}}{c_{\text{FN}} + c_{\text{FP}}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the "$\hat{p}$ near t^* routes to a human" edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) Number of repayment reminders sent: target leakage. Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) Median mobile-money balance over the 90 days *before* application: safe — *conditional* on discipline. The window $[t_0 - 90\text{d},\, t_0)$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) Standard deviation of balance over the loan term: temporal leakage. The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage. Repeat merchants appear on both sides of a random split, so the identifier

lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S; $\hat{f}$ is trained on S; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\,\mathbb{E}[\varepsilon\,(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}\big[\varepsilon\,(g - \hat{f})\big] \;=\; \mathbb{E}[\varepsilon]\;\mathbb{E}_S\big[g - \hat{f}\big] \;=\; 0 \cdot \mathbb{E}_S\big[g - \hat{f}\big] \;=\; 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S\big[(g - \hat{f})^2\big] = (g - \bar{f})^2 + 2\,(g - \bar{f})\,\mathbb{E}_S\big[\bar{f} - \hat{f}\big] + \mathbb{E}_S\big[(\bar{f} - \hat{f})^2\big].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] \;=\; 0$. The cross-term vanishes, leaving $\mathrm{Bias}^2 + \mathrm{Variance} + \sigma^2$, which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S-average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
```

```
drift = 2.5 * t                                  # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                           class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[:, 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                               # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
           .predict_proba(X[cut:])[:, 1])

print(f"random   split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")
```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one**; the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \le t$ for a single global t.

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t, i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t, parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B's approval rate is strictly lower at every useful threshold. Even in contrived

cases where the CDFs cross at some t, the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in Section 1.11.

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through Eq. (1.6)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as Section 1.11 prescribes.

Solution to Exercise 9 (Hybrid architecture).

A full-marks answer reproduces the topology of Fig. 1.9 with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline (Listing 1.5), served behind the versioned API of Listing 1.7. It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.

- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.

- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent's brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The

agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of Eq. (1.6), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M \mid X) = P(M)$ (Definition 2.1). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself (Definition 2.2). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception: $P(M \mid X)$ depends on X_{mis} (Definition 2.3). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per Remark 2.1, the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R \mid X) = P(R)$,

i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x \mid R = 1) \;=\; \frac{P(R = 1 \mid X = x)\, f(x)}{P(R = 1)} \;=\; \frac{P(R = 1)\, f(x)}{P(R = 1)} \;=\; f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{\mathrm{cc}} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E}\left[\frac{1}{n_{\mathrm{cc}}} \sum_{i:R_i=1} X_i \,\middle|\, n_{\mathrm{cc}} > 0\right] \;=\; \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \ldots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. □

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\widehat{\mathbb{E}}[X_{\mathrm{mis}} \mid X_{\mathrm{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\mathrm{Var}(X) \;=\; \underbrace{\mathbb{E}\big[\,\mathrm{Var}(X \mid X_{\mathrm{obs}})\big]}_{\text{residual spread}} \;+\; \underbrace{\mathrm{Var}\left(\mathbb{E}[X \mid X_{\mathrm{obs}}]\right)}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\mathrm{Var}(\mathbb{E}[X \mid X_{\mathrm{obs}}])$, understating $\mathrm{Var}(X)$ by the entire residual component $\mathbb{E}[\mathrm{Var}(X \mid X_{\mathrm{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t-test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p-values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) The outliers were the signal. In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in Section 2.3).

Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) The analyst created a training–serving mismatch. Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of Section 2.6. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score's μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} \;=\; \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} \;=\; 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be "detected" in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min})/(x_{\max} - x_{\min})$. Replace the maximum by $10\,x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' \;=\; \frac{m - x_{\min}}{10\,x_{\max} - x_{\min}}, \qquad \frac{m''}{m'} \;=\; \frac{x_{\max} - x_{\min}}{10\,x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median's scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of Eq. (2.8).

Solution to Exercise 2.6. Within level c, model each target as conditionally Bernoulli, $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \;\sim\; \text{Beta}(\alpha, \beta), \qquad \alpha = m\,\bar{y}, \quad \beta = m\,(1 - \bar{y}),$$

so that the prior mean is $\alpha/(\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c,\ \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] \;=\; \frac{\alpha + s_c}{\alpha + \beta + n_c} \;=\; \frac{m\,\bar{y} + n_c\,\bar{y}_c}{m + n_c} \;=\; \text{TE}(c),$$

which is Eq. (2.11) exactly. The limits confirm the shrinkage behaviour: as $n_c \to 0$, $\mathrm{TE}(c) \to \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \to \infty$, $\mathrm{TE}(c) \to \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must "outweigh" before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label (Section 2.5.3). □

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs "has this contract already terminated?" — which *is* the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of Section 2.7.2, hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of Section 2.7 to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model's. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns "end date $<$ snapshot"). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of Eq. (2.10) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\#\text{collisions}] \;=\; \binom{K}{2}\frac{1}{d} \;=\; \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} \;=\; \frac{99\,990\,000}{512} \;\approx\; 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12}–2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4-column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to "now" in the training warehouse) rather than to the scoring timestamp t, the feature ingests post-t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer's baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7\,\mathrm{d},\ t)$, enforced by the feature platform's point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from "implausible" to "modest" if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native `NaN` routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2's MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (Chapter 6), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training–serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the `Pipeline` of Listing 2.12 is a deployable contract, while "the library handles it" is not. Fifth, most of Lab 2's effort — parsing, validity rules, entity resolution, leakage

hunting — is not imputation at all, and no amount of native `NaN` support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: *"Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it."* The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association, not a causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the "effect" of late-night activity — can change substantially. It licenses no intervention claim ("discouraging late-night transactions would reduce defaults" does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio's 3% base rate the two are numerically close (OR $\approx$ RR when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1+e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)} = \sigma(z)\big(1-\sigma(z)\big).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \boldsymbol{\theta}^\top \mathbf{x}$:

$$\ell(\boldsymbol{\theta}) = -\,y \log \hat{p} - (1-y)\log(1-\hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\,\frac{y}{\hat{p}}\,\hat{p}(1-\hat{p}) + \frac{1-y}{1-\hat{p}}\,\hat{p}(1-\hat{p}) = -\,y(1-\hat{p}) + (1-y)\,\hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \boldsymbol{\theta} = \mathbf{x}$, $\nabla_{\boldsymbol{\theta}} \ell = (\hat{p} - y)\,\mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\boldsymbol{\theta}} J = \frac{1}{m}\sum_i (\hat{p}_i - y_i)\,\mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1-\hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in Chapter 6.

Solution to Exercise 3.3. From Eq. (3.11), $F_\beta = (1+\beta^2)PR \,/\, (\beta^2 P + R)$.

Limit $\beta \to 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \to \infty$: divide numerator and denominator by β^2:

$$F_\beta = \frac{(1+\beta^{-2})\,PR}{P+\beta^{-2}R} \xrightarrow[\beta\to\infty]{} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1+4)\cdot 0.40\cdot 0.75}{4\cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2\cdot 0.3/1.15 \approx 0.522$ because F_2 rewards this classifier's recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p\,c_{TP} + (1-p)\,c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p\,c_{FN} + (1-p)\,c_{TN}.$$

Flagging is optimal iff $p\,c_{TP} + (1-p)\,c_{FP} \;\le\; p\,c_{FN} + (1-p)\,c_{TN}$, i.e.

$$(1-p)\,(c_{FP} - c_{TN}) \;\le\; p\,(c_{FN} - c_{TP}).$$

Under the "reasonableness" conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* \;=\; \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP,\text{new}} - c_{FP,\text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from $(0,0)$: each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j. Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} \,/\, (m_+ m_-)$. Summing over all negatives,

$$\mathrm{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}\big[s_i^+ > s_j^-\big] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is Proposition 3.2.

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\mathrm{AUC} = P(s^+ > s^-) + \frac{1}{2} P(s^+ = s^-)$ of Eq. (3.15).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\mathrm{Precision}(t)] = P\big(y = 1 \mid \mathbf{x} \in A\big) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR "curve" of a label-independent scorer is therefore the horizontal line at height π, and its area (average precision) is

$$\mathrm{AP} = \int_0^1 \pi \, d(\mathrm{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03. Reporting "PR–AUC $= 0.31$" without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of Section 3.4.3 — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From Eq. (3.17), $\mathrm{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\mathrm{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525 / 0.03 = 17{,}500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of Section 3.6: on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter's toolkit: aggregate information across stratified folds (Definition 3.7), attach bootstrap intervals to whatever held-out estimate is reported (Section 3.6.4), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), Eq. (3.19) gives

$$\chi^2 = \frac{\big(|63 - 41| - 1\big)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B's advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models' accuracies by the same amount. McNemar's test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative (Section 3.3.1).

Solution to Exercise 3.9. Applying Eq. (3.20) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4\,\big[(0, 1, 0) - (1, 0, 0)\big] = (1, 0, 0) + 0.4\,(-1, 1, 0) = (0.6,\ 0.4,\ 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant's region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder's sum-to-one, single-one structure, and any coefficient interpretation (Example 3.1) built on "the region-A indicator" silently breaks.

SMOTE-NC's repair. SMOTE-NC interpolates only the *numeric* coordinates via Eq. (3.20), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as Section 3.7.2 already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w\,p}{w\,p + (1-p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w. Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \tfrac{1}{2} \iff w\,p \geq 1-p \iff p \geq \frac{1}{1+w} = \frac{c_{FP}}{c_{FP}+c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: "we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix" is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Chapter 4: Decision Trees, Ensembles and Hyperparameter Tuning

Solution to Exercise 4.1. *Independent voters.* With $p = 0.55$, the number of correct voters is $K \sim \text{Binomial}(M, 0.55)$ and the majority is correct when $K > M/2$. The normal approximation $K \approx \mathcal{N}\big(Mp,\ Mp(1-p)\big)$ is justified

for these M by the usual rule $Mp(1-p) \gg 1$ (it equals 2.7, 25.0 and 247.7 respectively; for $M = 11$ the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing $z = \big(M/2 - Mp\big)/\sqrt{Mp(1-p)}$ and $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$:

M	$-z$	P_{maj} (normal)	P_{maj} (exact)
11	0.332	0.630	0.633
101	1.005	0.843	0.846
1001	3.164	0.9992	0.9992

The committee converges on certainty, exactly as Proposition 4.1 promises.

Correlated blocks. If the M voters consist of ten blocks of $M/10$ perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever M is. For $M = 1001$ (≈ 100 clones per block) the relevant computation is the $M = 10$ (even; take $M = 11$ as the odd comparator) row, not the $M = 1001$ row: $P_{\text{maj}} \approx 0.63$, and no increase of M at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is Eq. (4.5) in disguise — within-block correlation $\rho = 1$ contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

Solution to Exercise 4.2. Let $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$, $\mathrm{Var}[f_m] = \sigma^2$ and pairwise correlation ρ. Then

$$\begin{aligned}\mathrm{Var}[\bar{f}\,] &= \sum_m c_m^2\sigma^2 + \sum_{m\neq m'} c_m c_{m'}\rho\sigma^2 \\ &= \sigma^2\Big[(1-\rho)\textstyle\sum_m c_m^2 + \rho\big(\sum_m c_m\big)^2\Big] = \sigma^2\Big[(1-\rho)\textstyle\sum_m c_m^2 + \rho\Big],\end{aligned}$$

using $\sum_{m\neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$. Since $\rho \leq 1$, minimising the variance means minimising $\sum_m c_m^2$ subject to $\sum_m c_m = 1$. By Cauchy–Schwarz (or a Lagrange multiplier), $\sum_m c_m^2 \geq \frac{1}{M}(\sum_m c_m)^2 = \frac{1}{M}$, with equality iff $c_m = 1/M$ for all m: *uniform weights are optimal*, and they recover Proposition 4.2.

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in Listing 4.2 where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances σ_m^2 (and $\rho = 0$ for clarity) gives optimal weights $c_m \propto 1/\sigma_m^2$: precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking (Section 4.8) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

Solution to Exercise 4.3. *The limit without logarithms.* From $1 - x \le e^{-x}$ with $x = \frac{1}{n}$,

$$\left(1 - \tfrac{1}{n}\right)^n \le e^{-1}.$$

For the matching lower bound, apply $1 - x \ge e^{-x/(1-x)}$ (valid for $0 \le x < 1$, and itself a rearrangement of $1 + u \le e^u$ with $u = x/(1-x)$) to get $\left(1 - \frac{1}{n}\right)^n \ge e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \to e^{-1}$. The two bounds pinch the sequence to e^{-1}. For $n = 20$, $\left(\frac{19}{20}\right)^{20} = 0.3585$, already within 2.5% of $e^{-1} \approx 0.3679$ — the asymptotics arrive early, which is why "37% out of bag" is a safe rule of thumb even for modest samples.

Why an OOB-chosen threshold misbehaves. (i) *Statistical*: each OOB prediction $\bar{f}_{-i}^{\text{oob}}(x_i)$ in Eq. (4.6) averages only the $\approx 0.37M$ trees that missed row i, whereas the deployed model averages all M. A mean of $0.37M$ trees is a noisier, differently calibrated score than a mean of M trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal*: the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold's economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

Solution to Exercise 4.4. After stage m, the new weight mass on the rows that f_m got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i:\, f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m}\, \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m}(1 - \varepsilon_m) W^{(m)},$$

where $W^{(m)} = \sum_i w_i^{(m)}$. Substituting $e^{\alpha_m} = \sqrt{(1-\varepsilon_m)/\varepsilon_m}$ from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\tfrac{1-\varepsilon_m}{\varepsilon_m}}\, \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m(1-\varepsilon_m)}\, W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of f_m on the updated weights is exactly $\frac{1}{2}$: each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

Solution to Exercise 4.5. *(a) Binary log-loss.* With $y \in \{0,1\}$, logits F, and $\sigma(F) = (1+e^{-F})^{-1}$,

$$L(y,F) = -\big[y \ln \sigma(F) + (1-y)\ln(1-\sigma(F))\big].$$

Using $\sigma'(F) = \sigma(F)(1-\sigma(F))$,

$$\frac{\partial L}{\partial F} = -\Big[y\frac{\sigma'}{\sigma} - (1-y)\frac{\sigma'}{1-\sigma}\Big] = -\big[y(1-\sigma) - (1-y)\sigma\big] = \sigma(F) - y,$$

so the pseudo-residual Eq. (4.7) is $r = y - \sigma(F)$: the error in predicted probability, as claimed in Section 4.6.3.

(b) Pinball loss. At quantile level $\tau \in (0,1)$,

$$L_\tau(y,F) = \begin{cases} \tau\,(y-F), & y \geq F, \\ (1-\tau)\,(F-y), & y < F, \end{cases} \qquad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at $y = F$; any subgradient value in $[\tau-1, \tau]$ serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes τ and $1-\tau$; the fixed point of that asymmetry is the τ-quantile.

Use in collections. A model of the $\tau = 0.9$ quantile of repayment delay answers, per borrower, "by what day will this loan have been repaid with 90% confidence?" That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent's call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as Chapter 3, applied to a regression target.

Solution to Exercise 4.6. Under squared error $L = \frac{1}{2}(y-F)^2$ we have $g_i = \partial_F L = F(x_i) - y_i = -r_i$ (minus the ordinary residual) and $h_i = \partial_F^2 L = 1$. Hence for leaf j, $G_j = -\sum_{i \in I_j} r_i$ and $H_j = |I_j|$, and Eq. (4.11) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow[\lambda = 0]{} \frac{1}{|I_j|}\sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting (Section 4.6.3) is recovered as the unregularised special case. For $\lambda > 0$,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j}\sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor $H_j/(H_j+\lambda) \in (0,1)$. The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding $H_j = 500$ loans with $\lambda = 5$ keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward "no adjustment" — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in Listing 4.4.

Solution to Exercise 4.7. *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` (x, in days), and suppose the unconstrained model's fitted risk is piecewise constant:

$$F(x) \;=\; \begin{cases} 0.10, & 0 \le x < 10, \\ 0.30, & 10 \le x < 20, \\ 0.20, & 20 \le x < 30, \\ 0.50, & x \ge 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of F on x has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at $x = 25$ than at $x = 15$.

Governance failure. Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender's own decisions: A's appeal, a regulator's file review, or a journalist's paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of Definition 4.5 removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in Section 4.7.3.

Solution to Exercise 4.8. By Proposition 4.5, an equal-resolution grid with $B = 729$ and $d = 6$ uses $r = 729^{1/6} = 3$ distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the $3^4 = 81$ settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

Probability the grid never enters the best decile of both important axes simultaneously. Model each axis as $[0,1]$ with the "best decile" an interval of length 0.1 whose location is unknown, and take the grid's three values per axis

to be evenly spaced with a uniformly random offset, $\{u, u+\frac{1}{3}, u+\frac{2}{3}\}$ with $u \sim \mathrm{U}[0, \frac{1}{3})$ (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval $0.1 < \frac{1}{3}$, at most one grid value can fall in it, and the probability that one does is $3 \times 0.1 = 0.3$ per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \qquad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability $0.1 \times 0.1 = 0.01$, so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

Solution to Exercise 4.9. With $N = 81$, $b = 25$ and $\eta = 3$, the rungs are:

Rung	1	2	3	4	5
Configurations alive	81	27	9	3	1
Budget each (rounds)	25	75	225	675	2025
Cost of rung	2025	2025	2025	2025	2025

Each rung costs exactly $81 \times 25 = 2025$ configuration-rounds — the $1/\eta$ survival rate and the $\times\eta$ budget growth cancel — so the total is $5 \times 2025 = 10{,}125$ configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost $81 \times 2025 = 164{,}025$: successive halving explores the same 81 candidates for 6.2% of the cost, a 16.2× saving, at the price of trusting rung-1 rankings.

Where the trust fails. The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with $\nu = 0.02$ is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a $\nu = 0.3$ configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low-ν candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

Solution to Exercise 4.10. Each measured score is $\widehat{\mathcal{V}}_t = v + \varepsilon_t$ with $\varepsilon_t = sZ_t$ and $Z_t \sim \mathcal{N}(0,1)$ i.i.d. The search reports

$$\max_t \widehat{\mathcal{V}}_t = v + s \max_t Z_t \quad \Longrightarrow \quad \mathbb{E}\big[\max_t \widehat{\mathcal{V}}_t\big] = v + s\,\mathbb{E}\big[\max_{t\le T} Z_t\big],$$

since v is common and the maximum commutes with the shift. All T configurations are *equally good* by construction, so the entire excess over v is optimism manufactured by selection. Using the standard extreme-value approximation $\mathbb{E}[\max_{t\le T} Z_t] \approx \sqrt{2\ln T}$: for $T = 100$, $\sqrt{2\ln 100} = \sqrt{9.21} \approx 3.03$, so with fold-level noise $s = 0.004$ in PR–AUC the expected reported best is

$$v + 0.004 \times 3.03 \;\approx\; v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly $+0.012$ PR–AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner's curse, and Table 4.7's tuned-model row would read ≈ 0.28 rather than 0.294 if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as $\sqrt{\ln T}$ (slowly — more trials are not catastrophic) but linearly in s, so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter's final pitfall box.

Chapter 5: Unsupervised Learning and Dimensionality Reduction

Solution to Exercise 5.1. Let $J^*(k)$ denote the optimal inertia with k clusters. Take an optimal solution for k with centroids $\mu_1, \dots, \mu_k$ and assignment c. Build a feasible $(k+1)$-cluster solution by choosing any point x_r that is not already alone in its cluster, adding the centroid $\mu_{k+1} = x_r$, and reassigning x_r to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of x_r falls from $\|x_r - \mu_{c(r)}\|^2 \ge 0$ to 0. Therefore $J^*(k+1) \le J^*(k)$. At $k = m$ we may place one centroid on each point, giving $J^*(m) = 0$.

Consequently "minimise inertia over k" always returns $k = m$, which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise J; it examines the *rate* at which J falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

Solution to Exercise 5.2. **Partition $C_1 = \{0,1,2\}$, $C_2 = \{10,11,12\}$.** By symmetry it suffices to do C_1.

- $x = 0$: $a = \frac{1+2}{2} = 1.5$; $b = \frac{10+11+12}{3} = 11$; $s = (11 - 1.5)/11 = 0.8636$.
- $x = 1$: $a = \frac{1+1}{2} = 1$; $b = \frac{9+10+11}{3} = 10$; $s = (10 - 1)/10 = 0.9000$.
- $x = 2$: $a = \frac{2+1}{2} = 1.5$; $b = \frac{8+9+10}{3} = 9$; $s = (9 - 1.5)/9 = 0.8333$.

C_2 mirrors these exactly, so $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$.

Partition $C_1' = \{0,1\}$, $C_2' = \{2,10,11,12\}$.

- $x = 0$: $a = 1$; $b = \frac{2+10+11+12}{4} = 8.75$; $s = (8.75 - 1)/8.75 = 0.8857$.
- $x = 1$: $a = 1$; $b = \frac{1+9+10+11}{4} = 7.75$; $s = 0.8710$.
- $x = 2$: $a = \frac{8+9+10}{3} = 9$; $b = \frac{2+1}{2} = 1.5$; $s = (1.5 - 9)/9 = -0.8333$.
- $x = 10$: $a = \frac{8+1+2}{3} = 3.6\overline{6}$; $b = \frac{10+9}{2} = 9.5$; $s = (9.5 - 3.667)/9.5 = 0.6140$.
- $x = 11$: $a = \frac{9+1+1}{3} = 3.6\overline{6}$; $b = 10.5$; $s = 0.6508$.
- $x = 12$: $a = \frac{10+2+1}{3} = 4.3\overline{3}$; $b = 11.5$; $s = 0.6232$.

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$.

Comment. The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value, $s(2) = -0.833$: the point $x = 2$ is on average much closer to C_1' than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that $\bar{s}$ for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

Solution to Exercise 5.3. Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

	P_1	P_2	P_3	P_4	P_5	P_6
x_1 (KES)	1000	1100	1200	9000	9100	9200
x_2 (count)	2	40	78	2	40	78

Raw data. Squared distances are dominated by x_1, whose spread is ~ 8000 against ~ 76 for x_2. K-means with $k = 2$ therefore splits on value: $\{P_1, P_2, P_3\}$ versus $\{P_4, P_5, P_6\}$. (Check: the within-cluster sum of squares for this split is $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$, whereas splitting on count gives clusters spanning 8000 in x_1 and costs $\sim 1.3 \times 10^8$.)

Standardised data. After centring and dividing by the sample standard deviations ($\sigma_1 \approx 4271$, $\sigma_2 \approx 34.9$), both axes have unit spread and the configuration becomes a symmetric 2×3 grid at $x_1 \approx \pm 0.94$ and $x_2 \approx \{-1.09, 0, 1.09\}$. The two candidate splits now cost the same by symmetry along x_1, but the three-level x_2 axis has the larger total spread (± 1.09 against ± 0.94), so K-means prefers to cut along x_2, returning e.g. $\{P_1, P_4\}$ (low count) against $\{P_2, P_3, P_5, P_6\}$. Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are "correct" for their respective metrics.

Which to defend. The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: "are these transactions large or small in KES?" is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

Solution to Exercise 5.4. **Construction.** Work in $\mathbb{R}^2$ with $\varepsilon = 1.05$ and `minPts` $= 8$. Build two clusters, each a dense blob plus one outlying "antenna" point:

- Cluster A: ten points inside a ball of radius 0.05 centred at $(-0.6, 0)$, plus a single point $p_A = (0, 0)$.
- Cluster B: ten points inside a ball of radius 0.05 centred at $(2.6, 0)$, plus a single point $p_B = (2, 0)$.
- The candidate border point $q = (1, 0)$.

Every blob point is core: it sees the other nine blob points (mutual distance ≤ 0.1) and its antenna (distance $0.6 \pm 0.05 \leq \varepsilon$), so $|N_\varepsilon| = 11 \geq 8$. *Each antenna is core*: p_A sees its blob (ten points at ≈ 0.6), itself, and q (distance exactly $1 \leq \varepsilon$), so $|N_\varepsilon(p_A)| = 12 \geq 8$. *$q$ is not core*: its ε-ball contains only q, p_A and p_B, since both blobs sit at distance $\approx 1.6 > \varepsilon$; thus $|N_\varepsilon(q)| = 3 < 8$. Finally A and B are distinct components, because $d(p_A, p_B) = 2 > \varepsilon$ and q — the only object between them — is not core, so it cannot propagate reachability.

Therefore q is a border point directly density-reachable from p_A and from p_B, which belong to different clusters. DBSCAN assigns q to whichever cluster is expanded first: process p_A before p_B and q joins A; reverse the visitation order and q joins B. Its label is genuinely order-dependent.

Why this does not undermine reproducibility. By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose ε-ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

Solution to Exercise 5.5. With $k = 2$, $\pi_1 = \pi_2 = \frac{1}{2}$, and shared variance σ^2, the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i-\mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i-\mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i-\mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1+\exp\left(\frac{(x_i-\mu_1)^2-(x_i-\mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares, $(x-\mu_1)^2 - (x-\mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1-\mu_2)\left(x - \frac{\mu_1+\mu_2}{2}\right)$. Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w\,(x_i - x_0)), \qquad w = \frac{\mu_1-\mu_2}{\sigma^2}, \qquad x_0 = \frac{\mu_1+\mu_2}{2},$$

where $\sigma_{\text{logistic}}(z) = (1+e^{-z})^{-1}$. So the responsibility is exactly a logistic function of x with **slope parameter** $w = (\mu_1-\mu_2)/\sigma^2$ and **midpoint** at the arithmetic mean of the two component means, where $\gamma_{i1} = \frac{1}{2}$.

As $\sigma^2 \to 0^+$ with $\mu_1 \neq \mu_2$ fixed, $|w| \to \infty$: the logistic curve steepens into a step at x_0, so $\gamma_{i1} \to \mathbb{1}\{x_i \text{ nearer } \mu_1\}$ for every $x_i \neq x_0$. This is precisely Proposition 5.5 in one dimension: the midpoint x_0 is the perpendicular bisector (here, a single point), and the soft assignment collapses to K-means' hard nearest-centroid rule. The general lesson is that σ^2 acts as a temperature: large σ^2 gives soft, hedged memberships; small σ^2 gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

Solution to Exercise 5.6. **Divergence.** Take $m \geq 2$ distinct points and the family

$$\theta_\epsilon: \quad \pi_1 = \pi_2 = \tfrac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x},\ \sigma_2^2 = s^2 > 0 \text{ fixed}.$$

The first component's density at x_1 is $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$, so the contribution of observation 1 to the log-likelihood is at least $\log\left(\frac{1}{2}(2\pi\epsilon)^{-1/2}\right) \to$

$+\infty$ as $\epsilon \to 0^+$. Every other observation $i \geq 2$ retains the second component's contribution, bounded below by $\log\big(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)\big) > -\infty$, which does not depend on ϵ. Hence $\ell(\theta_\epsilon) \to +\infty$: the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

Effect of `reg_covar`. Adding $\lambda > 0$ to the diagonal replaces σ_j^2 by $\sigma_j^2 + \lambda$, so the achievable density at a point is bounded by $(2\pi\lambda)^{-1/2}$ and the per-observation log-likelihood by $\log\big(\pi_{\max}(2\pi\lambda)^{-1/2}\big) < \infty$. Summing over m observations bounds ℓ above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

Price paid. Three costs. (i) The estimator is biased: every fitted covariance is inflated by λ, so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed* λ. (iii) λ is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep λ at the smallest value that prevents collapse (the scikit-learn default 10^{-6} on standardised data is usually right) and treat a fit that needs a large λ as evidence that k is too big.

Solution to Exercise 5.7. Free parameters split into weights, means and covariances. Weights contribute $k-1$ (they sum to one); means contribute kn. Covariances:

`covariance_type`	Covariance parameters	Total p
`full`	$k\,\frac{n(n+1)}{2}$	$(k-1) + kn + k\frac{n(n+1)}{2}$
`tied`	$\frac{n(n+1)}{2}$	$(k-1) + kn + \frac{n(n+1)}{2}$
`diag`	kn	$(k-1) + 2kn$
`spherical`	k	$(k-1) + kn + k$

For $n = 32$, $k = 5$: $k - 1 = 4$ and $kn = 160$.

- `full`: $\frac{32\cdot 33}{2} = 528$ per component, so $p = 4 + 160 + 5(528) = \mathbf{2804}$.
- `tied`: $p = 4 + 160 + 528 = 692$.
- `diag`: $p = 4 + 2(160) = \mathbf{324}$.
- `spherical`: $p = 4 + 160 + 5 = 169$.

With $m = 150{,}000$, $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$. For `full` to beat `diag` on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} \;<\; p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned}\ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \tfrac{1}{2}(2804 - 324)(11.9184) \\ &= \tfrac{1}{2}(2480)(11.9184) \approx \mathbf{14{,}778}.\end{aligned}$$

Averaged over the sample this is only $14{,}778/150{,}000 \approx 0.0985$ nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at $m = 2000$ and the required gap becomes ≈ 0.5 nats per observation, at which point `diag` usually wins. The general moral is that the "right" covariance type is a function of sample size, not of taste.

Solution to Exercise 5.8. **Independent coordinates.** A short simulation (numpy default generator, seed 0; $m = 1000$ points and one query point drawn from the same distribution) gives ratios of the order

n	2	10	100	1000
i.i.d. coordinates	121	3.04	0.44	0.12
5-factor latent model	179	10.4	8.88	7.55

(means over 20 replications; the $n = 2$ column is enormous and wildly variable because $D_{\min}$ can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like $n^{-1/2}$, exactly as Proposition 5.6 predicts from $\operatorname{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$.

Latent factor model. Generate $u_i \in \mathbb{R}^5$ i.i.d. standard normal and set $x_i = Au_i + 0.05\,\eta_i$ with $A \in \mathbb{R}^{n\times 5}$ fixed and η_i small isotropic noise (second row of the table). The ratio now stays around 8–10 from $n = 10$ all the way to $n = 1000$: raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

Explanation. The proof of Proposition 5.6 needs the coordinate contributions δ_t to be (near-)independent, so that $\operatorname{Var}[D_n^2]$ grows like n while $\mathbb{E}[D_n^2]$ grows like n, forcing the relative variance to 0. Under the factor model the δ_t are strongly dependent: the effective, or intrinsic, dimension is 5, not n, and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter's strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

Solution to Exercise 5.9. **Proof.** From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m}\sum_i \|x_i\|^2 - \operatorname{tr}(W^\top S W).$$

For centred data $\frac{1}{m}\sum_i \|x_i\|^2 = \operatorname{tr}(S) = \sum_{t=1}^{n} \lambda_t$. At the optimum $W = W_d$ we have, by Theorem 5.1, $\operatorname{tr}(W_d^\top S W_d) = \sum_{t \le d} \lambda_t$. Hence

$$E(W_d) = \sum_{t=1}^{n} \lambda_t - \sum_{t \le d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

Numerical verification. With $S = \left(\begin{smallmatrix} 4 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ from Example 5.2, $\lambda_1 = 6$, $\lambda_2 = 2$, $v_1 = \frac{1}{\sqrt{2}}(1,1)^\top$. For $d = 1$ the formula predicts $E = \lambda_2 = 2$. Take three centred points consistent with S, for instance

$$x_1 = (3,1)^\top, \quad x_2 = (-1,-3)^\top, \quad x_3 = (-2,2)^\top.$$

Projection: $z_i = v_1^\top x_i$ gives $z_1 = 4/\sqrt{2} = 2\sqrt{2}$, $z_2 = -4/\sqrt{2} = -2\sqrt{2}$, $z_3 = 0$. Reconstruction $\hat{x}_i = z_i v_1$: $\hat{x}_1 = (2,2)^\top$, $\hat{x}_2 = (-2,-2)^\top$, $\hat{x}_3 = (0,0)^\top$. Residuals: $x_1 - \hat{x}_1 = (1,-1)^\top$ with squared norm 2; $x_2 - \hat{x}_2 = (1,-1)^\top$, squared norm 2; $x_3 - \hat{x}_3 = (-2,2)^\top$, squared norm 8. Mean $= (2+2+8)/3 = 4$.

The mean over this particular triple is 4, not 2, because three points do not reproduce S: their own sample covariance is $\left(\begin{smallmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{smallmatrix}\right)$, whose second eigenvalue is $14/3 - 2/3 = 4$ — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that λ_t are properties of the sample you actually projected, not of an assumed population.

Solution to Exercise 5.10. A defensible one-page answer covers four things.

What to measure next. (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it is; high predictability means the "behavioural" representation substantially encodes geography. (iv) Outcome data where available — if this segment's historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

Instruments engaged. Both discussed in the chapter's regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject's rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable "cluster 3" limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

Two remedies with costs. (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of Chapter 4 with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

When shipping unchanged is defensible. If the disparity is *explained by* a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

Chapter 6: Deep Learning Fundamentals

Solution to Exercise 6.1. By Proposition 6.1, the three linear hidden layers collapse to a single affine map. Writing $\mathbf{W}^{(1)} \in \mathbb{R}^{128\times 40}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{64\times 128}$, $\mathbf{W}^{(3)} \in \mathbb{R}^{32\times 64}$ and $\mathbf{w}^{(4)} \in \mathbb{R}^{1\times 32}$, the network computes

$$\hat{p} = \sigma\left(\widetilde{\mathbf{w}}^{\top}\mathbf{x} + \tilde{b}\right), \qquad \widetilde{\mathbf{w}}^{\top} = \mathbf{w}^{(4)}\mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)} \in \mathbb{R}^{1\times 40}.$$

The model class is therefore **logistic regression on the original 40 features** — nothing more. Note in particular that the widths $128, 64, 32$ do not en-

large the class: the composite matrix has rank at most 1 regardless, because the output layer is one-dimensional.

Counting parameters layer by layer, using Remark 6.1:

$$\underbrace{41 \times 128}_{5248} + \underbrace{129 \times 64}_{8256} + \underbrace{65 \times 32}_{2080} + \underbrace{33 \times 1}_{33} \;=\; 15{,}617,$$

against $40 + 1 = 41$ for the equivalent logistic regression. The network therefore carries $15{,}617 - 41 = \mathbf{15{,}576}$ redundant parameters, roughly 381 times the minimum, while representing exactly the same set of functions.

Two qualifications are worth making, because the exercise is not *only* a counting exercise. First, redundant parameterisation is not free at training time: it multiplies memory traffic and gradient computation for no gain in expressiveness. Second, the over-parameterised form is not perfectly equivalent *as an optimisation problem*: the loss surface of a deep linear network is non-convex in $\boldsymbol{\theta}$ even though the induced function class is the convex-friendly logistic one, and the factorised parameterisation induces an implicit bias in the trajectory of gradient descent. The correct diagnosis remains that the architecture is a mistake: if a linear model is wanted, fit a linear model; if non-linearity is wanted, insert an activation function.

Solution to Exercise 6.2. **Part 1.** By Proposition 6.2, $\sigma'(z) = s(1-s)$ with $s = \sigma(z)$. Solving $s(1-s) = 0.01$, that is $s^2 - s + 0.01 = 0$, gives

$$s = \frac{1 \pm \sqrt{1 - 0.04}}{2} = \frac{1 \pm \sqrt{0.96}}{2} \;\in\; \{0.010102,\ 0.989898\}.$$

Since $s \mapsto s(1-s)$ is a downward parabola, $\sigma'(z) < 0.01$ precisely when s lies outside $[0.010102, 0.989898]$. Converting the upper root back to the logit scale,

$$z = \ln \frac{s}{1-s} = \ln \frac{0.989898}{0.010102} = \ln(97.99) = 4.5849,$$

and by the symmetry $\sigma(-z) = 1 - \sigma(z)$ the lower root gives -4.5849. Hence

$$\sigma'(z) < 0.01 \iff |z| > 4.585 \quad \text{(to three decimal places)}.$$

A pre-activation of magnitude only about 4.6 — entirely unremarkable for unstandardised inputs — already suppresses the gradient by a factor of at least 25 relative to its maximum of $1/4$.

Part 2. With $\|\mathbf{W}^{(l)}\| = 1$ and $\gamma = 1/4$, Proposition 6.10 gives the bound $(\gamma w)^{L-1} = 4^{-(L-1)}$. Requiring $4^{-(L-1)} \le 10^{-6}$,

$$(L-1)\ln 4 \ge 6 \ln 10 \quad\Longrightarrow\quad L - 1 \ge \frac{6 \times 2.302585}{1.386294} = 9.97,$$

so $L-1=10$ multiplicative factors suffice, that is a network of $\mathbf{L=11}$ layers. Ten sigmoid layers are enough to reduce the error signal reaching the first layer by six orders of magnitude *even when the weight matrices are perfectly scaled*. This is the quantitative content of the claim that deep sigmoid stacks do not train, and it is why Section 6.9 treats initialisation and Section 6.10 treats normalisation as necessities rather than refinements.

Solution to Exercise 6.3. Take the convention of Example 6.2, in which $\mathbf{z}^{(1)}=\mathbf{W}^{(1)\top}\mathbf{x}+\mathbf{b}^{(1)}$, and set $\mathbf{x}=(-0.5,\,1.2)^{\top}$:

$$z_1^{(1)}=1(-0.5)+0.5(1.2)=0.1, \qquad z_2^{(1)}=-2(-0.5)+1(1.2)-0.4=1.8,$$

so $\mathbf{z}^{(1)}=(0.1,\,1.8)^{\top}$ and, since both components are positive, $\mathbf{h}^{(1)}=\mathrm{ReLU}(\mathbf{z}^{(1)})=(0.1,\,1.8)^{\top}$. The output logit is

$$z^{(2)}=2(0.1)-1(1.8)-0.1=-1.7, \qquad \hat{p}=\sigma(-1.7)=\frac{1}{1+e^{1.7}}=0.15447.$$

With $y=1$ the loss is $\mathcal{L}=-\ln(0.15447)=1.868$, and by Proposition 6.5 the output error signal is

$$\delta^{(2)}=\hat{p}-y=0.15447-1=-0.84553.$$

It is negative, correctly instructing the network to *increase* $z^{(2)}$ for this borrower.

Applying (6.17) at the output layer,

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}^{(2)}}=\delta^{(2)}\mathbf{h}^{(1)}=-0.84553\begin{pmatrix}0.1\\1.8\end{pmatrix}=\begin{pmatrix}-0.08455\\-1.52196\end{pmatrix}, \quad \frac{\partial\mathcal{L}}{\partial b^{(2)}}=-0.84553.$$

The second component is eighteen times the first, purely because $h_2^{(1)}=1.8$ is eighteen times $h_1^{(1)}$: gradient magnitude at a weight is proportional to the activation it multiplies, which is the algebraic reason unscaled activations produce unbalanced updates.

Applying the recursion (6.16) with $\mathrm{ReLU}'(z)=1$ for $z>0$,

$$\boldsymbol{\delta}^{(1)}=\left(\mathbf{w}^{(2)}\delta^{(2)}\right)\odot\begin{pmatrix}1\\1\end{pmatrix}=\begin{pmatrix}2(-0.84553)\\-1(-0.84553)\end{pmatrix}=\begin{pmatrix}-1.69107\\0.84553\end{pmatrix},$$

and, since $z_k^{(1)}=\sum_j W_{jk}^{(1)}x_j+b_k^{(1)}$ implies $\partial\mathcal{L}/\partial W_{jk}^{(1)}=x_j\delta_k^{(1)}$,

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(1)}}=\mathbf{x}\,\boldsymbol{\delta}^{(1)\top}=\begin{pmatrix}0.84553 & -0.42277\\-2.02928 & 1.01464\end{pmatrix}, \qquad \frac{\partial\mathcal{L}}{\partial\mathbf{b}^{(1)}}=\begin{pmatrix}-1.69107\\0.84553\end{pmatrix}.$$

The comparison with Example 6.2. Neither hidden unit fails to update here: both pre-activations are positive, so $\mathrm{ReLU}'(z_k^{(1)})=1$ for $k=1,2$ and

both rows of $\partial \mathcal{L}/\partial \mathbf{W}^{(1)}$ are non-zero. This is the opposite of Example 6.2, where the second unit had $z_2^{(1)} = -1.70 < 0$, its activation derivative was zero, and the Hadamard product in (6.16) annihilated its error signal, leaving its incoming weights untouched by that example.

The instructive point is that the two examples together show ReLU inactivity is a property of the *example*, not of the unit. Unit 2 was silent for the first borrower and highly active for the second, and over a mini-batch it receives gradient from whichever examples excite it. A unit becomes *dead* in the sense of the pitfall in Section 6.3 only when its pre-activation is negative for *every* example in the training set, at which point no batch can revive it.

Solution to Exercise 6.4. Take $\mathcal{L}(y, \hat{p}) = \frac{1}{2}(y - \hat{p})^2$ with $\hat{p} = \sigma(z)$. By the chain rule and Proposition 6.2,

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{p}} \cdot \frac{d\hat{p}}{dz} = -(y - \hat{p})\,\sigma'(z) = (\hat{p} - y)\,\hat{p}\,(1 - \hat{p}).$$

Comparing with $\partial \mathcal{L}/\partial z = \hat{p} - y$ from Proposition 6.5, the squared-error gradient carries the extra factor $\hat{p}(1 - \hat{p}) = \sigma'(z) \le \frac{1}{4}$.

That factor is harmful in the worst possible place. Consider an example with $y = 1$ that the network confidently misclassifies, $\hat{p} = 0.01$. The residual $\hat{p} - y = -0.99$ is nearly maximal, but the gradient is $-0.99 \times 0.01 \times 0.99 = -0.0098$: the network learns *a hundred times more slowly* from the example it gets most badly wrong. Cross-entropy, by contrast, delivers the full residual -0.99. The saturating factor makes the learning rate effectively example-dependent in exactly the wrong direction, and it re-introduces the vanishing-gradient mechanism of Proposition 6.10 at the output layer, where Proposition 6.5 had shown it could be eliminated by pairing the loss with the activation correctly. A secondary objection is that the squared error composed with a sigmoid is non-convex in z even for a single linear layer, whereas cross-entropy is convex, so the choice also costs optimisation guarantees in the simplest case.

Solution to Exercise 6.5. By Proposition 6.6, $\mathrm{Var}[\widehat{\mathbf{g}}_B] = \mathrm{Var}[\mathbf{g}_i]/B$, so the standard deviation scales as $B^{-1/2}$. Increasing B from 64 to 512 multiplies the batch size by 8 and therefore multiplies the gradient standard deviation by

$$\sqrt{\tfrac{64}{512}} = \frac{1}{\sqrt{8}} = 0.354,$$

a reduction of about 64.6%.

Is the trade favourable? Under the stated cost model — time strictly proportional to B — it is not. Eight times the computation buys a factor of $\sqrt{8} \approx 2.83$ in noise. The comparison that matters is against the alternative use of the same budget: eight successive steps at $B = 64$. Those eight

steps consume the same computation, and if the parameters move little over the eight steps their averaged gradient also has standard deviation reduced by $\sqrt{8}$ — the same noise reduction, but with eight actual parameter updates instead of one. Small batches therefore dominate at equal compute in this idealised accounting, which is the theoretical reason mini-batch SGD outperforms batch gradient descent per unit of work.

The assumptions under which this holds, and why practice departs from them:

1. *The variance model.* Examples are drawn i.i.d. (with replacement); without replacement a finite-population correction applies, which matters only when B is a non-negligible fraction of n.

2. *Stationarity within the comparison.* The eight-step argument assumes $\mathrm{Var}[\mathbf{g}_i]$ and ∇J are approximately constant over those eight updates. Near a sharp minimum or with a large learning rate this fails.

3. *Strict proportionality of time to B.* This is the assumption that most clearly breaks on real hardware. A GPU processing a batch of 64 is largely idle; a batch of 512 may cost far less than eight times as much wall-clock time, and in that regime the large batch wins outright.

4. *Fixed learning rate.* Larger batches permit a larger stable η, which can convert the noise reduction into faster progress rather than merely quieter steps.

Finally, noise is not purely a cost: Keskar et al. (2017) give evidence that the extra gradient noise of small batches biases the optimiser toward flatter minima that generalise better, so a pure variance-minimisation objective is the wrong target in the first place.

Solution to Exercise 6.6. Apply Proposition 6.7 with $\lambda_{\min} = 0.02$ and $\lambda_{\max} = 8$.

Largest stable learning rate. Convergence holds if and only if $0 < \eta < 2/\lambda_{\max}$, so

$$\eta_{\text{crit}} = \frac{2}{8} = \mathbf{0.25},$$

and any $\eta \geq 0.25$ diverges along the sharpest curvature direction.

Optimal learning rate.

$$\eta^\star = \frac{2}{\lambda_{\min} + \lambda_{\max}} = \frac{2}{8.02} = \mathbf{0.24938}.$$

Note how close $\eta^\star$ lies to η_{crit}: the optimal step is within 0.25% of the divergence boundary. This is characteristic of ill-conditioned problems and explains why tuning η on a badly conditioned loss feels like balancing on

a knife edge — and why standardising inputs, which reduces κ, is worth more than heroic learning-rate search.

Contraction factor. With $\kappa = 8/0.02 = 400$,

$$\rho = \frac{\kappa - 1}{\kappa + 1} = \frac{399}{401} = \mathbf{0.995012}.$$

Iterations for a hundredfold error reduction. Requiring $\rho^t \le 10^{-2}$,

$$t \;\ge\; \frac{\ln 10^{-2}}{\ln \rho} = \frac{-4.60517}{-0.0049999} = 921.03 \quad\Longrightarrow\quad t = \mathbf{922} \text{ iterations.}$$

For comparison, a perfectly conditioned problem with $\kappa = 1$ has $\rho = 0$ and converges in a single step. The 922 iterations are the entire price of the condition number, and since $\rho \approx 1 - 2/\kappa$ for large κ, the iteration count grows roughly linearly in κ: halving the condition number by rescaling one badly scaled feature halves the training time.

Solution to Exercise 6.7. **Part 1.** By Proposition 6.11, $\varsigma^2 = 2/n_{\text{in}} = 2/256 = 0.0078125$, so

$$\varsigma = \sqrt{0.0078125} = \frac{1}{\sqrt{128}} = \mathbf{0.08839}.$$

This is what `he_normal` uses: a zero-mean Gaussian with that standard deviation.

Part 2. Repeat the derivation with $\mathrm{LReLU}_\alpha(u) = \max(\alpha u, u)$, that is $x = u$ for $u > 0$ and $x = \alpha u$ for $u \le 0$. Let u be symmetric about zero with variance τ_{pre}^2. Splitting the expectation over the two half-lines and using symmetry, which makes each half contribute $\frac{1}{2}\mathbb{E}[u^2]$,

$$\begin{aligned}\mathbb{E}[x^2] &= \mathbb{E}\big[u^2 \mathbf{1}\{u > 0\}\big] + \alpha^2\, \mathbb{E}\big[u^2 \mathbf{1}\{u \le 0\}\big] \\ &= \tfrac{1}{2}\tau_{\text{pre}}^2 + \alpha^2 \cdot \tfrac{1}{2}\tau_{\text{pre}}^2 = \tfrac{1}{2}\left(1 + \alpha^2\right)\tau_{\text{pre}}^2.\end{aligned}$$

Substituting into $\mathrm{Var}(z) = n_{\text{in}}\varsigma^2\mathbb{E}[x^2]$ from the first part of Proposition 6.11 and requiring $\mathrm{Var}(z) = \tau_{\text{pre}}^2$ for variance preservation across layers,

$$n_{\text{in}}\,\varsigma^2\,\tfrac{1}{2}\left(1 + \alpha^2\right)\tau_{\text{pre}}^2 = \tau_{\text{pre}}^2 \quad\Longrightarrow\quad \varsigma^2 = \frac{2}{(1 + \alpha^2)\, n_{\text{in}}},$$

as required. Two sanity checks confirm the formula. At $\alpha = 0$ the activation is ordinary ReLU and the result reduces to $2/n_{\text{in}}$. At $\alpha = 1$ the activation is the identity, no half of the distribution is discarded, and the result becomes $1/n_{\text{in}}$ — the classical variance-preserving scale for a linear layer. For the usual $\alpha = 0.01$ the correction factor is $1/(1 + 10^{-4})$, numerically indistinguishable from He, so in practice `he_normal` is used for leaky ReLU without adjustment.

Solution to Exercise 6.8. **Part 1.** In Proposition 6.12 the penalty term is $q(1-q)\sum_j w_j^2 x_j^2$ with $q = 1-p$ the keep probability. At $p = 0.5$ we have $q = 0.5$ and

$$q(1-q) = 0.5 \times 0.5 = \mathbf{0.25},$$

so the implied penalty is $0.25\sum_j w_j^2 x_j^2$, and the point prediction is additionally shrunk deterministically by the factor $q = 0.5$. Averaged over the training set, the effective per-coordinate ridge coefficient is $0.25\,\mathbb{E}[x_j^2]$: dropout is a ridge penalty *weighted by each feature's second moment*. Note also that $q(1-q)$ is maximised at $q = 1/2$, so $p = 0.5$ is the most aggressive drop rate in this sense; both $p \to 0$ and $p \to 1$ send the penalty to zero, the latter for the uninteresting reason that the model is destroyed.

Part 2. Suppose feature j is left in raw KES with typical magnitude 10^6, so $\mathbb{E}[x_j^2] \approx 10^{12}$, while a standardised feature k has $\mathbb{E}[x_k^2] = 1$. The implied penalties are then in the ratio

$$\frac{0.25 \times 10^{12}}{0.25 \times 1} = 10^{12}.$$

Coordinate j is penalised a trillion times more heavily than coordinate k, which drives w_j to essentially zero: the feature is silently deleted from the model by the regulariser, no matter how predictive it is. The reverse case is equally bad — a feature measured in millionths would be effectively unregularised and free to overfit.

This is the same lesson as Section 6.12, arriving from a different direction. There, unstandardised inputs damaged the *conditioning* of the optimisation via Proposition 6.7; here they damage the *regularisation* by making the implied penalty an arbitrary function of the measurement unit. In both cases the defect is that a modelling decision has become a consequence of whether an analyst recorded an amount in shillings or in thousands of shillings. Standardising the inputs makes $\mathbb{E}[x_j^2] \approx 1$ for every j and restores the intended uniform penalty.

Solution to Exercise 6.9. **Counting.** Of 120,000 agents, 71% appear fewer than five times, that is $0.71 \times 120{,}000 = 85{,}200$ low-support agents, leaving 34,800 with support of five or more.

Without collapsing, the table has one row per agent plus one reserved out-of-vocabulary row:

$$(120{,}000 + 1) \times 16 = \mathbf{1{,}920{,}016} \text{ parameters.}$$

Collapsing the 85,200 low-support agents into the reserved index 0 leaves 34,800 individual rows plus that shared row:

$$(34{,}800 + 1) \times 16 = \mathbf{556{,}816} \text{ parameters,}$$

a saving of 1,363,200 parameters, or 71% of the table — unsurprisingly the same proportion as the collapsed vocabulary, since the cost is linear in the number of rows.

Which variant to submit, and why. Submit the collapsed variant. The statistical argument is decisive: each low-support agent has 16 free parameters fitted from at most four labelled observations. With a 4% base rate, the overwhelming majority of those agents have *no defaults at all* in the training set, so the fitted embedding encodes the accident of which handful of borrowers happened to use that kiosk rather than any property of the agent. That is memorisation, and it will not replicate on the test fold.

Three further arguments reinforce the choice. It trains the out-of-vocabulary path: index 0 receives gradient from 85,200 agents' worth of examples, so a genuinely unseen agent at inference time is mapped to a meaningful average rather than to an untrained initialisation — the failure mode of the pitfall in Section 6.13. It reduces the parameter count from roughly 400 times the rest of the network to roughly 115 times, which materially reduces the memory footprint of the served artefact. And it is more defensible under the governance expectations of Section 6.15, since a reviewer can be shown the support threshold as an explicit, documented modelling decision.

What evidence would settle it. Fit both variants under an identical protocol — same frozen splits, same preprocessing, same search budget, same early-stopping criterion — and compare, in order: test PR-AUC and expected cost; performance restricted to the *low-support agent subgroup*, which is where the two variants differ and where the aggregate metric may hide the difference; and calibration by county. A useful diagnostic beyond the headline metrics is to plot the norm of each learned embedding row against that agent's training support in the uncollapsed model: if the rare rows have drifted far from their initialisation, the model is fitting noise, and the plot makes that visible to a non-technical reviewer. Finally, the support threshold itself should be treated as a hyperparameter — 5 is a convention, not a result — and swept over, say, $\{3, 5, 10, 25\}$ on the validation fold.

Solution to Exercise 6.10. Proposition 6.14 establishes that $z \mapsto \sigma(z/T)$ is strictly increasing, hence order-preserving. Everything follows from distinguishing quantities that depend on the scores *only through their order* from those that depend on their *values*.

1. **ROC-AUC: unchanged.** It is a functional of the ranking alone (the probability that a random positive outranks a random negative), and the ranking is preserved.

2. **PR-AUC: unchanged.** Precision and recall at each operating point depend only on which examples fall above the cut, and the family of sets obtainable by thresholding is identical before and after scaling.

3. **Brier score: changes.** It is the mean squared difference between predicted probability and outcome, a function of the values themselves. This is precisely the quantity temperature scaling is fitted to improve; with $T = 1.8 > 1$ the logits shrink toward zero and the probabilities toward 0.5, which reduces the Brier score for an over-confident model and would increase it for an under-confident one.

4. **Expected cost at the fixed threshold $\hat{p} = 0.15$: changes.** A fixed threshold on the *probability* scale is not a fixed threshold on the score scale. Before scaling, $\hat{p} = 0.15$ corresponds to the logit $\text{logit}(0.15) = -1.7346$; after scaling, $\sigma(z/1.8) = 0.15$ requires $z = 1.8 \times (-1.7346) = -3.1223$. A different set of applicants is declined, so the confusion matrix and hence the cost move.

5. **Expected cost at the re-derived cost-optimal threshold: unchanged.** Optimising the threshold means minimising cost over the family of decision sets obtainable by thresholding the score. Since the transformation is a strictly increasing bijection, that family is identical before and after, so the minimum over it is identical too. The optimal *threshold value* moves; the optimal *cost* does not.

Remark B.1. The last two items together carry the practical message. Temperature scaling cannot improve the best achievable cost — item 5 — so it is not a performance technique. What it does is make the probability scale trustworthy, so that the analytically derived threshold is the right one. Under the cost matrix of the running case study the indifference point is where $\hat{p} \cdot 10{,}000 = (1 - \hat{p}) \cdot 800$, that is

$$\hat{p}^\star = \frac{800}{10{,}800} = 0.0741.$$

Applying that threshold to *uncalibrated* scores gives the cost-optimal decision only by luck; applying it after calibration is a principled derivation. That is the entire value proposition: item 5 says calibration costs nothing in ranking, and this remark says it buys correctness in the mapping from scores to decisions.

Chapter 7 — Computer Vision with Convolutional Neural Networks

Solution to Exercise 7.1. Apply Proposition 7.1: (7.2) for shapes and (7.3) for counts. For `padding="same"` with odd f and stride s, the output side is $\lceil n_{\text{in}}/s \rceil$.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
`Conv2D(64, 7, s=2, same)`	$112 \times 112 \times 64$	$7^2 \cdot 3 \cdot 64 + 64 = 9{,}472$
`MaxPooling2D(3, s=2, same)`	$56 \times 56 \times 64$	0
`Conv2D(128, 3, same)`	$56 \times 56 \times 128$	$3^2 \cdot 64 \cdot 128 + 128 = 73{,}856$
`MaxPooling2D(2)`	$28 \times 28 \times 128$	0
`Conv2D(256, 3, same)`	$28 \times 28 \times 256$	$3^2 \cdot 128 \cdot 256 + 256 = 295{,}168$
`MaxPooling2D(2)`	$14 \times 14 \times 256$	0
`GlobalAveragePooling2D`	256	0
`Dense(5, softmax)`	5	$256 \cdot 5 + 5 = 1{,}285$
Total		**379,781**

With a $384 \times 384 \times 3$ input, **no parameter count changes at all**. The spatial dimensions become $192 \to 96 \to 96 \to 48 \to 48 \to 24$, and global average pooling still produces a 256-vector, so the dense head is unaffected. This is Proposition 7.1: kernel tensors are indexed by $(f, f, c_{\text{in}}, c_{\text{out}})$ and never by H or W.

What does change is *activation* memory and compute. The first convolution's output tensor grows from $112^2 \cdot 64$ to $192^2 \cdot 64$ entries, a factor of $(192/112)^2 \approx 2.94$; since every layer scales the same way, both peak memory and multiply–accumulate cost rise by roughly $(384/224)^2 \approx 2.94$. The common student error is to expect a bigger model; the correct statement is a bigger *computation* with the same model.

Solution to Exercise 7.2. Use $r_0 = 1$, $j_0 = 1$, $r_\ell = r_{\ell-1} + (f_\ell - 1) j_{\ell-1}$, $j_\ell = j_{\ell-1} s_\ell$.

ℓ	f_ℓ	s_ℓ	r_ℓ	j_ℓ
1	7	2	$1 + 6 \cdot 1 = 7$	2
2	3	2	$7 + 2 \cdot 2 = 11$	4
3	3	1	$11 + 2 \cdot 4 = 19$	4
4	3	1	$19 + 2 \cdot 4 = 27$	4
5	3	2	$27 + 2 \cdot 4 = 35$	8
6	3	1	$35 + 2 \cdot 8 = 51$	8

After layer 6 the jump is $j = 8$, so each further 3×3 stride-one layer adds $(3 - 1) \cdot 8 = 16$ pixels. Starting from 51: $67, 83, 99, 115$. Three extra layers give 99, which does *not* exceed 100; **four** extra layers are required, reaching 115.

Two remarks worth making to students. First, receptive field grows in proportion to the accumulated stride, so layers placed after a downsample are far more "efficient" per layer than early ones — this is why architectures front-load striding. Second, the theoretical receptive field is an upper bound on influence, not a description of it: the *effective* receptive field is

approximately Gaussian and considerably smaller, because centre pixels contribute through many more paths than edge pixels.

Solution to Exercise 7.3. Take the one-dimensional input $x = (1,0,0,0,0,0)$, kernel $K = (1,0,0)$, padding $p = 0$, stride $s = 2$. The admissible left edges are $t = 0, 2$ (and $t = 3$ is excluded since $3 + 2 > 5$), so

$$\mathcal{C}(x) = (x_0,\ x_2,\ x_4) = (1,0,0).$$

Now shift the input by one position: $x' = (0,1,0,0,0,0)$. Then

$$\mathcal{C}(x') = (x'_0,\ x'_2,\ x'_4) = (0,0,0).$$

The output of the shifted input is the zero vector, which is not any translate of $(1,0,0)$: the feature has disappeared entirely rather than moved. This violates equivariance. Contrast a shift of *two* positions, $x'' = (0,0,1,0,0,0)$, which gives $\mathcal{C}(x'') = (0,1,0)$ — exactly $\mathcal{C}(x)$ translated by $1 = \tau/s$, as Proposition 7.2 predicts.

Practical consequence. A borrower photographing an identity card by hand produces images whose content is shifted by arbitrary amounts, not by multiples of the stride. A strided network's response to the same document therefore varies with sub-stride jitter, and since the accumulated stride of a ResNet-50 backbone is 32, the relevant jitter is not one pixel but up to thirty-one.

The two mitigations named in Section 7.2.2. First, include small random translations in the augmentation policy, so that approximate invariance is learned from data rather than assumed from the architecture; by the argument of Section 7.4.5 this is legitimate precisely because a small shift genuinely does preserve the label. Second, avoid evaluating on a single fixed crop: averaging predictions over a small ensemble of crops averages out the phase of the sampling grid, at a cost linear in the number of crops.

Solution to Exercise 7.4. With $C = 512$, a plain 3×3 convolution costs $9C^2 = 9 \cdot 512^2 = 2{,}359{,}296$ weights.

A bottleneck of width $b = 128$ costs

$$\underbrace{1^2 \cdot 512 \cdot 128}_{65{,}536} + \underbrace{3^2 \cdot 128^2}_{147{,}456} + \underbrace{1^2 \cdot 128 \cdot 512}_{65{,}536} = 278{,}528,$$

so the plain block is $2{,}359{,}296 / 278{,}528 = 8.47$ times more expensive.

Break-even solves $9C^2 = 2Cb + 9b^2$, i.e. $9b^2 + 2Cb - 9C^2 = 0$, whose positive root is

$$b = \frac{-2C + \sqrt{4C^2 + 324C^2}}{18} = \frac{C\left(\sqrt{82} - 1\right)}{9} \approx 0.895\,C \approx 458.3.$$

ResNet uses $b = C/4 = 128$, far below break-even, and the reason is that parameter parity is the wrong objective. The design goal is to spend a fixed

budget on as much *depth* as possible, because Theorem 7.1 makes depth trainable and Section 7.2.4 shows it to be a better use of a weight budget than width. Choosing b near break-even would buy no depth at all. Choosing b much smaller than $C/4$ would eventually starve the 3×3 layer of capacity; $C/4$ is the empirical compromise, and the $8.5\times$ saving is what funds the 16 blocks of stages 2–5.

Solution to Exercise 7.5. **(a)** $\alpha\beta^2\gamma^2 = 1.2 \cdot 1.1^2 \cdot 1.15^2 = 1.2 \cdot 1.21 \cdot 1.3225 = 1.9203$, which is ≈ 2 as required.

(b) Using $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$ and the FLOP multiplier $(\alpha\beta^2\gamma^2)^\phi$ from Proposition 7.4:

ϕ	d	w	r	FLOP multiplier
1	1.200	1.100	1.150	1.920
2	1.440	1.210	1.322	3.687
3	1.728	1.331	1.521	7.081

(c) Solve $1.9203^\phi = 8$:

$$\phi = \frac{\ln 8}{\ln 1.9203} = \frac{2.0794}{0.6526} = 3.187.$$

The implied resolution factor is $\gamma^{3.187} = 1.15^{3.187} = 1.561$, so the input side becomes $224 \times 1.561 \approx 350$ pixels. In practice one would round to a network-friendly value such as 352 (a multiple of 32, the total stride), and note that depth and width scale to $1.20^{3.187} = 1.77$ and $1.10^{3.187} = 1.37$ respectively.

(d) The arithmetic gives a compute budget, not an accuracy guarantee, and the two should not be confused. Compound scaling was calibrated on ImageNet, where 1.2 million labelled images make capacity the binding constraint. On a five-class document task with a few thousand images the binding constraint is data, and Figure 7.14 shows exactly which regime that puts us in: the flat left-hand region where additional capacity buys nothing. A larger ϕ would raise the parameter count, the training time and the deployed size, increase the risk of overfitting, and in all likelihood lower the held-out macro-F_1. The defensible use of this calculation is the reverse direction: given a fixed serving budget, it tells you which family member you can afford.

Solution to Exercise 7.6. Write $\bar{A}^k = (H'W')^{-1} \sum_{i,j} A^k_{i,j}$ and $y^c = \sum_k w^c_k \bar{A}^k + b^c$. Since $\bar{A}^k$ is linear in the entries of A^k with coefficient $1/(H'W')$ on each, and b^c is a constant,

$$\frac{\partial y^c}{\partial A^k_{i,j}} = w^c_k \cdot \frac{\partial \bar{A}^k}{\partial A^k_{i,j}} = \frac{w^c_k}{H'W'} \qquad \text{for every } (i,j).$$

Substituting into the definition of α_k^c,

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{w_k^c}{H'W'} = \frac{1}{H'W'} \cdot H'W' \cdot \frac{w_k^c}{H'W'} = \frac{w_k^c}{H'W'}.$$

Hence

$$L^c = \mathrm{ReLU}\left(\sum_k \frac{w_k^c}{H'W'} A^k \right) = \frac{1}{H'W'} \mathrm{ReLU}\left(\sum_k w_k^c A^k \right),$$

using $\mathrm{ReLU}(\lambda u) = \lambda \,\mathrm{ReLU}(u)$ for $\lambda > 0$. The map is therefore the class activation map up to a positive constant, and since Grad-CAM maps are normalised to $[0, 1]$ before display, the constant is invisible.

The bias. The term b^c contributes nothing, because it does not depend on A^k and so $\partial b^c / \partial A_{i,j}^k = 0$. Intuitively, a class-independent offset shifts every score equally and therefore carries no spatial information.

What breaks with a hidden layer. If the head is

$$y^c = \sum_m v_m^c \,\mathrm{ReLU}\left(\sum_k u_{mk} \bar{A}^k \right),$$

the derivative becomes $\partial y^c / \partial A_{i,j}^k = (H'W')^{-1} \sum_m v_m^c u_{mk} \mathbb{1}[\text{unit } m \text{ active}]$, which still does not depend on (i, j) but now depends on the *input* through the activation pattern. The proportionality to a fixed weight vector is lost: there is no longer a single set of class weights w_k^c, and α_k^c becomes example-specific. Grad-CAM remains well defined — that is precisely the generalisation it offers — but the identification with CAM, and with it the guarantee that the map reflects a fixed linear read-out, no longer holds.

Solution to Exercise 7.7. **(a)** By Proposition 7.5,

$$t^\star = \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}} = \frac{800}{800 + 10{,}000} = \frac{800}{10{,}800} = 0.0741.$$

Escalate to human review whenever the tamper probability exceeds about 7.4%.

(b) The always-approve policy never raises a false positive and incurs a false negative on every tampered document, so its expected cost per application is

$$\pi \, C_{\mathrm{FN}} = 0.04 \times 10{,}000 = \text{KES } 400.$$

For comparison, always-reject costs $(1 - \pi) C_{\mathrm{FP}} = 0.96 \times 800 = \text{KES } 768$. Always-approve is therefore the better of the two trivial policies, and it is the baseline to beat: a model that cannot get below KES 400 per application is not earning its keep.

(c) Accuracy at $t = 0.5$ is misleading for two compounding reasons. It is measured at the wrong threshold: 0.5 is optimal only when $C_{\mathrm{FP}} = C_{\mathrm{FN}}$, and here the costs differ by a factor of 12.5, so the reported operating point is not one the business would ever choose. And accuracy itself is uninformative under a 4% positive rate, because the always-approve policy achieves 96% — a number that sounds excellent and corresponds to catching no fraud at all.

Solution to Exercise 7.8. *Inference mode* uses the stored running statistics:

$$\hat{z}_R = \frac{1.5 - 0}{1} = 1.5, \qquad \tilde{z}_R = 1.2 \times 1.5 - 0.3 = 1.5.$$

Training mode uses the current batch statistics:

$$\hat{z}_B = \frac{1.5 - 0.8}{2.5} = 0.28, \qquad \tilde{z}_B = 1.2 \times 0.28 - 0.3 = 0.036.$$

The difference is $1.5 - 0.036 = 1.464$, i.e. the activation that the head was trained to read as 1.5 arrives as 0.036 — a change of over 97% of its value, and a change of sign relative to the shift β.

Why this compounds. ResNet-50 contains 53 batch-normalisation layers. The discrepancy is an affine map of z with slope $\gamma\left(\sigma_R^{-1} - \sigma_B^{-1}\right)$, so each layer applies its own such distortion and the distortions compose multiplicatively through the stack. Even a modest per-layer factor of, say, 0.7 in effective scale becomes $0.7^{53} \approx 2 \times 10^{-9}$ over the whole backbone. The features reaching the head bear no relation to the features it was trained on in Phase A, which is why validation accuracy does not degrade gracefully but collapses within one or two epochs of unfreezing. Note also that the corruption is *persistent*: the running statistics themselves are overwritten during training, so the damage survives a switch back to inference mode and the checkpoint on disk is already spoiled. This is why the fix must be applied before training, not diagnosed afterwards.

Solution to Exercise 7.9.

Transformation	Cassava	Document	Reason
Horizontal flip	yes	**no**	A mirrored leaf is still a diseased leaf; mirrored text on an ID is itself evidence of tampering.
$\pm 90^\circ$ rotation	yes	**no**	Leaves have no canonical orientation; documents do, and a sideways ID is a capture failure the model should flag, not ignore.
$\pm 8^\circ$ rotation	yes	yes	Both are photographed handheld; small tilt is ordinary capture noise in each domain.
Brightness $\pm 20\%$	yes	yes	Field lighting and indoor lighting both vary widely; brightness is not diagnostic of either label.
Random JPEG compression	yes	**caution**	Realistic for leaves. For documents, compression artefacts can be a re-photography signal, so this risks erasing evidence.
Random 30×30 occlusion	yes	yes	Both benefit: it discourages reliance on a single region and mimics fingers, glare and shadow.

Formal argument. The decomposition in Section 7.4.5 writes the augmented risk as

$$\big(\bar{f}(\mathbf{x}) - y\big)^2 + \mathrm{Var}_T[f(T\mathbf{x})]\,,$$

so the objective explicitly rewards a model whose output does *not* vary across G. That is desirable only when the label is genuinely constant on the orbit $\{T\mathbf{x} : T \in G\}$.

Suppose it is not. Take $\mathbf{x}$ a genuine document with $y = 0$, and T the horizontal flip, with $y(T\mathbf{x}) = 1$ because mirrored text indicates tampering. The augmented objective still trains $f(T\mathbf{x})$ towards $y(\mathbf{x}) = 0$, so the pair $(T\mathbf{x}, 0)$ enters the training set as a *mislabelled* example. Worse, the variance term actively penalises the model for distinguishing $\mathbf{x}$ from $T\mathbf{x}$ — it is not merely noise but a regulariser pulling towards the wrong invariance. The Bayes-optimal classifier for the augmented distribution is therefore strictly worse than the Bayes-optimal classifier for the true one, and no amount of additional data will recover the gap.

Solution to Exercise 7.10. **(a)** Storage is $P \times b/8$ bytes.

Model	Parameters	float32	float16	int8
EfficientNetV2-B0	7.2M	28.8 MB	14.4 MB	7.2 MB
ResNet50V2	25.6M	102.4 MB	51.2 MB	25.6 MB

(b) Against a 10 MB cap, only **EfficientNetV2-B0 in int8** (7.2 MB) qualifies. B0 in float16 is 14.4 MB and still fails; ResNet50V2 fails at every precision, needing better than $3\times$ compression beyond int8 to fit — achievable

only with pruning or distillation, which is a different project. This is the concrete sense in which backbone choice is a product decision rather than an accuracy decision: on this requirement the ResNet is excluded before its accuracy is even measured.

(c) For values in $[-3, 3]$ quantised symmetrically to 8 bits, the step is

$$\Delta = \frac{2r}{255} = \frac{6}{255} = 0.02353,$$

and the maximum rounding error is $\Delta/2 = 0.01176$, i.e. about 0.2% of the full range.

Whether that matters is best judged against the noise already in the system. The cassava dataset carries substantial label noise — disease grades assigned from photographs by human annotators disagree at rates in the several-percent range — and the model's own uncertainty is far larger than a 0.2% perturbation of individual weights. Empirically, int8 post-training quantisation with a representative dataset typically costs well under one point of accuracy on this class of task. The honest framing for a design review is therefore: the quantisation error is roughly an order of magnitude smaller than the label noise, so it should not be the first thing to blame if accuracy disappoints. The caveats worth stating are that per-tensor quantisation is riskier than per-channel for layers whose weight ranges vary widely across channels, and that the effect must be *measured* on the held-out set rather than assumed — a one-line comparison of the float and TFLite models that belongs in every deployment notebook.

Chapter 8: Sequence Models and Time-Series Forecasting

Solution to Exercise 8.1 (The covariate taxonomy).

(a) **County**: static, $\mathbf{s}$. It does not vary over the horizon, and it is available at every origin.

(b) **Transactions in the previous hour**: past-observed, $\mathbf{z}$. It is measured, so it exists only up to the origin.

(c) **Tomorrow being a public holiday**: known-future, $\mathbf{x}$. It comes from a calendar published in advance.

(d) **Tomorrow's rainfall from the published forecast**: known-future, $\mathbf{x}$. What makes it admissible is not that it is accurate but that it is *available* at 18:00 today. Its errors degrade performance; they do not cause leakage.

(e) **Tomorrow's actual rainfall**: unusable. It is not known at the origin, so a model trained on it cannot be run in production at all. Including it produces excellent backtests and a system that fails on its first live day.

(f) **Total commission over the following week**: unusable, and worse than (e), because it is a function of the target itself over a window that extends beyond the horizon. This is target leakage in the sense of Chapter 2, and it will produce near-perfect validation scores.

The general test is the one in Figure 8.5: ask whether the value could be written into a file at 18:00 today. If not, it is not a feature.

Solution to Exercise 8.2 (Window arithmetic). **(a)** A window occupies $L + H = 192$ consecutive observations, and the number of starting positions in a block of length n is $n - (L + H) + 1$. For the undivided series,

$$8{,}760 - 192 + 1 = 8{,}569 \text{ windows.}$$

(b) The chronological split gives blocks of 6,132, 1,314 and 1,314 observations. Cutting windows inside each block:

$$\begin{aligned} \text{train: } & 6{,}132 - 191 = 5{,}941, \\ \text{valid: } & 1{,}314 - 191 = 1{,}123, \\ \text{test: } & 1{,}314 - 191 = 1{,}123, \end{aligned}$$

totalling 8,187 windows, so $8{,}569 - 8{,}187 = 382$ are lost. The lost windows are exactly the 2×191 that would have straddled one of the two boundaries.

(c) Each discarded window would have had its input in one block and part of its target in the next, so a model trained on it would have seen validation or test observations during training. Losing 4.5% of the examples is a small price for a validation number that means what it claims; the alternative is a larger training set and an unusable estimate of generalisation.

Solution to Exercise 8.3 (The pinball loss and the quantile). **(a)** With $\hat{y}^{(q)} = 100$ and $e = y - \hat{y}^{(q)}$:

	$y = 120$ $(e = +20)$	$y = 80$ $(e = -20)$
$q = 0.9$	$0.9 \times 20 = 18$	$0.1 \times 20 = 2$
$q = 0.1$	$0.1 \times 20 = 2$	$0.9 \times 20 = 18$

At $q = 0.9$ an under-forecast costs nine times as much as an over-forecast of the same magnitude, so the minimiser is pushed high; at $q = 0.1$ the asymmetry is reversed. The loss encodes the risk appetite directly, which is why the choice of q is a business decision rather than a modelling one.

(b) The distribution function is

$$F(10) = 0.1, \quad F(20) = 0.3, \quad F(30) = 0.7, \quad F(40) = 0.9, \quad F(50) = 1.0.$$

By Proposition 8.1 the minimiser is any a with $F(a) = q$, interpreted as the smallest a with $F(a) \geq q$ when F jumps. For $q = 0.5$, F first reaches 0.5 at $a = 30$; for $q = 0.9$, at $a = 40$. Direct evaluation confirms both: $\mathbb{E}[L_{0.5}(Y, a)]$ is minimised at $a = 30$ with value 4.0, and $\mathbb{E}[L_{0.9}(Y, a)]$ at $a = 40$ with value 2.0.

(c) At $q = 0.5$ the derivative computed in the proof is $g'(a) = F(a) - 1/2$, so the minimiser is the *median*. For a right-skewed distribution the median lies below the mean, so an MAE-trained model predicts below the mean by construction. This is correct behaviour: MAE was chosen precisely because the occasional enormous hour should not drag the forecast upward. If the mean is genuinely wanted, use squared error, and accept that a single flash crowd will dominate the fit.

Solution to Exercise 8.4 (Metrics that disagree). The model's absolute errors are

$$(5,\ 10,\ 10,\ 10,\ 10,\ 5), \qquad \text{so MAE} = 50/6 \approx 8.33,$$

and the seasonal-naïve absolute errors are

$$(10,\ 5,\ 5,\ 10,\ 5,\ 10), \qquad \text{so MAE}_{\text{snaive}} = 45/6 = 7.5.$$

Hence

$$\text{MASE} = \frac{8.33}{7.5} \approx 1.11, \qquad \text{sMAPE} \approx 6.79\%.$$

The sMAPE looks respectable and is misleading on its own. The MASE exceeds one, which says the model is *worse than seasonal naïve* on this sample. The recommendation is not to tune the network. It is to ship seasonal naïve, which requires no training pipeline, no GPU and no drift surface, and to redirect the effort to data quality and covariates. Any deep-forecasting review should begin with this comparison, and a good many end there.

Solution to Exercise 8.5 (Why plain recurrence forgets). **(a)** Proposition 8.2 gives the bound λ^k with $\lambda = 0.9$ and $k = 168$:

$$0.9^{168} \approx 2.05 \times 10^{-8}.$$

In single precision, whose resolution near one is about 10^{-7}, this is numerically indistinguishable from zero: the first observation of the window has no influence on the loss.

(b) Solve $0.9^k < 10^{-6}$, giving $k > \ln(10^{-6})/\ln(0.9) \approx 131.1$, so $k = 132$ steps. Roughly five and a half days of hourly data suffice to extinguish the signal.

(c) By Theorem 8.1 the factor along that coordinate is $\prod_j f_j = 0.99^{168} \approx 0.185$. The gradient is attenuated by a factor of about five rather than by eight orders of magnitude, and with $f = 0.999$ it would be 0.845, essentially undamped.

(d) In the plain RNN the product is over $\mathbf{W}_h^\top \operatorname{diag}(\tanh')$, a fixed learned matrix acting at every step; keeping its norm near one is a global constraint on a matrix that is simultaneously responsible for all the computation the layer performs. In the LSTM the product is over *gate activations*, which are computed per step, per coordinate, from the input, so the network can hold one coordinate open for long-range memory while others forget quickly. The mechanism is controllable because it is separate from the computation it protects.

Solution to Exercise 8.6 (Counting gate parameters). **(a)** With $d_x = 32$ and $d_h = 64$, the common factor is $(d_x + d_h + 1)d_h = 97 \times 64 = 6{,}208$. Hence

$$\text{LSTM: } 4 \times 6{,}208 = 24{,}832, \qquad \text{GRU: } 3 \times 6{,}208 = 18{,}624,$$

a ratio of exactly $4/3$, since the LSTM computes four internal layers and the GRU three.

(b) A single GRU of width 128 has $3(32 + 128 + 1) \times 128 = 3 \times 161 \times 128 = 61{,}824$ parameters. Two stacked GRUs of width 64 have $3(32 + 64 + 1) \times 64 = 18{,}624$ for the first and $3(64 + 64 + 1) \times 64 = 24{,}768$ for the second, totalling 43,392: about thirty percent fewer.

Beyond the count, the stack composes two nonlinear recurrences, so it can represent temporal patterns of patterns, which a single wider layer cannot. What is lost is capacity per layer, so each state is a narrower summary, and depth brings the usual optimisation difficulty. The stack is also strictly slower per step in wall-clock time, because the two layers cannot be executed in parallel.

Solution to Exercise 8.7 (Why attention divides by $\sqrt{d_k}$). **(a)** Write $\mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. By independence and zero means, $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$, so the sum has mean zero. The terms are uncorrelated, so variances add, and $\operatorname{Var}[q_i k_i] = \mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$, giving $\operatorname{Var} = d_k$.

(b) For $d_k = 64$ the standard deviation is $\sqrt{64} = 8$, so scores of ± 8 are entirely typical. For the unscaled vector $(8, 0, -8)$,

$$\text{softmax} \approx (0.99966,\ 0.00034,\ 1.1 \times 10^{-7}),$$

which is a hard maximum in all but name. Scaling by $1/\sqrt{64} = 1/8$ gives scores $(1, 0, -1)$ and

$$\text{softmax} \approx (0.665,\ 0.245,\ 0.090),$$

a genuine weighted average.

(c) The Jacobian of the softmax is $\operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$, whose entries are products of the weights and therefore vanish when one weight approaches one and the rest approach zero. In the unscaled case the gradient with respect to the scores is numerically zero, so the query and key projections receive no learning signal, and the layer is frozen at whatever pattern its initialisation happened to produce. The scaling keeps the softmax in the region where it has a usable derivative, which is the same consideration that governed initialisation in Chapter 6.

Solution to Exercise 8.8 (Attention against recurrence). **(a)** From Proposition 8.3, attention costs $\mathcal{O}(L^2 d)$ and recurrence $\mathcal{O}(L d^2)$. With $L = 168$ and $d = 64$,

$$L^2 d = 168^2 \times 64 = 1{,}806{,}336, \qquad L d^2 = 168 \times 64^2 = 688{,}128,$$

a ratio of 2.625. Attention is about two and a half times more expensive here.

(b) The costs coincide when $L^2 d = L d^2$, that is when $L = d$. The ratio is exactly L/d, which is the useful form to remember: attention is cheaper than recurrence whenever the sequence is shorter than the model is wide.

(c) At $L = 4{,}096$ the ratio is $4096/64 = 64$: attention costs 1.07×10^9 operations against 1.68×10^7. The quadratic term dominates completely, which is what motivates the efficient-attention variants of Chapter 9.

(d) Attention retains constant gradient path length between any two positions, so no long-range dependency has to survive a chain of updates, and full parallelism across positions, since no output waits on another. The disadvantage that grows is memory: the $L \times L$ attention matrix must be materialised per head, which is $\mathcal{O}(L^2)$ and becomes the binding constraint long before the operation count does.

Solution to Exercise 8.9 (Causal masking, carefully). **(a)** Position i may attend to positions $j \le i$, so the surviving pairs are the lower triangle including the diagonal:

$$\frac{L(L+1)}{2} = \frac{168 \times 169}{2} = 14{,}196 \quad \text{of} \quad L^2 = 28{,}224,$$

a fraction of $(L+1)/(2L) \approx 50.3\%$. Masking therefore saves slightly less than half the computation, and asymptotically exactly half.

(b) The full softmax of $(2.0, 1.0, 0.5)$ is $(0.6285, 0.2312, 0.1402)$. Zeroing the last entry afterwards leaves $(0.6285, 0.2312, 0)$, which sums to 0.8598 rather than to one. Masking correctly, by setting the third score to $-\infty$ before the softmax, gives $(0.7311, 0.2689, 0)$. The colleague's first weight is too small by $0.7311 - 0.6285 = 0.1026$, an error of about fourteen percent, and the output is systematically shrunk towards zero because the weights no longer sum to one. The error grows with the number of masked positions,

so it is worst at the start of the horizon, exactly where accuracy matters most.

(c) Without the mask, each position attends to later positions and the model can read the answer it is being asked to predict. Training and validation loss both fall to near zero, the curves look ideal, and the model produces nonsense the first time it is run at an origin where the future genuinely does not exist. The signature is a validation score that is not merely good but implausible, and it should always be investigated rather than celebrated.

Solution to Exercise 8.10 (Variable selection, implemented). A minimal implementation, continuing the notation of Listing 8.4:

```
def variable_selection(x, static_ctx, n_vars, d):
    """x: [B, T, n_vars]. Returns (selected [B, T, d], weights [B, T, n_vars
    ])."""
    # 1. embed each variable separately, then transform it with its own GRN
    per_var = [grn(x[..., j:j+1], d) for j in range(n_vars)]      # n_vars x
     [B,T,d]
    stacked = tf.stack(per_var, axis=-2)                          # [B, T,
    n_vars, d]

    # 2. selection weights from a GRN over the flattened inputs, conditioned
    #    on the static context broadcast across time
    flat = layers.Reshape((-1, n_vars * d))(stacked)              # [B, T,
    n_vars*d]
    ctx  = tf.repeat(static_ctx[:, None, :], tf.shape(flat)[1], axis=1)
    w    = layers.Dense(n_vars, activation="softmax")(
               grn(layers.Concatenate(-1)([flat, ctx]), d))       # [B, T,
    n_vars]

    # 3. weighted sum over the variable axis
    selected = tf.reduce_sum(stacked * w[..., None], axis=-2)     # [B, T, d
    ]
    return selected, w

past_sel, past_w = variable_selection(past, s, n_past, d)
enc, h, c = layers.LSTM(d, return_sequences=True, return_state=True)(
                past_sel, initial_state=[layers.Dense(d)(s)] * 2)
```

Three implementation points are worth noting. The softmax runs over the variable axis and not over time, so the weights answer "which input mattered at this step" rather than "which step mattered". Each variable gets its *own* GRN, so a variable that needs a nonlinear transformation can have one without forcing the same on the others. And the static context enters the weight computation, which is what allows a rural agent and a city-centre agent to rely on different inputs at the same hour.

Using the weights. Averaging `past_w` over the backtest test folds gives a global importance ranking, which is the chart of Figure 8.26 and goes

directly into the validation pack that the CBK Digital Credit Providers Regulations expect: it documents which inputs the model relies on, in a form a non-specialist reviewer can read, and it is cheap enough to regenerate on every retrain, so it can be tracked over time as a drift signal.

If the county embedding ranked in the top three, the correct response is investigation rather than removal. County is a plausible proxy for ethnicity and income, so a high weight raises a fairness question, but it is also a legitimate carrier of genuine geographic signal such as market days and harvest calendars. Three steps follow. First, audit the residuals by county: a consistently negative mean error in particular counties means those agents are systematically under-provisioned and losing income, which is a defect regardless of what the aggregate MASE says. Second, test whether the county signal survives once the underlying mechanisms are supplied explicitly, by adding market-day and harvest-calendar features and refitting; if the county weight collapses, it was standing in for those. Third, if it does not collapse, document the residual dependence, the segment-level error table and the mitigation considered, and escalate the decision to keep it rather than making it silently in a configuration file. Deleting the feature without any of this merely hides the dependence, since the model will reconstruct county from the agent embedding.

B.1 Chapter 9: Transformers, LLMs and retrieval

Solution to Exercise 9.1. **(a)** With $d_k = 2$ the scale factor is $1/\sqrt{2}$, and $QK^\top = \left(\begin{smallmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{smallmatrix}\right)$, so

$$S = \frac{QK^\top}{\sqrt{2}} = \begin{pmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{pmatrix}.$$

(b) Write $e = \exp(0.7071) = 2.0281$, so that $e^2 = \exp(1.4142) = 4.1133$. Row 1 has exponentials $(e, 1, e)$ summing to 5.0562, and row 2 has the same values permuted; row 3 has exponentials (e, e, e^2) summing to 8.1695. Normalising each row,

$$\alpha = \begin{pmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{pmatrix},$$

and multiplying by V,

$$O = \alpha V = \begin{pmatrix} 1.2033 & 1.1922 \\ 0.7967 & 2.0056 \\ 1.0000 & 1.4965 \end{pmatrix}.$$

Two sanity checks worth making every time: each row of α sums to one, and every entry of $\boldsymbol{O}$ lies inside the range spanned by the corresponding column of $\boldsymbol{V}$, here $[0,2]$ and $[0,4]$. Attention averages; it never extrapolates.

(c) With the causal mask, row 1 can attend only to position 1, so $\alpha_1 = (1,0,0)$ and $\boldsymbol{o}_1 = \boldsymbol{v}_1 = (2,0)$. Row 2 attends to positions 1 and 2 with exponentials $(1,e)$, giving $\alpha_2 = (0.3302,\ 0.6698)$ and $\boldsymbol{o}_2 = (0.6605,\ 2.6790)$. Row 3 is unchanged, since it could already see everything. The first output row is simply a copy of the first value vector, which is exactly right: at the first position a decoder has seen no context at all, and any other behaviour would mean the model was reading tokens it will be asked to predict.

Solution to Exercise 9.2. **(a)** A dot product of two d_k-dimensional vectors is a sum of d_k independent terms, each of variance one when the components have unit variance; variances of independent quantities add, so $\mathrm{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_k = 64$ and the raw scores have standard deviation $\sqrt{64} = 8$.

(b) Without the scaling,

$$\mathrm{softmax}(8.0,\ 2.4,\ -1.6) = (0.9962,\ 0.0037,\ 0.0001),$$

whereas dividing by $\sqrt{d_k} = 8$ turns the logits into $(1.0,\ 0.3,\ -0.2)$ and gives

$$\mathrm{softmax}(1.0,\ 0.3,\ -0.2) = (0.5562,\ 0.2762,\ 0.1675).$$

(c) For the largest coordinate, $p(1-p) = 0.9962 \times 0.0038 = 0.0037$ unscaled, against $0.5562 \times 0.4438 = 0.2468$ scaled: a factor of roughly 66. The unscaled model is not merely peaked, it is *stuck*. Its attention distribution has collapsed onto one token, the output is a copy of one value vector, and the gradient reaching the query and key projections is two orders of magnitude smaller, so the model cannot learn its way out. The $\sqrt{d_k}$ is therefore not cosmetic normalisation but the condition under which the layer is trainable at all, and it is the same saturation pathology as a sigmoid driven to its tails.

Solution to Exercise 9.3. **(a)** The ratio of attention cost to projection-plus-MLP cost is $2T^2d / 12Td^2 = T/(6d)$. With $d = 4096$, $6d = 24{,}576$:

T	512	2048	8192	32768
ratio $T/(6d)$	0.021	0.083	0.333	1.333
attention share of the layer	2.0%	7.7%	25.0%	57.1%

(The share is ratio$/(1+\text{ratio})$.)

(b) Attention passes half the layer when the ratio reaches 1, that is at $T = 6d = 24{,}576$ tokens.

(c) The compliance assistant. At $T = 64$ attention is 0.26% of the classifier's layer cost, so optimising it would be optimising nothing, whereas at $T \approx 6000$ the assistant is already paying roughly a quarter of its compute

on attention and that share grows with every extra retrieved chunk. The one-sentence argument: effort spent shortening the assistant's context, by reranking to five chunks instead of stuffing fifty, buys a super-linear saving, while the same effort spent on the classifier buys nothing measurable.

Solution to Exercise 9.4. **(a)** Fertility is tokens per word: $156/120 = 1.30$ for tokenizer A and $372/120 = 3.10$ for tokenizer B, a ratio of 2.38.

(b) The corpus is $2 \times 10^6 \times 24 = 48$ million words. Under A that is $48 \times 1.30 = 62.4$ million tokens; under B, $48 \times 3.10 = 148.8$ million. At USD 0.15 per thousand tokens the input-token cost is USD 9,360 under A and USD 22,320 under B, a difference of USD 12,960, which at 130 KES/USD is about **KES 1.68 million** per year, for identical data and identical work.

(c) The calculation prices only the tokens. It misses that the same fertility shrinks the *effective context window*: a 8k-token budget holds roughly 2,580 words under B against 6,150 under A, so tokenizer B forces shorter prompts, fewer retrieved chunks or more truncation. It also misses the quality effect, since a word shattered into five fragments carries less usable signal than one that survives as a single token, which typically shows up as a per-language accuracy gap rather than as a line on an invoice.

Solution to Exercise 9.5. **(a)** $D \approx 20N = 20 \times 1.5 \times 10^9 = 3.0 \times 10^{10}$ tokens (30 billion). Then $C \approx 6ND = 6 \times 1.5 \times 10^9 \times 3.0 \times 10^{10} = 2.7 \times 10^{20}$ FLOPs.

(b) At 1.2×10^{14} FLOP/s sustained, $C/(1.2 \times 10^{14}) = 2.25 \times 10^6$ seconds $= 26.0$ GPU-days. Note the policy implication: this is one card for under a month, or eight cards for three days, which is why compute-optimal pretraining at the 1.5B scale is within reach of a university lab, unlike anything above about 10B.

(c) Chinchilla minimises *training* loss for a fixed training budget, but a deployed model pays inference cost on every request for the life of the product, and that cost scales with N, not with D. Spending extra training tokens on a smaller model is therefore a rational trade: you accept a worse loss-per-training-FLOP in exchange for a permanently cheaper model to serve, and for PesaCred, where the classifier scores millions of messages a month, the serving side dominates the arithmetic entirely.

Solution to Exercise 9.6. **(a)** For one 4096×4096 matrix at $r = 16$, the adapter holds $r(d + k) = 16 \times 8192 = 131{,}072$ parameters against $dk = 16{,}777{,}216$, a fraction of **0.78**%.

(b) Across 32 layers with four projections each,

$$32 \times 4 \times 131{,}072 = 16{,}777{,}216 \approx \mathbf{16.8} \text{ million parameters.}$$

At two bytes each in bfloat16, the saved adapter is about **33.6** MB.

(c) Full fine-tuning of a 7B model produces a 14 GB artifact per task, roughly 400 times larger. Serving eight tasks means eight 14 GB checkpoints, none of which can share GPU memory, so you need either eight

loaded models or eight sequential loads. With adapters you load *one* base model and hold all eight adapters simultaneously in about 270 MB, switching between them per request. The operational consequences follow immediately: one deployment artifact to version instead of eight, a storage bill three orders of magnitude smaller, and, because $\boldsymbol{W}_0 + \frac{\alpha}{r}\boldsymbol{BA}$ can be merged offline into a matrix of the original shape, no inference penalty for any of it.

Solution to Exercise 9.7. **(a)** Base weights: $7 \times 10^9 \times 4.13/8 = 3.61$ GB. Adapter: $40 \times 10^6 \times 2 = 0.08$ GB. Optimizer and the associated master copy: $40 \times 10^6 \times 14 = 0.56$ GB. Total fixed cost $\approx$ **4.25** GB.

(b) On a 16 GB card, roughly $16 - 4.25 =$ **11.75** GB remains for activations, attention workspace, the CUDA context and fragmentation. In practice budget one to two GB for context and fragmentation, leaving about 10 GB of usable activation memory, which is comfortable at $T = 64$ for SMS and tight at multi-thousand-token sequences.

(c) Any two of the following. Reduce the per-device batch size and raise the gradient-accumulation steps, which leaves the effective batch unchanged. Enable gradient checkpointing, trading roughly 30% extra compute for a large activation saving. Shorten the maximum sequence length to the true 99th percentile of the corpus rather than to a round number. Or use paged optimizers, so that transient spikes spill to host memory instead of aborting the run.

Solution to Exercise 9.8. The ranks are $1, 3, 2, \text{miss}, 1, 5, 2, 1, \text{miss}, 4$ over ten queries. A miss contributes zero to every metric.

(a) Recall@3 counts ranks ≤ 3: queries 1, 2, 3, 5, 7, 8, which is six of ten, so **0.60**.

(b) Recall@5 additionally counts ranks 5 and 4: eight of ten, so **0.80**.

(c) MRR $= \frac{1}{10}\left(1 + \frac{1}{3} + \frac{1}{2} + 0 + 1 + \frac{1}{5} + \frac{1}{2} + 1 + 0 + \frac{1}{4}\right) = \frac{4.7833}{10} =$ **0.4783**.

(d) The reranker moves every gold chunk already in the top 5 to rank 1, so the eight hits all become rank 1 and the two misses stay missing. Recall@3 rises to **0.80**, Recall@5 stays at **0.80**, and MRR rises to **0.80**. Recall@5 is the metric the reranker cannot improve, and the reason is structural rather than incidental: a reranker only reorders the candidate set handed to it by the retriever, so it can never surface a chunk the first-stage retrieval failed to return. This is the practical argument for retrieving generously, say $k = 50$, before reranking to five: recall is bought in the first stage and precision in the second, and no amount of second-stage cleverness recovers a document that was never a candidate.

Solution to Exercise 9.9. **(a)** All five vectors are unit length, so the cosine similarity is just the inner product with $\boldsymbol{q} = (0.6, 0.8, 0)$:

$$\cos(\boldsymbol{q}, \boldsymbol{a}) = 0.96, \qquad \cos(\boldsymbol{q}, \boldsymbol{d}) = 0.936, \qquad \cos(\boldsymbol{q}, \boldsymbol{b}) = 0.80, \qquad \cos(\boldsymbol{q}, \boldsymbol{c}) = 0.36.$$

The top-2 ranking is $\boldsymbol{a}$ then $\boldsymbol{d}$.

(b) For unit vectors the denominator of the cosine is 1, so cosine similarity and inner product coincide and `IndexFlatIP` computes exactly the quantity we want, with no per-query normalisation cost. If the vectors are not normalised, inner product silently becomes $\|\boldsymbol{e}_a\|\|\boldsymbol{e}_b\|\cos(\cdot)$ and the index starts ranking by *magnitude* as well as by direction. Long chunks, which tend to have larger norms, then win retrievals they do not deserve. The failure is quiet: nothing errors, recall just degrades, which is why `normalize_embeddings=True` in Listing 9.6 is load-bearing rather than decorative.

(c) Chunk $\boldsymbol{c}$ is the correct answer and ranks last, which is a first-stage retrieval failure. The component that exists to fix it is the **cross-encoder reranker** in Figure 9.17: because it scores the query and the chunk jointly rather than comparing two independently computed vectors, it can recognise a relevant passage whose bi-encoder embedding sits in the wrong neighbourhood, provided the passage is inside the candidate set. On the chunking side, the most likely cause is a chunk that mixes several topics, so its single embedding is an average that resembles nothing in particular. Splitting on clause boundaries with a smaller token cap, as in Section 9.6.4, gives each claim its own vector. Adding hybrid BM25 scoring would also help, since the exact statutory phrase probably appears verbatim in $\boldsymbol{c}$.

Solution to Exercise 9.10. **(a)** Of 50 unanswerable questions, 45 are refused, so the correct-refusal rate is $45/50 = \mathbf{0.90}$. Of the 60 refusals, $60 - 45 = 15$ fell on answerable questions, so the over-refusal rate is $15/150 = \mathbf{0.10}$. The precision of refusals is $45/60 = \mathbf{0.75}$.

(b) Raising the retrieval score threshold means fewer chunks clear the bar, so the system refuses more often. The correct-refusal rate rises towards 1.0, the over-refusal rate also rises as genuinely answerable questions start being turned away, and refusal precision *falls*, because the extra refusals are disproportionately wrong ones. Faithfulness improves, and this is the trap: faithfulness is computed only over answers actually produced, so a system that answers less looks more faithful while helping less. That is precisely the degenerate solution Definition 9.5 exists to expose, and it is why the two refusal rates must be reported together with faithfulness rather than after it.

(c) The choice follows the asymmetric-cost reasoning of Chapter 3, applied to this use case rather than to the credit model. The costs are wildly unequal: an over-refusal wastes an analyst's time, perhaps twenty minutes of looking the clause up manually, whereas a confident hallucinated statement about Section 35 can propagate into a lending policy and an ODPC finding, with regulatory and reputational costs orders of magnitude larger. PesaCred should therefore operate at a *high-threshold*, high-refusal point, accepting an over-refusal rate around 0.10 to 0.15 in exchange for a correct-refusal rate above 0.95, and should make the refusal useful by returning

the top retrieved clauses alongside the refusal so the analyst starts from something rather than nothing. Two conditions make this defensible rather than lazy: the refusal must be visibly a refusal and never a hedged half-answer, and the over-refusal rate must be monitored as a service-level metric, because an assistant that refuses a third of real questions will simply be abandoned, which returns the organisation to unassisted manual search and helps nobody.

B.2 Chapter 10: MLOps, packaging, serving and CI/CD

Solution to Exercise 10.1. **(a)** By Proposition 10.1, $t^\star = 800/(800 + 10{,}000) = 800/10{,}800 = \mathbf{0.0741}$.

(b) Expected cost per applicant is total cost divided by the number of applicants. At $t = 0.5$,

$$\frac{260 \times 10{,}000 + 150 \times 800}{10{,}000} = \frac{2{,}600{,}000 + 120{,}000}{10{,}000} = \mathbf{272}\ \text{KES},$$

and at $t^\star$,

$$\frac{90 \times 10{,}000 + 1{,}900 \times 800}{10{,}000} = \frac{900{,}000 + 1{,}520{,}000}{10{,}000} = \mathbf{242}\ \text{KES}.$$

The cost-optimal threshold declines far more people, 1,900 good borrowers against 150, and is still KES 30 per applicant cheaper, because each avoided default saves as much as twelve and a half wrongful declines cost.

(c) The new threshold is $t^\star = 1{,}500/11{,}500 = \mathbf{0.1304}$. Recomputing the two operating points of part (b) with $C_{FP} = 1{,}500$: at $t = 0.5$ the cost is $(2{,}600{,}000 + 150 \times 1{,}500)/10{,}000 = \mathbf{282.5}$ KES, while the *old* optimum $t = 0.074$ now costs $(900{,}000 + 1{,}900 \times 1{,}500)/10{,}000 = \mathbf{375}$ KES. The threshold that was optimal yesterday is now worse than the naive 0.5, and neither is the new optimum, which lies at 0.13 and requires a fresh confusion matrix to price. The lesson for versioning is direct: a change to the cost matrix is a change to the decision function of Definition 10.1, so it must produce a new model version, re-run the gates and pass through approval like any retraining. A cheap unit test enforces the link, asserting that the threshold stored in the metadata equals $C_{FP}/(C_{FP} + C_{FN})$ computed from the cost matrix stored beside it, so that nobody can edit one without the other.

Solution to Exercise 10.2. **(a)** By Proposition 10.2, the probability that none of the calls is slow is q^n, so the fraction hitting at least one component's p_{99} is $1 - 0.99^5 = 1 - 0.9510 = \mathbf{4.9}\%$ with five services, and $1 - 0.99^8 = 1 - 0.9227 = \mathbf{7.7}\%$ with eight.

(b) We need $q^5 \geq 0.99$, so $q \geq 0.99^{1/5} = 0.99799$. Each component must meet its target at roughly the $\mathbf{p_{99.8}}$ level, a much harder target than its own p_{99}. This is the quantitative reason why every added dependency in a request path is a latency liability even when it is individually fast.

(c) The 900 milliseconds exist precisely because of part (a). The components' budgets in Figure 10.11 are typical allocations, and with five dependencies roughly one request in twenty will see at least one of them run into its tail. Headroom absorbs that tail, allows one retry of a timed-out feature lookup (250 ms) without breaching the SLA, and tolerates the variability of a telco gateway that PesaCred does not control. A budget that sums exactly to the SLA guarantees violations on the tail, which on USSD means a borrower whose session simply drops.

Solution to Exercise 10.3. **(a)** The offered load is $a = \lambda \bar{S} = 120 \times 0.025 = \mathbf{3.0}$ workers' worth of work, so $c = \lceil 3.0/0.6 \rceil = \mathbf{5}$ workers, at utilisation $3/5 = 0.60$.

(b) At month-end, $a = 300 \times 0.025 = \mathbf{7.5}$, and $c = \lceil 7.5/0.6 \rceil = \lceil 12.5 \rceil = \mathbf{13}$ workers, at utilisation $7.5/13 = 0.58$.

(c) Thirteen copies of a 350 MB model need $13 \times 0.35 = \mathbf{4.55}$ GB for the models alone, before the Python interpreter, TensorFlow's own footprint and request buffers. This is the hidden cost of the process-per-worker design: memory scales with workers, and a larger model can make memory, not CPU, the binding constraint.

(d) By Little's law, $L = \lambda W = 300 \times 0.040 = \mathbf{12}$ requests in flight on average. Note that this exceeds the 7.5 requests actually being served (a), the difference of 4.5 being requests that are queued or in transit; that queue is what the utilisation ceiling keeps from growing without bound.

Solution to Exercise 10.4. **(a)** From $T(b)/b = 8/b + 0.15$: at $b = 1$, $\mathbf{8.15}$ ms per row; at $b = 32$, $0.25 + 0.15 = \mathbf{0.40}$ ms; at $b = 256$, $0.03125 + 0.15 = \mathbf{0.181}$ ms.

(b) Through the online endpoint, $3 \times 10^6 \times 8.15$ ms $= 24{,}450$ s, about $\mathbf{6.8}$ hours. Through the batch endpoint, $3 \times 10^6 \times 0.18125$ ms $= 543.75$ s, about $\mathbf{9.1}$ minutes, a speed-up of roughly $\mathbf{45}$ times. The difference decides whether the nightly refresh fits in its window, and it also frees the online workers of Exercise 10.3 for the daytime traffic they were sized for.

(c) USSD requests arrive one at a time, from independent borrowers. To form a batch the service would have to *wait* for other requests to arrive, and that waiting is queueing delay added to every borrower's response. At 120 requests per second, filling a batch of 32 takes about a quarter of a second, so the average request would wait more than a tenth of a second, most of the model's 150 ms budget, to save 7.75 ms of compute. Dynamic batching with a few milliseconds of waiting is sometimes worth it for GPU models with a large fixed cost, which is one argument for the model server of Section 10.3.2.4; for a small CPU model it is not.

Solution to Exercise 10.5. **(a)** Using $t = S/B$ with S in bits: $2.1 \times 8 \times 10^9/10^7 = 1{,}680$ s $=$ **28** minutes, and $0.9 \times 8 \times 10^9/10^7 = 720$ s $=$ **12** minutes.

(b) Only the changed application layer is pulled: $0.04 \times 8 \times 10^9/10^7 =$ **32** seconds. Everything below it is already cached at the branch.

(c) With `COPY . .` placed before `pip install`, any change to any file in the repository invalidates the copy layer and therefore every layer above it, including the dependency layer holding TensorFlow. The release then ships the dependency layer as well as the code. If that layer is, say, 0.7 GB, the pull takes $0.7 \times 8 \times 10^9/10^7 = 560$ s, about 9.3 minutes instead of 32 seconds, on every release, for a change of perhaps one line. The same ordering mistake also slows every CI build, since the cache is invalidated there too.

Solution to Exercise 10.6. **(a)** Configuration A: mean 0.8080, standard deviation 0.00515, half-width $2.776 \times 0.00515/\sqrt{5} = 0.0064$, so the 95% interval is [**0.8016**, **0.8144**]. Configuration B: mean 0.8028, standard deviation 0.00814, half-width 0.0101, interval [**0.7927**, **0.8129**].

(b) The claim compares an extreme of one sample with the opposite extreme of the other, which is the most optimistic comparison available and would "show" that almost any configuration beats almost any other. The honest comparison is between the means, and there A is ahead by 0.005. Even that difference is not established: the intervals overlap substantially, and a Welch t-test gives $p \approx 0.27$. The correct conclusion is that five seeds cannot distinguish the two configurations, and, if anything, the evidence leans the other way from the colleague's claim. B's larger standard deviation is also worth noticing, since a configuration that is less stable across seeds is less attractive for a pipeline that retrains weekly.

(c) Three changes. Run more seeds, since the half-width shrinks like $1/\sqrt{k}$ and twenty seeds halve it. *Pair* the seeds, using the same seed and data order for both configurations so that shared randomness cancels in the differences; with these numbers the paired t-statistic is about -1.41, still well short of the critical 2.78, which confirms that no conclusion is warranted. And compare on the business metric, expected cost at $t^\star$ on the frozen holdout, rather than AUC, because that is the number the promotion gate will use.

Solution to Exercise 10.7. **(a)** With $n = 4{,}000$, the standard error is $310/\sqrt{4{,}000} = 4.90$ and the interval is $-9 \pm 1.96 \times 4.90 = [-\mathbf{18.6}, \mathbf{0.6}]$. Its upper end is above zero, so the gate of Equation (10.9) **fails**, even though the challenger is in truth cheaper. With $n = 10{,}000$, the standard error is 3.10 and the interval is $[-\mathbf{15.1}, -\mathbf{2.9}]$, entirely below zero: the gate **passes**.

(b) The gate passes when $1.96 \times 310/\sqrt{n} < 9$, that is $\sqrt{n} > 67.5$, or $n > 4{,}557.8$. The smallest holdout is **4,558** applicants. This is a power calculation in disguise, and it should be done before the holdout is frozen: a holdout too small to detect the savings the business cares about produces

a gate that blocks good models at random.

(c) Most applicants receive the same decision from both models, so their paired difference is exactly zero, and the standard deviation of d_i is small. The cost of a single applicant, by contrast, varies enormously: with the $t^\star$ error rates of Exercise 10.1 (0.9% defaults approved at KES 10,000, 19% good borrowers declined at KES 800), its standard deviation is $\sqrt{0.009 \times 10{,}000^2 + 0.19 \times 800^2 - 242^2} \approx 981$ KES. An unpaired comparison of two independent means has standard error $\sqrt{2} \times 981/\sqrt{n} \approx 1{,}388/\sqrt{n}$, against $310/\sqrt{n}$ paired, so it needs $(1{,}388/310)^2 \approx \mathbf{20}$ times as many applicants, about 91,000, for the same power. Pairing is not a refinement; it is what makes the gate feasible with the data PesaCred has.

Solution to Exercise 10.8. **(a)** Nairobi: $1{,}820/2{,}000 = \mathbf{0.910}$. North Eastern: $95/120 = \mathbf{0.792}$. The gap is $\Gamma = 0.910 - 0.792 = \mathbf{0.118} > 0.10$, so on point estimates the gate **fails**.

(b) The 95% Wilson interval for Nairobi is $[\mathbf{0.897}, \mathbf{0.922}]$; for North Eastern it is $[\mathbf{0.711}, \mathbf{0.855}]$. The first is narrow because it rests on two thousand borrowers; the second is more than five times wider because it rests on 120.

(c) The intervals imply a plausible range for the gap from $0.897 - 0.855 = 0.042$, which would pass comfortably, to $0.922 - 0.711 = 0.211$, which would fail badly. A point-estimate gate therefore decides this case largely by chance. Which bound to use depends on who bears each error. A false pass deploys a model that systematically under-serves good borrowers in a region that is already under-banked, a harm that falls on borrowers, that DPA-motivated proxy-discrimination concerns make serious, and that is invisible in aggregate metrics. A false fail delays a better model, a cost borne by the lender. The asymmetry argues for a rule in two tiers: *block* promotion whenever the point estimate or the pessimistic bound exceeds τ, as here, and require a named risk officer to review the evidence and either investigate the region's data coverage or collect more outcomes before an exception is granted. Using the optimistic bound would let small, poorly served regions pass by default, which inverts the purpose of the gate.

Solution to Exercise 10.9. **(a)** $\hat{p} = 184/200 = \mathbf{0.920}$ and, from Equation (10.11) with $z = 1.96$, LB $= \mathbf{0.874}$, which clears $\tau = 0.85$: the gate **passes**.

(b) $\hat{p} = 176/200 = \mathbf{0.880}$ and LB $= \mathbf{0.828}$, below τ: the gate **fails**. The prompt edit cost eight golden cases, and the gate catches it even though the point estimate still looks respectable.

(c) $\hat{p} = 46/50 = 0.920$, identical to part (a), but LB $= \mathbf{0.812}$, which fails. The same observed quality fails on the smaller set because fifty cases cannot rule out a true pass rate in the low eighties. A golden set must be large enough that a system at the target quality can pass at all; with $\tau = 0.85$ and a system around 0.92, fifty cases are too few and two hundred are about the minimum. The quicker set would not speed the team up; it

would block every release.

Solution to Exercise 10.10. **(a)** The cost gate passes, since the upper end of $[-14, -2]$ is below zero. The fairness gate passes, $0.08 \leq 0.10$. The behavioural gate passes, $0.6\% \leq 1\%$. The holdout identity check passes. The latency gate **fails**: $162 > 150$ ms. Every gate must pass, so v5 **may not** become `@champion`, however good its accuracy. The latency budget is not a preference; it is derived from the USSD step time in Figure 10.11.

(b) Two families of options. Make the model faster: distil v5 into a smaller student network, prune or quantise it, or remove expensive features whose contribution is marginal, then re-run all gates, since each change creates a new version. Or change where the time goes: move any feature computation still done at request time into the nightly batch of Section 10.3.1, which frees budget for inference, or serve the model from a dedicated, faster model server (Section 10.3.2.4), at the price of one extra hop that must itself fit the budget. What is *not* an option is raising the gate's threshold on the engineering team's own authority; if the product owner decides to reallocate the headroom, the budget and the gate change together, as a recorded decision.

(c) Rollback is the alias move of Section 10.4.3: `@champion` is repointed to the version tagged `previous_champion`, v4, and the CD workflow rebuilds or redeploys the v4 image and routes the canary's traffic back to it. The records produced are the alias-change event in the registry, the GitHub Actions run with its approval, the deployment log, and the decision logs, in which every request carries its `model_version`, so the exact set of borrowers decided by v5 can be listed. Defaults cannot yet say whether v5 was wrong because loans made a week ago are nowhere near their due dates; outcomes arrive thirty or more days after disbursement. The falling approval rate is only a *proxy*. It could mean v5 is stricter than intended, or that the applicant population shifted, for example after salary day. The way to tell them apart is to compare v5's and v4's decisions on the same requests, which is exactly what a shadow deployment would have provided before the canary, and why shadow comes first for credit models.

B.3 Chapter 11: monitoring, drift and system design

Solution to Exercise 11.1. **(a)** From Proposition 11.1,

$$w = \frac{\pi'}{\pi} \cdot \frac{1-\pi}{1-\pi'} = \frac{0.07}{0.04} \cdot \frac{0.96}{0.93} = \mathbf{1.806}.$$

The deployed model's odds must be multiplied by about 1.8 to reflect the new prior.

(b) With $p = 0.05$, the old odds are $0.05/0.95 = 0.05263$; multiplying by w gives 0.09506, so

$$\tilde{p} \;=\; \frac{0.09506}{1 + 0.09506} \;=\; \mathbf{0.0868}.$$

The decision **changes**: the raw score 0.05 is below $t^\star = 0.074$ and would be approved, while the corrected probability 0.0868 is above it and is declined. This is the whole point of the correction. The model's *ranking* of applicants is untouched, but the applicants near the threshold are exactly the ones whose decision the prior change flips, and they are numerous.

(c) From Equation (11.3),

$$t' \;=\; \frac{t^\star}{w(1 - t^\star) + t^\star} \;=\; \frac{0.0741}{1.806 \times 0.9259 + 0.0741} \;=\; \mathbf{0.0424}.$$

Applying $t' = 0.042$ to the uncorrected scores produces exactly the same decisions as applying $t^\star$ to the corrected ones, which is a useful check: the applicant of part (b), scored 0.05, is declined under both.

Moving the threshold, or equivalently recalibrating, is preferable to retraining here for three reasons. It is *correct*: under label shift the likelihood ratio $P(x \mid Y)$ is unchanged, so the model's discrimination is still valid and only its scale is wrong, and Proposition 11.1 fixes the scale exactly. It is *fast*: π' can be estimated from the most recent matured cohort in minutes, while a retraining cycle costs a week and passes through every gate and an approval. And it is *safer*: retraining on data from a drought period would bake a temporary condition into the model's weights, and would have to be undone when the rains return. The corresponding risk is that the assumption $P(X \mid Y)$ constant may be false, so the correction should be verified on the first cohort of matured labels, and reversed if the discrimination has also degraded, which would indicate concept drift rather than label shift.

Solution to Exercise 11.2. **(a)** With $p_b = 0.1$ in every bin,

$$\mathrm{PSI} \;=\; \sum_{b=1}^{10} (q_b - 0.1) \ln \frac{q_b}{0.1} \;=\; \mathbf{0.149},$$

which falls in the middle band of Figure 11.16, between 0.10 and 0.25: **investigate**, but not yet an automatic alarm to act on.

(b) The first decile contributes $(0.191 - 0.1)\ln(1.91) = 0.0589$ and the tenth contributes $(0.048 - 0.1)\ln(0.48) = 0.0382$. Together they account for $0.0971/0.149 = \mathbf{65}\%$ of the total, and the two middle deciles contribute almost nothing. This attribution is the practical advantage of PSI over a single summary number: the alert arrives with an explanation attached.

(c) The applicant population has shifted towards lower monthly inflows: the lowest reference decile now holds nearly twice its expected

share and the highest holds half. Roughly, incomes have fallen by about a fifth.

This evidence establishes *covariate shift* in that one feature, and nothing more. It cannot distinguish between a genuine change in the applicant population, a change in the marketing mix that attracts poorer applicants, and a data-pipeline bug that has rescaled the field, which is why the first triage question of Figure 11.28 is about units and schemas. It also says nothing about $P(Y \mid X)$: whether lower inflows still mean what they used to mean is a question only matured labels can answer. If the relationship is unchanged, the model may be perfectly fine on the new population while approving fewer people, which is a business event rather than a model failure.

Solution to Exercise 11.3. **(a)** The critical value is $1.36\sqrt{2/n}$, so

$$n = 2{,}000: \; D_{\text{crit}} = 1.36\sqrt{2/2000} = \mathbf{0.043}, \qquad n = 50{,}000: \; D_{\text{crit}} = \mathbf{0.0086}.$$

With $D = 0.02$, the test **does not reject** at $n = 2{,}000$ and **rejects** at $n = 50{,}000$. The data have not changed between the two cases; only the amount of it has.

(b) Solving $1.36\sqrt{2/n} < 0.02$ gives $n > 2(1.36/0.02)^2 = \mathbf{9{,}248}$ observations per window.

(c) At 1,800 decisions a day, that is $9{,}248/1{,}800 \approx \mathbf{5.1}$ days. Any window of a week or more, which is the minimum Section 11.3.1 recommends so that weekday effects cancel, is already large enough to make a two-hundredths-of-a-standard-deviation shift "significant". A monitoring rule based on KS p-values would therefore fire on this feature every week regardless of whether anything worth acting on had happened, and the rule would get more sensitive as PesaCred grew. The remedy is Section 11.3.4: alert on an effect size, here a normalised Wasserstein distance or a PSI, whose value is about 0.001 for a shift this small and does not move with n.

Solution to Exercise 11.4. **(a)** $\Pr(\text{at least one}) = 1 - 0.95^{40} = \mathbf{0.871}$. The expected number of false alarms is $m\alpha = \mathbf{2.0}$ per night and **60** per 30-day month.

(b) With Bonferroni, each feature is tested at $\alpha/m = 0.00125$, so

$$\Pr(\text{at least one}) = 1 - (1 - 0.00125)^{40} = \mathbf{0.049},$$

and the expected number of false alarms is **0.05** per night and **1.5** per month.

(c) A 30% share of 40 columns is **12** columns. Under $\text{Bin}(40, 0.05)$, $\Pr(X \geq 12) = \mathbf{3.7 \times 10^{-7}}$, which is about one stable day in seven thousand years of nightly checks.

(d) The drift-share rule, with the per-column threshold left uncorrected, is the best default, for a reason that goes beyond its false-alarm rate. Bonferroni buys its quiet by making each individual test insensitive, so a genuine, large shift in a single important feature can be missed. The share rule keeps each column's test sensitive and asks for corroboration before raising an alarm, which matches how real shifts behave: a change in the applicant population moves many correlated features at once, while noise moves them independently. In practice one deploys the share rule for the dataset-level alarm *and* a separate, stricter rule on the two or three features with the highest attribution, plus the score itself, so that an isolated but important movement is not silently outvoted by thirty-seven stable columns.

Solution to Exercise 11.5. **(a)** Per-feature statistics compare marginal distributions only. A domain AUC of 0.62 with every marginal stable says that the windows differ in their *joint* structure: the features have stopped co-occurring as they used to. A plausible story for PesaCred is a new acquisition channel that brings applicants with long mobile-money tenure but low transaction counts, a combination that was rare before. Each marginal is unchanged because the new applicants replace old ones with the same values on each axis separately.

(b) Solving $\Phi(\delta/\sqrt{2}) = 0.62$ gives $\delta/\sqrt{2} = \Phi^{-1}(0.62) = 0.3055$, so $\delta = \mathbf{0.43}$ standard deviations. A single-feature shift of that size would be obvious in any univariate test, which is a useful calibration: the joint change that the classifier has found is comparable in magnitude to a shift that no per-feature monitor could have missed. That is what makes the result worth an investigation rather than a footnote.

(c) Two diagnostics. First, inspect the classifier's feature importances or its permutation importances, which name the features involved in the separation, and then plot the top two against each other for both windows to see the interaction directly. Second, check whether the separable rows share a value on some non-monitored field, for instance an acquisition channel, an application-form version or a handset type, which would identify the population responsible.

The mistake to look for in the construction is leakage of an identifier that encodes time: a timestamp, an auto-incrementing application number, a batch or file identifier. Any of these lets the classifier recover which window a row came from perfectly, producing an AUC near 1.0 that carries no information about drift. A related and subtler error is reporting an in-sample AUC: a flexible model can separate any two finite samples, so only out-of-fold predictions, as in Listing 11.2, mean anything.

Solution to Exercise 11.6. **(a)** At 1,800 decisions a day: **12,600** decisions in the first seven days, and $1{,}800 \times 90 = \mathbf{162{,}000}$ by the time matured labels could reveal the problem, of which 149,400 fall between day 7 and day 90.

(b) The extra cost is $268 - 223 =$ KES 45 per applicant. Acting on the

day-7 signal and retraining within a week means the bad model decides for 14 days, so

$$1{,}800 \times 14 \times 45 \;=\; \text{KES } \mathbf{1{,}134{,}000}.$$

Waiting for matured labels, detecting on day 90 and retraining within the following week means 97 days of exposure,

$$1{,}800 \times 97 \times 45 \;=\; \text{KES } \mathbf{7{,}857{,}000},$$

about seven times as much, a difference of roughly KES 6.7 million. That difference is the financial argument for label-free monitoring, and it is worth putting in front of a board that is deciding whether monitoring deserves an engineer.

(c) Two earlier signals. The *approval rate*, available in real time, moves as soon as the score distribution moves, and would have shown a change within a day or two; it tells you that the model's behaviour has changed, but not whether the change is justified by a genuinely riskier population. The *data-quality checks* on each batch, also available within minutes, would have told you whether the change is explained by nulls, out-of-vocabulary codes or a rescaled field, which is the difference between a pipeline fix and a retraining. Neither can establish concept drift, since both are computed from inputs and outputs alone; they buy time and narrow the hypothesis, which in this case is worth several million shillings.

Solution to Exercise 11.7. **(a)** With $t^\star = 0.0741$, the first seven applicants ($p \le 0.06$) are approved and the last three (0.09, 0.15, 0.40) are declined, an approval rate of 70%. The estimated cost per applicant from Equation (11.9) is

$$\widehat{\text{Cost}} = \tfrac{1}{10}\big[(50+100+200+300+400+500+600) + (728+680+480)\big] \;=\; \text{KES } \mathbf{403.8}.$$

The approved terms are $p_i C_{FN}$ and the declined ones $(1-p_i)C_{FP}$; note that the declined applicant with $p = 0.09$ contributes KES 728, almost as much as the riskiest approved one, which is a good illustration of how expensive declining is when the cost matrix is this asymmetric.

(b) Multiplying each applicant's odds by 1.5 gives true probabilities 0.0075, 0.0149, 0.0297, 0.0443, 0.0588, 0.0732, 0.0874, 0.1292, 0.2093, 0.5000. The model's scores are unchanged, so it makes the *same* decisions: it still approves the first seven, including the applicant whose true probability is now 0.0874, above the threshold. The realised expected cost is

$$\text{Cost} = \tfrac{1}{10}\big[(74.8+149.3+297.0+443.3+588.2+731.7+873.8) + (696.7+632.6+400.0)\big] \;=\; \text{KES } \mathbf{488.7},$$

an increase of KES 84.9 per applicant, or 21%. For comparison, a model that knew the true probabilities would decline the seventh applicant as well and

pay KES 474.4, so about KES 14 of the increase is the model's error and the rest is the genuinely riskier population.

(c) The estimate of part (a) is computed *from the model's own scores*, which did not change, so it still reports KES 403.8 while the truth is KES 488.7. The estimator is unbiased only when the probabilities remain calibrated, which Equation (11.9) assumes and which fails here by construction: this is concept drift in the sense of Definition 11.1, since $P(Y \mid X)$ has moved while the model has not. Worse, the failure is in the optimistic direction, so the monitor is reassuring precisely when it should not be.

What would reveal it is the verdict layer: performance on matured labels, where the realised default rate among approved applicants exceeds the predicted one, which is a calibration failure on the labelled cohort rather than a drift statistic. An early warning is also available: the realised default rate on a *proxy*, such as first-payment behaviour at day seven, can be compared with the rate the model predicted for the same cohort, which detects mis-calibration long before the full label matures. Input drift monitors would stay silent throughout, because $P(X)$ is unchanged, and that silence is itself the signature of concept drift.

Solution to Exercise 11.8. **(a)** Nairobi: $1{,}820/2{,}000 = \mathbf{0.910}$. North Eastern: $95/120 = \mathbf{0.792}$. The gap is **0.118**, or 11.8 percentage points.

(b) With $z = 1.96$, the Wilson intervals are

$$\text{Nairobi: } [\mathbf{0.897},\ \mathbf{0.922}], \qquad \text{North Eastern: } [\mathbf{0.711},\ \mathbf{0.855}].$$

The second is nearly six times wider, entirely because it rests on 120 borrowers rather than 2,000.

(c) The gaps consistent with those intervals run from $0.897 - 0.855 = \mathbf{0.042}$ to $0.922 - 0.711 = \mathbf{0.211}$. A policy threshold of $\tau = 0.10$ sits inside that range, so a rule based on point estimates decides this case largely by sampling noise: the same underlying population could produce a comfortable pass or a clear fail next week.

Which rule to adopt follows from who bears each error. A *false pass* leaves a model in production that under-serves creditworthy borrowers in a region that is already under-banked; the harm falls on borrowers, accumulates for as long as the model runs, and is invisible in the aggregate approval rate. A *false fail* delays a model release; the cost falls on the lender and is measured in weeks. The asymmetry argues for a two-tier rule rather than a single test. *Block* promotion whenever the point estimate exceeds τ, as here, treating the wide interval as a reason for scrutiny and not as an excuse. *Investigate* before granting any exception: is the region's data coverage thin at training time, are its features systematically stale, is the gap explained by a legitimate risk difference? And *require* a named risk officer to sign any exception, with the slice report attached. Using the optimistic end of the interval, by contrast, would let small and poorly served groups

pass by default, which inverts the purpose of the gate, since those are exactly the groups the gate exists to protect. The complementary engineering action is to widen the window for small slices, monitoring North Eastern monthly rather than weekly so that the interval narrows, while keeping the alarm threshold unchanged.

Solution to Exercise 11.9. **(a)** *Cause*: a data-pipeline change, almost certainly a unit or schema change upstream, not drift. Distribution shift in the world does not move every feature by an order of magnitude overnight, and a simultaneous jump in the out-of-vocabulary share points at a producer that has changed its encoding. *Diagnostic*: compare a handful of raw records from before and after the change against the data contract of Chapter 10, and check the upstream team's deployment log for the same timestamp. *Remedy*: fix the producer or the ingestion mapping, backfill the affected window, and re-run the drift check. Do not retrain: retraining would encode the bug in the weights.

(b) *Cause*: concept drift. Stable inputs and a stable score distribution with degraded performance on matured labels is the signature of $P(Y \mid X)$ having moved, and it is the one case the label-free monitors cannot see (Table 11.1). *Diagnostic*: check calibration on the matured cohort, since concept drift usually shows as predicted probabilities that no longer match realised default rates; then break the degradation down by segment and by vintage to find when and where the relationship changed. *Remedy*: trigger continuous training on recent labelled data, and register the result as `@challenger` for the usual gates. Consider whether a feature that used to carry the signal has lost it, which would call for feature work rather than a straight retrain.

(c) *Cause*: benign covariate shift. Inputs moving gradually with a falling approval rate and a flat estimated cost is what Equation (11.9) looks like when the model is still calibrated: the population has genuinely become riskier, the model is correctly declining more of it, and the expected cost per applicant is unchanged because the extra declines are justified. *Diagnostic*: confirm calibration on the most recent matured cohort, and check coverage, that is, whether the new applicants fall in a region where training data was thin. *Remedy*: record the shift, inform the business that volumes will fall, and watch. Retrain only if coverage is poor or if calibration degrades later.

(d) *Cause*: recurring drift, the school-fee cycle of Figure 11.10. September fees depress inflows and raise the model's risk estimates for a few weeks. *Diagnostic*: compare the window with the same month last year rather than with August, which is the correct reference for a seasonal effect. *Remedy*: no model change. The right fix is in the monitoring configuration and possibly the model: add a school-fee-month feature so the model expects the pattern, and switch the seasonal features to a year-on-year reference so that the detector stops firing. An alert that fires every September and is dismissed every September is training the team to dismiss alerts.

Solution to Exercise 11.10. This is a design question, so the marks are for the coherence of the plan rather than for any particular threshold. A strong answer resembles the following.

(a) A plausible plan, one line per quantity.

- *Operational*: p_{99} scoring latency against the 150 ms budget of Chapter 10, and the 5xx error rate against 1%, both over a ten-minute window, owned by the on-call engineer.
- *Data quality*: per-batch null rate and out-of-vocabulary share for `region`, against a 5% threshold, referenced to the ingestion contract, owned by data engineering.
- *Model outputs*: PSI of the predicted probability against the frozen validation window, threshold 0.10; approval rate against a 30-day trailing baseline, threshold 5 percentage points; drift share across the input features, threshold 0.3; all owned by the model owner in risk analytics.
- *Performance*: AUC and cost per applicant on each cohort as its labels mature, against the values recorded for the champion at promotion, plus calibration on the same cohort; owned jointly by the model owner and the risk committee.

Every line names a reference, a statistic, a threshold and an owner, which is the minimum for a rule to be actionable.

(b) Only the drift share and the score PSI should trigger retraining automatically, and even then the trigger produces a `@challenger` that must pass every gate and the risk officer's approval, as in Listing 11.6. They are the right pair because they are label-free, computed on a full week of data rather than on a spike, and they measure a change in what the model is being asked to decide. Everything else opens a ticket. Latency and errors have nothing to do with the model's weights, so retraining cannot fix them. Data-quality failures are usually upstream bugs, and retraining on bad data is the specific harm that the first triage question exists to prevent. Performance on matured labels is the most decisive signal but also the slowest, and by the time it fires the right response is an investigation of *why*, not a reflex retrain.

(c) A four-rung ladder as in Figure 11.35: the champion model; on validation failure or a red drift gate, the previous registered version via its registry alias; if no model loads, a rule-based scorecard maintained for the purpose and re-validated quarterly; and finally referral to a human officer with an SMS promise of a decision. The bottom rung must **fail closed**: it must never approve a loan automatically, because an unavailable model is not evidence of creditworthiness. It must also be cheap enough to absorb a day of full traffic, and it must be exercised in CI like any other code path, since a fallback that has never run is a fallback that will not run.

(d) One defensible omission is per-feature drift alerting on the twenty lowest-importance features. They are monitored and reported weekly, but they do not raise alerts, because their movement rarely changes a decision and, as Exercise 11.4 shows, alerting on all of them guarantees a false alarm most nights. A second defensible omission is any detector on a feature that the service does not actually use in the current version; the temptation to monitor everything available should be resisted, since the cost of a monitor is not its computation but the attention it consumes when it fires.

Bibliography

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.

Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.

Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.

Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*.

Bantilan, N. (2024). pandera documentation. https://pandera.readthedocs.io.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43.

Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.

Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.

Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is "nearest neighbor" meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.

Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.

Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. *The World Bank Economic Review*, 34(3).

Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data*.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.

Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.

Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.

Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAccT)*, pages 77–91.

Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.

Central Bank of Kenya (2022). Central bank of kenya (digital credit providers) regulations, 2022.

Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*.

Chouldechova, A. (2017). Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data*, 5(2):153–163.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.

Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.

Dean, J. and Barroso, L. A. (2013). The tail at scale. *Communications of the ACM*, 56(2):74–80.

Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.

Docker Inc. (2024). Docker documentation. https://docs.docker.com.

Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.

Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the European Chapter of the ACL (EACL): System Demonstrations*.

Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.

Evidently AI (2025). Evidently documentation (version 0.7). https://docs.evidentlyai.com.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.

Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. O'Reilly Media, 2nd edition.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.

GitHub (2024). GitHub Actions documentation. https://docs.github.com/actions.

Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256.

Google Cloud (2020). MLOps: Continuous delivery and automation pipelines in machine learning. Cloud Architecture Center.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022a). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022b). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.

Guo, C. and Berkhahn, F. (2016). Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.

Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.

Hardt, M., Price, E., and Srebro, N. (2016). Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257.

Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.

Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, Melbourne, 3 edition.

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.

Iterative (2024). DVC: Data version control documentation. https://dvc.org/doc.

Jain, S. and Wallace, B. C. (2019). Attention is not explanation. In *Proceedings of NAACL-HLT*.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361.

Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Kreuzberger, D., Kühl, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.

Lakkaraju, H., Kleinberg, J., Leskovec, J., Ludwig, J., and Mullainathan, S. (2017). The selective labels problem: Evaluating algorithmic predictions in the presence of unobservables. In *Proceedings of KDD*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.

Lim, B., Arık, S. Ö., Loeff, N., and Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.

Lipton, Z. C., Wang, Y.-X., and Smola, A. (2018). Detecting and correcting for label shift with black box predictors. In *International Conference on Machine Learning (ICML)*.

Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.

Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.

Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.

McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.

Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.

Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*.

Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.

Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.

Rabanser, S., Günnemann, S., and Lipton, Z. C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ramírez, S. (2024). FastAPI documentation. https://fastapi.tiangolo.com.

Republic of Kenya (2019). The data protection act, no. 24 of 2019. Government Printer, Nairobi.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the ACL*.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.

Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.

Saerens, M., Latinne, P., and Decaestecker, C. (2002). Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1):21–41.

Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.

Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

Siddiqi, N. (2006). *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. John Wiley & Sons.

Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.

Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.

Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328.

Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

Tan, M. and Le, Q. V. (2021). Efficientnetv2: Smaller models and faster training. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.

van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.

Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.

Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.

Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.